

Manual Técnico del Participante

Microcredencial — Prueba Piloto

Ingeniería Inversa mediante Escaneo 3D y Reconstrucción CAD

Beca Santander – FIMPES Investigación

Dr. Jorge Ramón Parra Michel
Dr. Emmanuel Ovalle Magallanes
Dr. Marco Antonio Escobar Acevedo

Universidad La Salle Bajío

León, Guanajuato — 2026

Índice general

Información General	6
I Nivel Básico	12
1 Fundamentos de Escaneo 3D y Triangulación	13
1.1 ¿Qué es la ingeniería inversa óptica?	13
1.2 Principio de triangulación	14
1.2.1 Geometría del triángulo base	14
1.2.2 Sensibilidad a la profundidad	14
1.3 Proyección de luz estructurada	16
1.3.1 Componentes del sistema	16
1.3.2 Codificación de patrones	17
1.3.3 De la fase a la profundidad	19
1.3.4 Estrategia combinada: Gray + Phase Shifting	19
1.4 Fotogrametría de múltiples vistas	19
1.4.1 Pipeline SfM	19
1.4.2 Meshroom (AliceVision)	21
1.5 Modelo de cámara <i>pinhole</i> y distorsión de lente	22
1.5.1 Modelo proyectivo	22
1.5.2 Distorsión de lente	23
1.6 Comparación práctica: luz estructurada vs. fotogrametría	23
1.7 Configuración del entorno de trabajo	25
1.7.1 Open3D (Python)	25
1.7.2 CloudCompare	26
1.7.3 MeshLab	26
1.7.4 Meshroom	26
1.7.5 Estructura de directorios del proyecto	28
2 Captura 3D: Planificación y Adquisición	29
2.1 Preparación del objeto	29
2.1.1 Limpieza y acondicionamiento superficial	29
2.1.2 Agente antirreflejante (spray revelador)	29
2.1.3 Marcadores fiduciales y de referencia	31
2.1.4 Selección de la pieza de prueba	32

2.2	Captura por luz estructurada	33
2.2.1	Configuración del sistema óptico	33
2.2.2	Secuencia de patrones y adquisición	34
2.2.3	Cobertura y planificación de vistas múltiples	36
2.3	Captura por fotogrametría (Meshroom)	37
2.3.1	Configuración de la cámara	37
2.3.2	Planificación de la trayectoria de captura	39
2.3.3	Marcadores codificados y barra de escala	41
2.3.4	Ejecución del pipeline SfM en Meshroom	41
2.4	Criterios de aceptación de la nube de puntos	42
2.5	Inspección y visualización de la nube de puntos	44
2.5.1	Vía A: Python con Open3D	44
2.5.2	Vía B: MeshLab (sin programación)	45
3	Preprocesamiento, Filtrado y Registro ICP Multivista	47
3.1	Filtrado estadístico de <i>outliers</i>	47
3.1.1	Filtrado por eliminación estadística (SOR)	47
3.1.2	Eliminación de piezas aisladas por diámetro	48
3.2	Estimación de normales y submuestreo	49
3.2.1	Submuestreo por vóxeles (<i>voxel grid downsampling</i>)	49
3.2.2	Estimación de normales	51
3.3	Algoritmo ICP: formulación y convergencia	52
3.3.1	Variante <i>point-to-point</i>	52
3.3.2	Variante <i>point-to-plane</i>	52
3.3.3	Esquema iterativo	53
3.3.4	Inicialización: el problema de los mínimos locales	53
3.4	Registro global de múltiples vistas	55
3.4.1	Encadenamiento de transformaciones	55
3.4.2	Cierre de bucle (<i>loop closure</i>)	55
3.4.3	Verificación del registro global	57
3.4.4	Fusión de nubes (<i>merging</i>)	57
3.5	Filtrado y registro: Vía A (Python con Open3D)	57
3.6	Filtrado y registro: Vía B (MeshLab, sin programación)	61
3.6.1	Paso 1: Importar las nubes parciales	61
3.6.2	Paso 2: Filtrado de <i>outliers</i>	61
3.6.3	Paso 3: Estimación de normales	61
3.6.4	Paso 4: Submuestreo	62
3.6.5	Paso 5: Registro ICP par-a-par	62
3.6.6	Paso 6: Fusionar capas	62
3.6.7	Resumen comparativo: Vía A vs. Vía B	65
4	Mallado, Reconstrucción de Poisson y Exportación	66
4.1	Reconstrucción de superficies de Poisson	66
4.1.1	Formulación variacional	66
4.1.2	Parámetros clave	67

4.2	Algoritmos alternativos: Ball-Pivoting y Marching Cubes	68
4.2.1	Ball-Pivoting Algorithm (BPA)	68
4.2.2	Marching Cubes	69
4.3	Validación topológica de la malla	70
4.3.1	Verificación y reparación	70
4.4	Suavizado y simplificación	72
4.4.1	Suavizado Laplaciano	72
4.4.2	Decimación (<i>Quadric Edge Collapse</i>)	73
4.5	Exportación en formatos neutros	73
4.6	Mallado y exportación: Vía A (Python con Open3D/Trimesh)	75
4.7	Mallado y exportación: Vía B (MeshLab, sin programación)	76
4.7.1	Paso 1: Reconstrucción de Poisson	76
4.7.2	Paso 2: Reconstrucción alternativa — Ball-Pivoting	77
4.7.3	Paso 3: Validación topológica	77
4.7.4	Paso 4: Suavizado	77
4.7.5	Paso 5: Decimación	78
4.7.6	Paso 6: Exportación	78
II	Nivel Intermedio	81
5	Calibración Avanzada y Registro Multisesión	82
5.1	Calibración intrínseca con patrón de tablero	82
5.1.1	El método de Zhang (2000)	82
5.1.2	Preparación del patrón y captura	83
5.2	Calibración extrínseca y estéreo	84
5.2.1	Par estéreo para luz estructurada	84
5.2.2	Par estéreo para fotogrametría	85
5.2.3	Significado físico de los parámetros extrínsecos	85
5.3	Criterios de aceptación de la calibración	85
5.4	Registro multisesión	88
5.4.1	Estrategia con targets fijos	88
5.5	Extracción de primitivas geométricas	89
5.5.1	Ajuste por mínimos cuadrados	90
5.5.2	Detección automática con RANSAC	90
5.5.3	Métricas de calidad del ajuste	92
5.6	Código Python: calibración con OpenCV	92
5.7	Extracción de primitivas: Vía A (Python) / Vía B (CloudCompare)	94
5.7.1	Vía A: Python con Open3D	94
5.7.2	Vía B: CloudCompare (interfaz gráfica)	95
6	Superficies NURBS y Comparación CAD-Scan	99
6.1	Fundamentos de curvas y superficies NURBS	99
6.1.1	Curvas B-spline	99
6.1.2	Superficies NURBS (producto tensorial)	100

6.1.3	Continuidad geométrica	100
6.2	Ajuste de superficies NURBS sobre la malla	101
6.2.1	Segmentación de la malla en regiones	101
6.2.2	Ajuste de cada parche	101
6.2.3	Ensamble con continuidad G^1	102
6.3	Flujo de trabajo en ANSYS SpaceClaim: scan $\rightarrow$ NURBS $\rightarrow$ STEP	102
6.4	Comparación CAD-scan con mapas de desviación	103
6.4.1	Comparación en ANSYS SpaceClaim	103
6.4.2	Comparación alternativa en CloudCompare	105
6.4.3	Histograma de desviaciones	105
6.5	Interpretación cuantitativa de resultados	105
6.5.1	Causas de desviaciones elevadas	108
6.6	Informe de conformidad dimensional	108
6.6.1	Incertidumbre expandida	109
Bibliografía		113

Información General

Nombre	Ingeniería Inversa mediante Escaneo 3D y Reconstrucción CAD
Tipo	Formación continua
Modalidad	Mixta (aprendizaje combinado)
Duración total	30 horas. Para la prueba piloto internacional se implementa una versión intensiva de 15 horas.
Distribución	70% práctica / 30% fundamento teórico
Disponibilidad	Validación de perfil
Institución	Universidad La Salle Bajío
Vinculación	Beca Santander–FIMPES; UniLaSalle Amiens (Francia)
Responsable	Dr. Jorge Ramón Parra Michel

Objetivo General

Formar participantes capaces de aplicar técnicas de ingeniería inversa mediante escaneo 3D, fotogrametría multivista, procesamiento de nubes de puntos, reconstrucción geométrica y comparación dimensional CAD–scan, con criterios de trazabilidad, reproducibilidad y validación metrológica.

Competencia a Desarrollar

Aplicar metodologías de adquisición 3D, calibración, registro, filtrado, mallado, extracción de primitivas, reconstrucción CAD/NURBS y comparación CAD–scan para generar modelos digitales técnicamente verificables a partir de objetos físicos, documentando errores, desviaciones dimensionales y criterios de aceptación.

Objetivos Específicos

1. Planificar y ejecutar procesos de captura 3D mediante luz estructurada y fotogrametría multivista, considerando cobertura, iluminación, textura, escala y estabilidad geométrica.
2. Procesar nubes de puntos mediante filtrado, estimación de normales, registro multivista, fusión y reconstrucción de mallas.

3. Extraer primitivas geométricas, reconstruir modelos CAD editables y comparar el modelo reconstruido contra el escaneo mediante mapas de desviación dimensional.
4. Documentar el proceso de ingeniería inversa mediante reportes técnicos con criterios cuantitativos de calidad, trazabilidad y conformidad dimensional.

Conocimientos Previos Requeridos

- Geometría analítica y álgebra lineal básica.
- Dibujo técnico, interpretación de planos y fundamentos de tolerancias dimensionales.
- Conocimientos básicos de CAD mecánico.
- Óptica geométrica elemental: formación de imagen, cámara, lente y distorsión.
- Programación básica en Python, deseable pero no indispensable.
- Familiaridad básica con procesos de manufactura, inspección dimensional o diseño mecánico.

Estructura de Niveles y Distribución Horaria

La microcredencial completa se organiza en dos niveles acumulativos con seis temas distribuidos en **30 horas totales**. Esta duración corresponde a la versión regular para educación continua, implementación institucional y adopción posterior en contextos industriales.

Para la prueba piloto internacional en UniLaSalle Amiens se implementa una **versión intensiva de 15 horas**. Esta versión conserva la misma secuencia temática, los mismos resultados de aprendizaje y las mismas evidencias técnicas, pero reduce el tiempo destinado a repetición experimental, comparación entre métodos, práctica extendida en software y elaboración detallada del reporte CAD-scan. La versión piloto no sustituye la estructura curricular completa; opera como validación inicial de transferencia metodológica, pertinencia técnica y aplicabilidad de los materiales.

Tema	Título	Horas regulares	Horas piloto	Nivel
1	Fundamentos de escaneo 3D y triangulación	4	2	Básico
2	Captura 3D: planificación y adquisición	6	3	Básico
3	Preprocesamiento, filtrado y registro ICP multivista	6	3	Básico
4	Mallado, reconstrucción de Poisson y exportación	4	2	Básico
5	Calibración avanzada, registro multisesión y extracción de primitivas	6	3	Intermedio
6	Reconstrucción CAD/NURBS y comparación CAD-scan	4	2	Intermedio
Total		30	15	

Configuraciones y Flujo Técnico

La microcredencial se estructura en torno a un flujo progresivo de ingeniería inversa óptica. En el Nivel Básico se aborda la adquisición 3D mediante luz estructurada y fotogrametría multivista, seguida del procesamiento de nubes de puntos, registro ICP, fusión y mallado. En el Nivel Intermedio se incorporan calibración avanzada, extracción de primitivas, reconstrucción CAD/NURBS y comparación dimensional CAD-scan.

Objeto físico → Captura 3D → Nube de puntos → Registro y fusión
→ Malla → CAD → Validación dimensional (1)

Enfoque Pedagógico

La formación se estructura como un **pipeline técnico reproducible**. Cada tema produce una salida verificable que sirve como entrada para el siguiente: imágenes de captura, nube de puntos, nube filtrada, nube registrada, malla reconstruida, primitivas geométricas, modelo CAD y reporte de desviación dimensional.

El 70% del tiempo formativo, tanto en la versión completa como en la versión piloto, se dedica a ejecución práctica: captura de datos, procesamiento computacional, registro de nubes, reconstrucción geométrica, comparación dimensional y documentación técnica. El 30% restante corresponde al fundamento teórico mínimo habilitante, introducido únicamente cuando permite ejecutar una acción técnica inmediata.

En la versión completa de 30 horas se dispone de mayor tiempo para repetir capturas, comparar luz estructurada contra fotogrametría, analizar sensibilidad de parámetros, depurar nubes de puntos, contrastar métodos de mallado, practicar reconstrucción CAD/NURBS y elaborar un reporte CAD-scan completo. En la versión piloto de 15 horas

se conserva el flujo completo del pipeline, pero se compactan las actividades de práctica extendida, comparación metodológica y documentación final.

Software Utilizado

- **Meshroom:** reconstrucción fotogramétrica mediante Structure from Motion (SfM) y Multi-View Stereo (MVS).
- **OpenCV:** calibración de cámara, corrección de distorsión y procesamiento de imágenes.
- **Open3D:** procesamiento de nubes de puntos, filtrado, registro ICP, estimación de normales y visualización 3D.
- **MeshLab:** limpieza, mallado, reconstrucción de Poisson, suavizado y exportación de mallas.
- **CloudCompare:** inspección de nubes, registro, comparación dimensional y mapas de desviación.
- **ANSYS SpaceClaim / FreeCAD:** reconstrucción CAD, superficies editables, primitivas geométricas y preparación de modelos.
- **Python** (NumPy, SciPy, Matplotlib, Open3D): automatización, análisis, visualización y generación de reportes.

Evidencia Final de la Microcredencial

La evidencia integradora consiste en un **archivo CAD reconstruido a partir de un objeto físico** y un **reporte técnico de comparación CAD–scan**. El reporte debe documentar el proceso completo de ingeniería inversa, desde la preparación de la pieza y la adquisición de datos 3D hasta la reconstrucción geométrica, la comparación dimensional y la conclusión de conformidad.

La evidencia no se limita a presentar una nube de puntos o una malla visualmente aceptable, porque eso sería confundir “se ve bonito” con “sirve para ingeniería”, una tragedia bastante común. El entregable debe demostrar que el participante puede generar un modelo digital técnicamente verificable, trazable y reproducible, con criterios cuantitativos de calidad geométrica.

Componentes mínimos del reporte

El reporte final deberá incluir, como mínimo, los siguientes elementos:

- Descripción de la pieza física analizada, incluyendo material, geometría general, dimensiones aproximadas y función mecánica o industrial.

- Descripción del método de captura utilizado: luz estructurada, fotogrametría multi-vista o combinación de ambas técnicas.
- Registro de las condiciones de adquisición: iluminación, distancia de trabajo, número de vistas, solapamiento entre imágenes, tipo de cámara, lente, resolución y uso de marcadores o barra de escala.
- Procedimiento de preparación superficial de la pieza, incluyendo limpieza, aplicación de spray antirreflejante, marcadores fiduciales y referencias dimensionales.
- Parámetros de calibración de cámara o sistema de captura, incluyendo matriz intrínseca, coeficientes de distorsión, escala y error de reproyección.
- Descripción de la nube de puntos obtenida, incluyendo densidad, cobertura, ruido, huecos, oclusiones y zonas descartadas.
- Procedimiento de preprocesamiento: filtrado de outliers, estimación de normales, submuestreo, limpieza y segmentación.
- Procedimiento de registro multivista, incluyendo método empleado, RMS de registro, número de vistas integradas y verificación de alineación global.
- Procedimiento de mallado, incluyendo método utilizado, parámetros principales, reparación topológica, suavizado y simplificación.
- Extracción de primitivas geométricas: planos, cilindros, ejes, agujeros, radios, superficies de referencia o entidades funcionales.
- Reconstrucción CAD/NURBS, incluyendo estrategia de generación de superficies, continuidad geométrica, edición de parches y exportación del modelo.
- Comparación CAD–scan mediante mapa de desviación dimensional, histograma de errores, desviación media, RMS, desviación máxima y percentil p_{95} .
- Discusión de fuentes de error: calibración, textura superficial, iluminación, oclusiones, densidad de nube, registro, mallado, suavizado y ajuste de superficies.
- Conclusión técnica sobre la conformidad geométrica del modelo reconstruido y su posible uso para rediseño, inspección, documentación o manufactura.

Criterios mínimos de aceptación

Para acreditar la evidencia final, el archivo CAD reconstruido y el reporte CAD–scan deberán cumplir los siguientes criterios cuantitativos:

- Cobertura mínima de captura ≥ 90 % de la superficie relevante de la pieza.
- Densidad de nube suficiente para representar las entidades funcionales de la pieza, preferentemente > 1000 pts/cm² en zonas críticas.

- Error de reproyección < 0.75 px en la calibración de cámara o sistema de captura.
- RMS de registro ICP ≤ 0.30 mm para registros parciales y RMS global ≤ 0.50 mm.
- Ajuste de primitivas geométricas con $R^2 > 0.95$ y RMS ≤ 0.10 mm en entidades funcionales.
- Desviación CAD–scan dentro del criterio definido para la pieza, preferentemente $p_{95} \leq 0.25$ mm o desviación global $\leq \pm 0.30$ mm cuando aplique.
- Continuidad geométrica mínima G^1 en uniones de superficies NURBS o parches CAD relevantes.
- Documentación completa de parámetros, archivos fuente, nube procesada, malla, modelo CAD y mapa de desviación.

Producto entregable

El entregable final estará constituido por:

1. Reporte técnico en formato PDF.
2. Carpeta de imágenes originales, capturas 3D o archivos de escaneo.
3. Archivo de calibración y parámetros de adquisición.
4. Nube de puntos cruda y nube de puntos procesada en formato PLY, XYZ, E57 u otro formato compatible.
5. Malla reconstruida en formato STL, OBJ o PLY.
6. Modelo CAD reconstruido en formato STEP, IGES, SLDPRT, F3D o formato equivalente.
7. Reporte CAD–scan con mapa de desviación dimensional.
8. Tabla de primitivas geométricas ajustadas y sus errores asociados.
9. Bitácora de procesamiento con software utilizado, parámetros principales y decisiones técnicas.
10. Scripts, archivos de proyecto o flujo de trabajo que permitan reproducir el análisis.

La acreditación de la microcredencial se otorgará únicamente cuando el participante demuestre que el modelo CAD reconstruido es geoméricamente consistente, dimensionalmente verificable y reproducible a partir de la documentación entregada.

Parte I

Nivel Básico

Capítulo 1

Fundamentos de Escaneo 3D y Triangulación

Duración: 4 h

Modalidad: Presencial

Este capítulo introduce los principios ópticos y geométricos que sustentan las dos técnicas de adquisición 3D empleadas en la microcredencial: **proyección de luz estructurada** y **fotogrametría multivista** (*Structure from Motion*, SfM). Al finalizar, el participante podrá distinguir las capacidades y limitaciones de cada técnica, manejar el modelo de cámara *pinhole* con distorsión, y verificar la instalación del software de trabajo.

1.1 ¿Qué es la ingeniería inversa óptica?

La **ingeniería inversa** (*reverse engineering*) consiste en obtener la representación geométrica digital de un objeto físico existente, sin disponer de su modelo CAD original. En un contexto industrial esto permite:

- Reproducir componentes discontinuados o sin documentación técnica.
- Certificar piezas fabricadas contra tolerancias dimensionales.
- Generar modelos CAD para rediseño, manufactura aditiva o simulación FEA.
- Documentar patrimonio cultural, fósiles o piezas únicas.

El flujo general de trabajo se resume en seis etapas:

Objeto → Captura 3D → Nube de puntos → Malla → Modelo CAD → Validación

(1.1)

Analogía: del molde de yeso al escáner 3D

Antes de la digitalización óptica, la ingeniería inversa se hacía con moldes de yeso, brazos articulados con palpadores (*CMM*) o plantillas de contorno. Estas técnicas son lentas, requieren contacto físico (pueden dañar el objeto) y generan pocos puntos. Un escáner de luz estructurada captura $\sim 10^6$ puntos en segundos sin tocar la pieza; la fotogrametría logra lo mismo con una cámara y software libre.

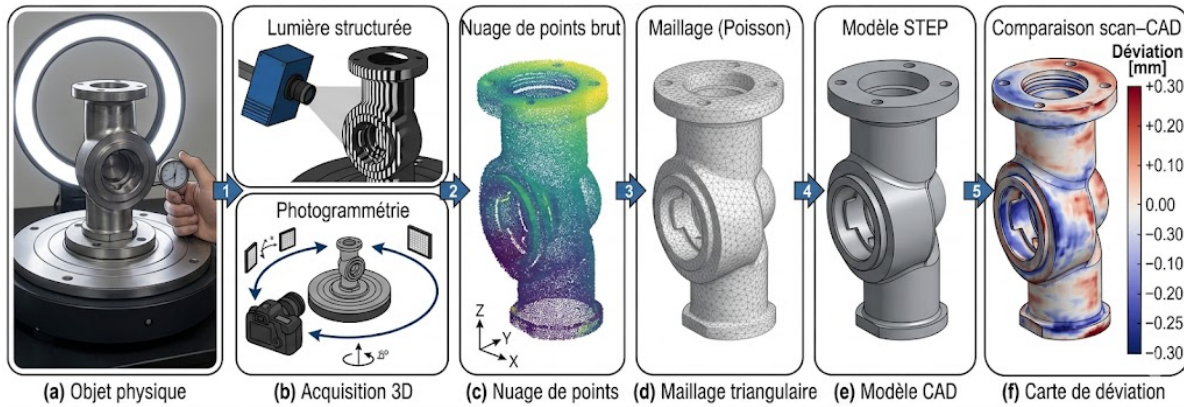


Figura 1.1: Pipeline de ingeniería inversa óptica. **(a)** Objeto físico (pieza mecánica de geometría mixta). **(b)** Captura 3D (escáner de luz estructurada o cámara para fotogrametría). **(c)** Nube de puntos cruda en falso color (escala de profundidad). **(d)** Malla triangular tras reconstrucción de Poisson. **(e)** Modelo CAD con superficies NURBS. **(f)** Mapa de desviación cromático (comparación CAD-scan). Flechas numeradas entre cada etapa.

1.2 Principio de triangulación

Toda medición 3D óptica se fundamenta en la **triangulación**: conocer la posición de un punto en el espacio a partir de la intersección de dos líneas de visión cuya geometría relativa es conocida.

1.2.1 Geometría del triángulo base

Considérese un sistema formado por una fuente de iluminación (o una segunda cámara) y un detector (cámara), separados una distancia conocida B (*baseline*). Un punto P en el objeto genera:

- Una línea de proyección desde la fuente hacia P .
- Una línea de observación desde el detector hacia P .

La profundidad Z del punto se obtiene por semejanza de triángulos:

$$Z = \frac{f \cdot B}{d} \quad (1.2)$$

donde:

- f es la distancia focal del detector [mm],
- B es la separación fuente-detector (*baseline*) [mm],
- d es la disparidad medida en el plano del sensor [mm o px].

1.2.2 Sensibilidad a la profundidad

Diferenciando la Ecuación (1.2) respecto a d :

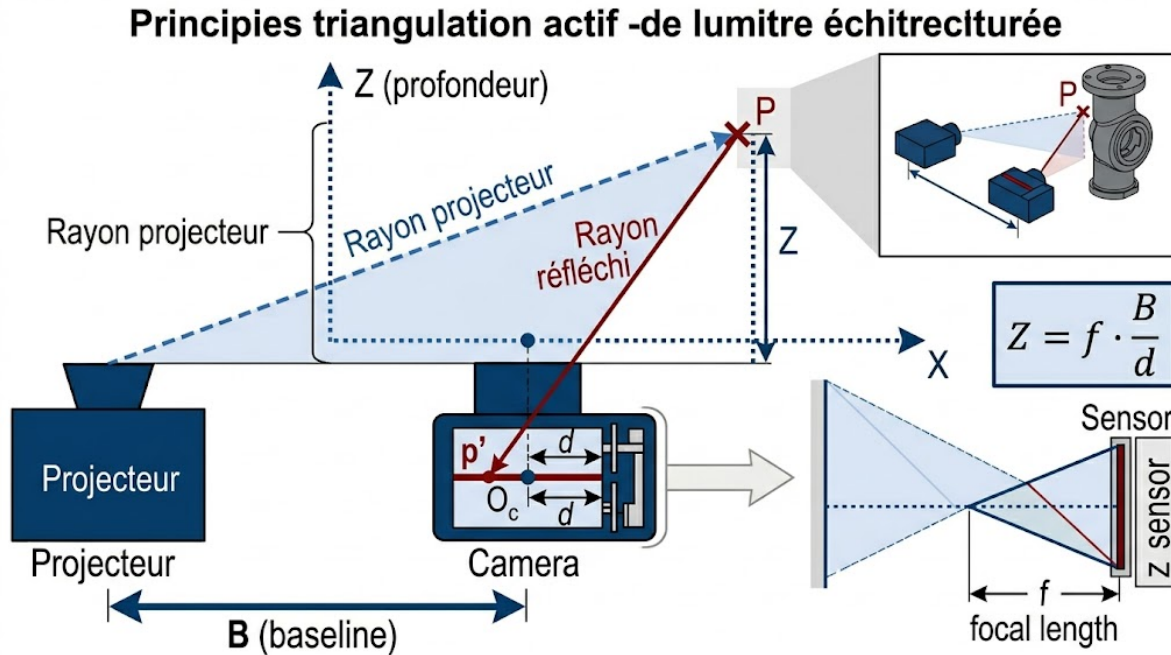


Figura 1.2: Geometría de la triangulación activa. Un proyector (izquierda) y una cámara (derecha) están separados por la *baseline* B . El punto P del objeto se observa con disparidad d en el sensor. La profundidad $Z = fB/d$ resulta de la semejanza de triángulos $\triangle(\text{Proyector}-P-\text{Cámara})$ y su proyección en el plano imagen.

$$\delta Z = -\frac{f \cdot B}{d^2} \delta d = -\frac{Z^2}{f \cdot B} \delta d \quad (1.3)$$

Esto implica que:

1. La incertidumbre en Z crece con el **cuadrado** de la distancia de trabajo.
2. Una *baseline* B mayor mejora la resolución en profundidad, pero aumenta las oclusiones (sombras).
3. La resolución del sensor (δd mínimo detectable) es el factor limitante fundamental.

Ejemplo numérico: resolución en profundidad

Para un sistema con $f = 12$ mm, $B = 200$ mm, distancia de trabajo $Z = 500$ mm y resolución de disparidad $\delta d = 0.1$ px (≈ 0.00035 mm con píxel de $3.5 \mu\text{m}$):

$$\delta Z = \frac{500^2}{12 \times 200} \times 0.00035 = \frac{250\,000}{2\,400} \times 0.00035 \approx 0.036 \text{ mm}$$

Es decir, $\sim 36 \mu\text{m}$ de resolución en profundidad — adecuado para aplicaciones de ingeniería inversa mecánica (tolerancia típica ± 0.10 – 0.30 mm).

Advertencia: oclusiones por baseline excesiva

Una *baseline* grande mejora δZ , pero genera **zonas de sombra** (oclusiones) en cavidades y ángulos pronunciados: el proyector ilumina regiones que la cámara no ve, o viceversa. En la práctica, el ángulo de triangulación $\theta = 2 \arctan(B/2Z)$ se mantiene entre 15° y 30° como compromiso.

1.3 Proyección de luz estructurada

Un sistema de **luz estructurada** (*Structured Light*, SL) proyecta un patrón conocido sobre el objeto y analiza cómo la geometría superficial deforma dicho patrón. La cámara captura la deformación y, conociendo la geometría proyector–cámara, reconstruye la profundidad punto a punto.

1.3.1 Componentes del sistema

Un sistema SL mínimo consta de:

1. **Proyector:** DLP o LCD, resolución $\geq 800 \times 600$ px, capaz de proyectar secuencias de patrones a ≥ 30 fps.
2. **Cámara:** sensor monocromático o color, ≥ 2 MP, con control manual de exposición.
3. **Soporte rígido:** barra o perfil que fija la distancia proyector–cámara (*baseline* B).

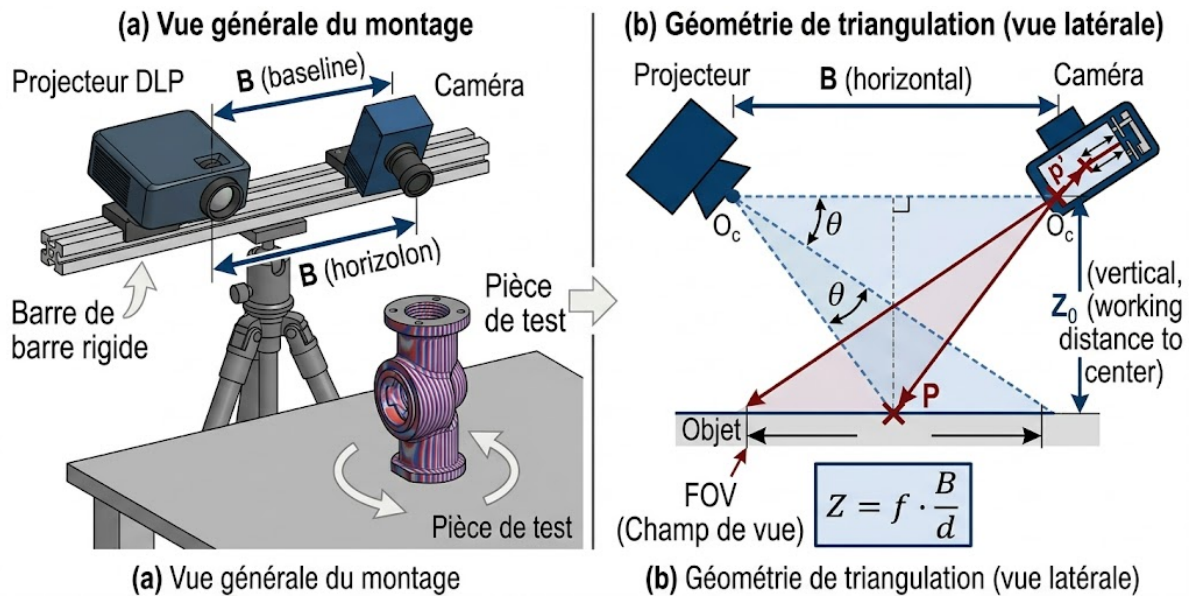


Figura 1.3: Configuración básica de un sistema de luz estructurada. **(a)** Vista general: proyector y cámara montados sobre barra rígida, con pieza de prueba sobre mesa. **(b)** Vista lateral esquemática: triángulo de triangulación proyector–objeto–cámara con ángulo θ . Anotaciones: B , θ , distancia de trabajo Z_0 , campo de visión (FOV).

1.3.2 Codificación de patrones

La profundidad se obtiene identificando la **correspondencia** entre cada píxel del proyector y cada píxel de la cámara. Para ello, se proyectan secuencias de patrones que codifican posiciones. Los dos esquemas principales son:

Código Gray (binario)

Se proyectan N patrones de franjas binarias (blanco/negro) con frecuencias $2^0, 2^1, \dots, 2^{N-1}$. Cada franja vertical recibe un código binario único de N bits, lo que permite identificar 2^N columnas del proyector.

$$N_{\text{patrones}} = \lceil \log_2(W_{\text{proj}}) \rceil \quad (1.4)$$

donde W_{proj} es el ancho del proyector en píxeles. Para un proyector de 1024 px: $N = 10$ patrones.

Ventaja del código Gray vs. binario natural

En un código binario natural, la transición de 0111 a 1000 cambia los 4 bits simultáneamente, lo que genera errores en la frontera. El **código Gray** garantiza que entre columnas adyacentes solo cambia **un bit**, reduciendo errores de decodificación en las transiciones.

Desplazamiento de fase (*phase shifting*)

Se proyectan $M \geq 3$ franjas sinusoidales desfasadas entre sí:

$$I_m(x, y) = a(x, y) + b(x, y) \cos[\phi(x, y) + \delta_m], \quad \delta_m = \frac{2\pi m}{M}, \quad m = 0, 1, \dots, M-1 \quad (1.5)$$

donde a es la iluminación de fondo, b es la modulación y $\phi(x, y)$ es la fase que codifica la geometría. Con M imágenes, la fase envuelta se calcula como:

$$\phi_w(x, y) = \arctan \left(\frac{\sum_{m=0}^{M-1} I_m \sin(\delta_m)}{\sum_{m=0}^{M-1} I_m \cos(\delta_m)} \right) \quad (1.6)$$

La fase resultante está **envuelta** en $(-\pi, \pi]$. Para obtener la fase absoluta $\Phi(x, y)$ se aplica un algoritmo de *phase unwrapping* (desenvolvimiento):

$$\Phi(x, y) = \phi_w(x, y) + 2\pi \cdot k(x, y) \quad (1.7)$$

donde $k(x, y) \in \mathbb{Z}$ es el orden de franja, determinado típicamente por los patrones Gray.

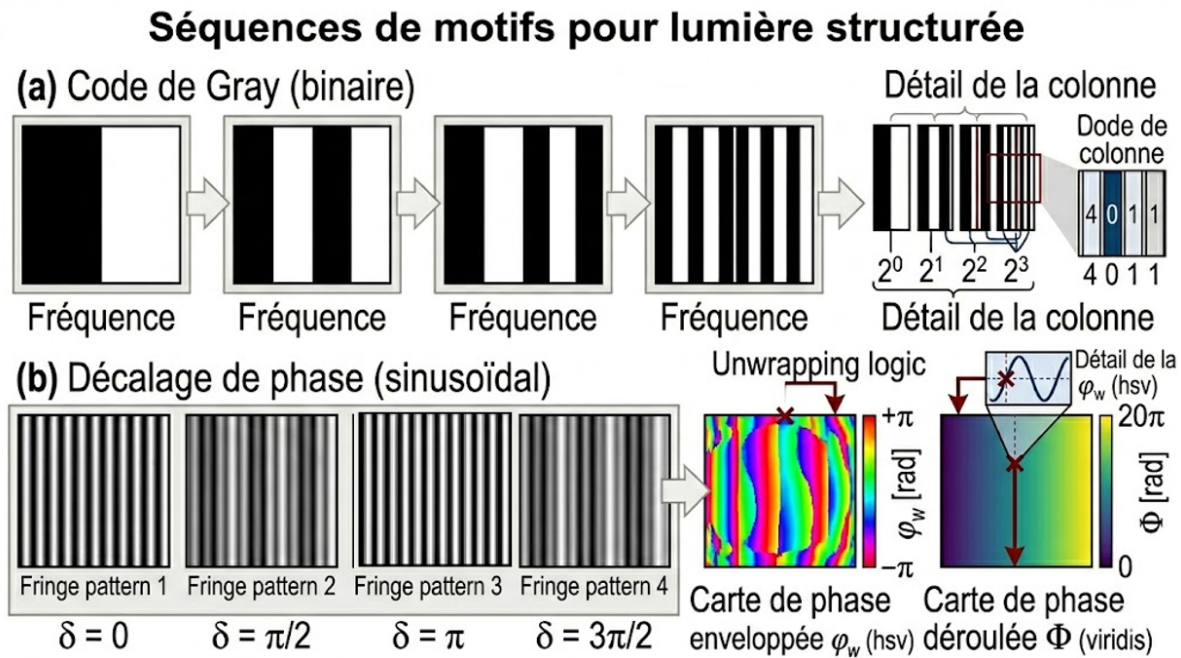


Figura 1.4: Secuencias de patrones para luz estructurada. **Fila superior:** código Gray — 4 patrones binarios con frecuencias crecientes (2^0 a 2^3), asignando un código de 4 bits a cada columna. **Fila inferior:** desplazamiento de fase — 4 patrones sinusoidales desfasados 0, $\pi/2$, π , $3\pi/2$. A la derecha, el mapa de fase envuelta ϕ_w resultante (colormap cíclico) y el mapa de fase desenvuelta Φ (colormap lineal).

1.3.3 De la fase a la profundidad

Una vez obtenida la fase absoluta $\Phi(x, y)$, la relación con la coordenada del proyector x_p es lineal:

$$x_p = \frac{\Phi(x, y)}{2\pi} \cdot T_p \quad (1.8)$$

donde T_p es el período del patrón en píxeles del proyector. Con la correspondencia $(x_c, y_c) \leftrightarrow x_p$ establecida, se aplica la ecuación de triangulación (Ec. (1.2)) para calcular $Z(x, y)$.

Ventajas y limitaciones de la luz estructurada

Ventajas:

- Captura de campo completo en pocos segundos (4–12 patrones).
- Alta resolución ($\delta Z \sim 10\text{--}100 \mu\text{m}$ según configuración).
- Funciona bien en superficies mate con textura limitada.

Limitaciones:

- Superficies especulares (metal pulido, vidrio) reflejan el patrón fuera del campo de la cámara.
- Oclusiones en cavidades profundas.
- Rango de profundidad limitado por la profundidad de campo del proyector.
- Sensible a iluminación ambiente (trabajar con luz controlada).

1.3.4 Estrategia combinada: Gray + Phase Shifting

La combinación más robusta utiliza **ambos esquemas**:

1. Los patrones Gray determinan el orden de franja $k(x, y)$ (Ec. (1.7)) — resolución gruesa.
2. Los patrones sinusoidales proporcionan la fase dentro de cada franja — resolución fina (subpíxel del proyector).

El número total de capturas es $N_{\text{Gray}} + M_{\text{PS}}$. Con un proyector de 1024 px y $M = 4$ fases: $10 + 4 = 14$ capturas, adquiridas en < 1 s a 30 fps.

1.4 Fotogrametría de múltiples vistas

La **fotogrametría multivista** reconstruye la geometría 3D a partir de múltiples fotografías tomadas desde diferentes ángulos, sin proyectar ningún patrón. El algoritmo central es **Structure from Motion** (SfM).

1.4.1 Pipeline SfM

El proceso se descompone en cuatro etapas:

1. **Detección de puntos clave** (*feature detection*): se identifican puntos de interés (esquinas, bordes) en cada imagen usando detectores como SIFT, SURF u ORB. Cada punto se describe con un vector de características (*descriptor*).
2. **Correspondencia entre vistas** (*feature matching*): se emparejan descriptores entre pares de imágenes. Se filtran correspondencias falsas con RANSAC sobre la matriz fundamental F .
3. **Estimación de pose y triangulación**: a partir de las correspondencias, se estiman las posiciones y orientaciones de cada cámara $[R_i|t_i]$ y se triangulan los puntos 3D. El resultado es una **nube de puntos dispersa** (*sparse point cloud*).
4. **Densificación** (*Multi-View Stereo, MVS*): se genera una **nube densa** comparando la textura entre imágenes vecinas, típicamente con algoritmos de parche (*patch-based MVS*).

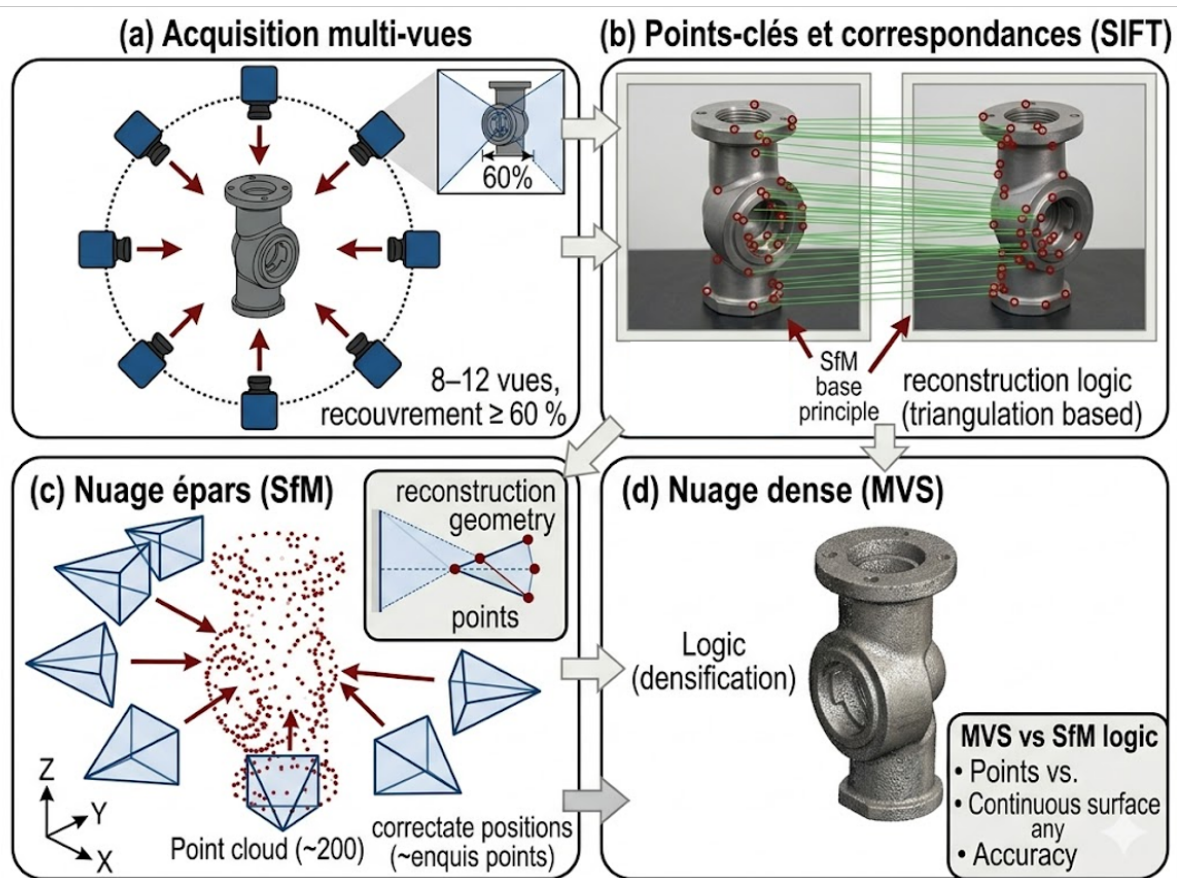


Figura 1.5: Pipeline de fotogrametría *Structure from Motion* (SfM). **(a)** Adquisición: fotografías desde múltiples ángulos alrededor del objeto. **(b)** Detección de puntos clave (SIFT) en dos imágenes, con correspondencias indicadas por líneas. **(c)** Nube de puntos dispersa con posiciones de cámara estimadas (pirámides azules). **(d)** Nube densa (MVS) coloreada por la textura del objeto.

1.4.2 Meshroom (AliceVision)

Meshroom es un software libre basado en el framework **AliceVision** que implementa el pipeline SfM + MVS completo mediante una interfaz gráfica de nodos (*node-based graph*).

El flujo estándar de nodos en Meshroom es:

CameraInit → FeatureExtraction → FeatureMatching → StructureFromMotion → DepthMap
→ Meshing → Texturing

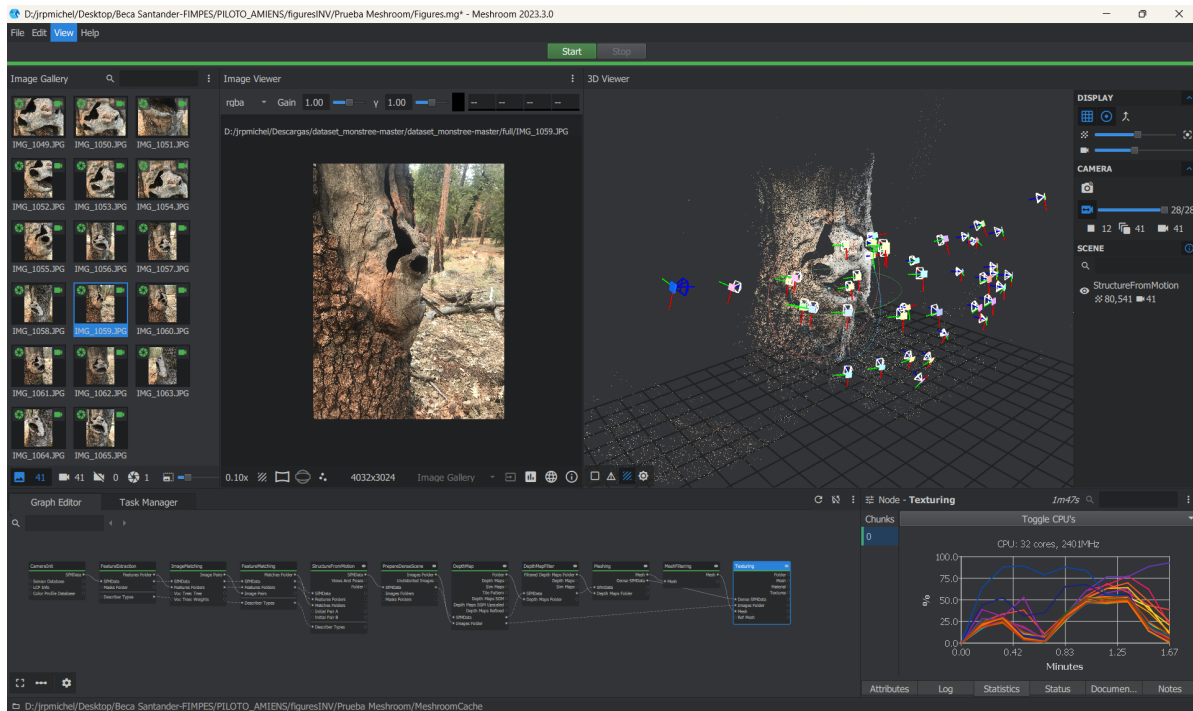


Figura 1.6: Interfaz de Meshroom mostrando el grafo de nodos del pipeline SfM/MVS estándar. Los nodos verdes indican etapas completadas; el panel derecho muestra la nube de puntos dispersa con las posiciones de cámara. *Source : images issues du site AliceVision (alice-vision.org), contenu sous licence CC BY-SA 4.0, sauf indication contraire.*

Ventajas y limitaciones de la fotogrametría

Ventajas:

- Equipo mínimo: una cámara DSLR o incluso un smartphone de buena calidad.
- Sin proyector: funciona en exteriores y con objetos grandes (edificios, terrenos).
- Software libre (Meshroom, COLMAP, OpenMVG).

Limitaciones:

- Requiere textura superficial suficiente (superficies uniformes fallan).
- Tiempo de procesamiento largo (minutos a horas según N imágenes).
- Menor resolución que SL para objetos pequeños ($\delta Z \sim 0.1\text{--}1\text{ mm}$).
- Escala absoluta requiere marcadores de referencia calibrados.

1.5 Modelo de cámara *pinhole* y distorsión de lente

Ambas técnicas (SL y fotogrametría) dependen del modelo de formación de imagen. El modelo fundamental es la **cámara *pinhole*** (cámara estenopeica).

1.5.1 Modelo proyectivo

Un punto 3D $\mathbf{P} = [X, Y, Z]^T$ se proyecta al plano imagen como:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{K}} \underbrace{\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}}_{\text{extrínsecos}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (1.9)$$

donde:

- $\mathbf{K}$ es la **matriz de intrínsecos**: f_x, f_y (distancias focales en píxeles), c_x, c_y (punto principal).
- $[\mathbf{R}|\mathbf{t}]$ son los **parámetros extrínsecos**: rotación y traslación del sistema de coordenadas del mundo al de la cámara.
- s es un factor de escala (profundidad proyectiva).

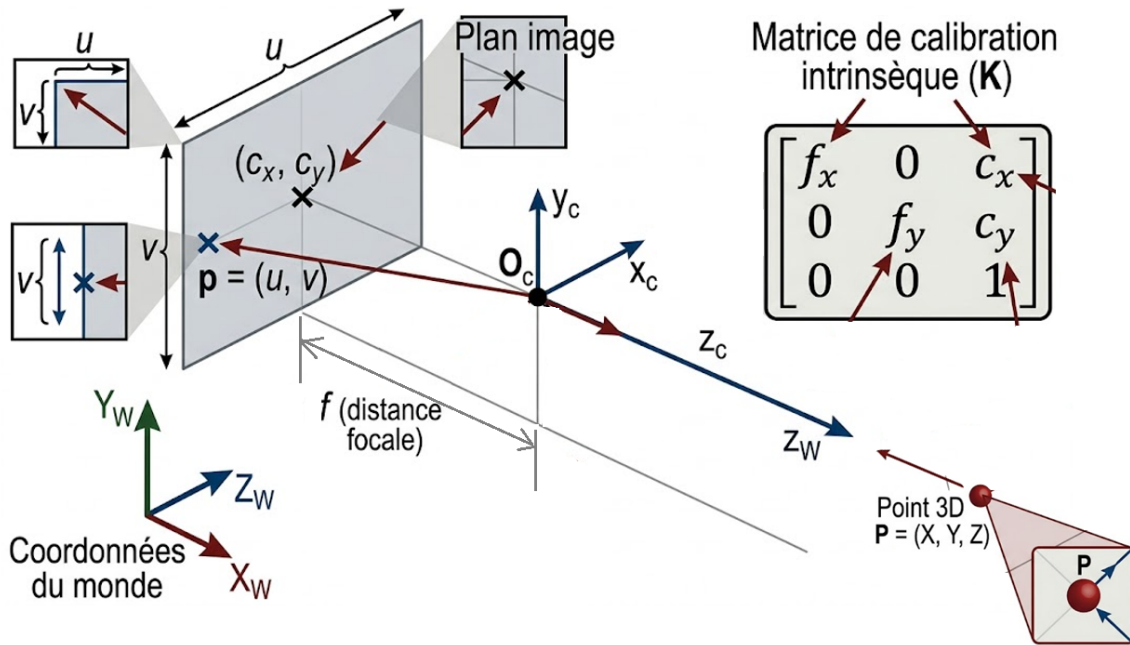


Figura 1.7: Modelo de cámara *pinhole*. Un punto 3D $\mathbf{P} = (X, Y, Z)$ se proyecta a través del centro óptico O al punto imagen (u, v) . Se muestran los ejes del sistema de coordenadas del mundo, de la cámara y del plano imagen, con las distancias focales f_x, f_y y el punto principal (c_x, c_y) .

1.5.2 Distorsión de lente

Las lentes reales introducen distorsiones que desvían los rayos del modelo *pinhole* ideal. Los dos tipos principales son:

Distorsión radial

$$\begin{aligned}x_d &= x_n(1 + k_1r^2 + k_2r^4 + k_3r^6) \\y_d &= y_n(1 + k_1r^2 + k_2r^4 + k_3r^6)\end{aligned}\tag{1.10}$$

donde (x_n, y_n) son las coordenadas normalizadas (sin distorsión), (x_d, y_d) las distorsionadas, y $r^2 = x_n^2 + y_n^2$.

- $k_1 > 0$: **distorsión de barril** — las líneas rectas se curvan hacia afuera (típica de gran angular).
- $k_1 < 0$: **distorsión de cojín** (*pincushion*) — las líneas se curvan hacia adentro (típica de teleobjetivo).

Distorsión tangencial

Causada por desalineación entre la lente y el sensor:

$$\begin{aligned}x_d &= x_n + 2p_1x_ny_n + p_2(r^2 + 2x_n^2) \\y_d &= y_n + p_1(r^2 + 2y_n^2) + 2p_2x_ny_n\end{aligned}\tag{1.11}$$

Advertencia: la corrección de distorsión es obligatoria

Tanto para luz estructurada como para fotogrametría, las imágenes **deben corregirse de distorsión** antes de cualquier cálculo de triangulación o correspondencia. Una distorsión de $k_1 = -0.1$ genera desviaciones de > 5 px en las esquinas de un sensor de 2048×1536 , lo que se traduce en errores de reconstrucción de varios milímetros. La calibración de cámara (Tema 5) proporciona los coeficientes necesarios.

1.6 Comparación práctica: luz estructurada vs. fotogrametría

La Tabla 1.1 resume las características operativas de ambas técnicas en el contexto de esta microcredencial.

Cuadro 1.1: Comparación de luz estructurada (SL) y fotogrametría multivista para ingeniería inversa de piezas mecánicas de tamaño mediano ($\sim 5\text{--}50\text{ cm}$).

Criterio	Luz estructurada (SL)	Fotogrametría (SfM + MVS)
Equipo mínimo	Proyector DLP + cámara industrial + barra rígida	Cámara DSLR (o smartphone de alta gama)
Costo aprox.	\$1 000–\$5 000 USD (sistema básico)	\$500–\$1 500 USD (cámara + trípode)
Resolución en Z	10–100 μm	100 μm – 1 mm
Tiempo de captura	Segundos ($< 1\text{ s}$ por vista)	Minutos (planificación + captura)
Tiempo de procesamiento	Segundos–minutos	Minutos–horas
Textura requerida	No (el patrón provee la textura)	Sí (superficies uniformes fallan)
Superf. especulares	Requiere spray antirreflejante	Requiere spray antirreflejante
Escala absoluta	Definida por calibración del sistema	Requiere marcadores de referencia
Portabilidad	Moderada (soporte rígido necesario)	Alta (solo cámara)
Software libre	Open3D, Python personalizado	Meshroom (AliceVision), COL-MAP

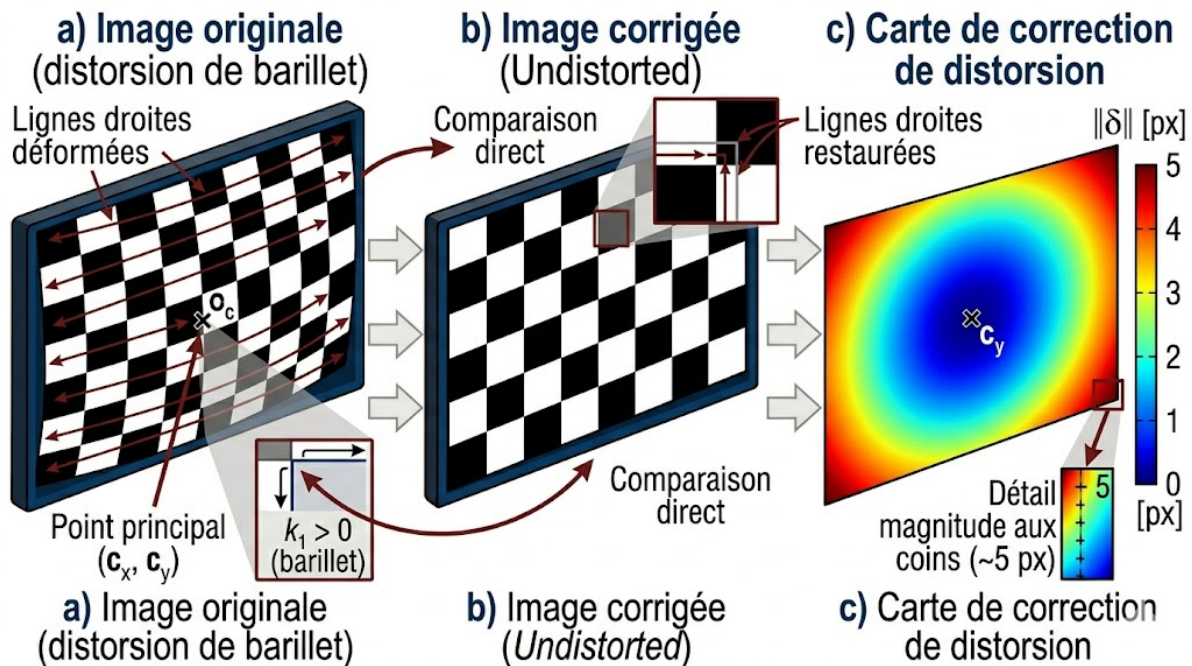


Figura 1.8: Efecto de la distorsión de lente y su corrección. **(a)** Imagen original de un patrón de tablero con distorsión de barril ($k_1 > 0$): las líneas rectas se curvan hacia afuera. **(b)** Imagen corregida (*undistorted*): líneas rectas restauradas. **(c)** Mapa de corrección de distorsión: magnitud del desplazamiento $\|\delta\|$ [px] en cada punto, máximo en las esquinas.

Criterio de selección

- Use **luz estructurada** cuando necesite alta resolución ($< 100 \mu\text{m}$), el objeto cabe en la mesa de trabajo y tiene acceso a un entorno de iluminación controlada.
- Use **fotogrametría** cuando el objeto sea grande, se encuentre en campo, tenga textura natural abundante, o no disponga de un proyector.
- **Ambas requieren** calibración de cámara y spray antirreflejante en superficies brillantes.

1.7 Configuración del entorno de trabajo

En esta sección se instalan y verifican los cuatro programas utilizados a lo largo de la microcredencial.

1.7.1 Open3D (Python)

Open3D es una biblioteca Python/C++ para procesamiento de datos 3D. Instalación:

```
# Crear entorno virtual (recomendado)
```

```
conda create -n inv3d python=3.10 -y
conda activate inv3d

# Instalar Open3D y dependencias
pip install open3d numpy scipy matplotlib trimesh
```

Verificación:

```
import open3d as o3d
mesh = o3d.data.MosterTreeMesh()
MosterTree = o3d.io.read_triangle_mesh(mesh.path)
MosterTree.compute_vertex_normals()
o3d.visualization.draw_geometries([MosterTree],
window_name="MosterTree - Verificación Open3D")
```

1.7.2 CloudCompare

CloudCompare es software libre para edición y comparación de nubes de puntos y mallas. Descarga desde <https://www.cloudcompare.org>.

Verificación: File → Open → cargar archivo PLY del Stanford Bunny → verificar visualización 3D y herramientas de medición.

1.7.3 MeshLab

MeshLab es software libre para procesamiento, edición y reparación de mallas triangulares. Descarga desde <https://www.meshlab.net>.

Verificación: File → Import Mesh → cargar Stanford Bunny (STL o PLY) → verificar visualización, opciones de renderizado (Render → Show Normals) y acceso al menú Filters.

1.7.4 Meshroom

Meshroom es el software de fotogrametría SfM/MVS. Descarga desde <https://alicevision.org/#meshroom>.

Requisito: GPU NVIDIA con CUDA

Meshroom requiere una tarjeta gráfica NVIDIA compatible con CUDA para ejecutar la densificación (DepthMap). Sin GPU compatible, es posible ejecutar hasta el nodo StructureFromMotion (nube dispersa).

Verificación: importar un conjunto de ≥ 10 imágenes de un objeto con textura (ejemplo: imágenes del dataset *Monstree* de AliceVision) → Start Computation → verificar que todos los nodos se ejecutan en verde.

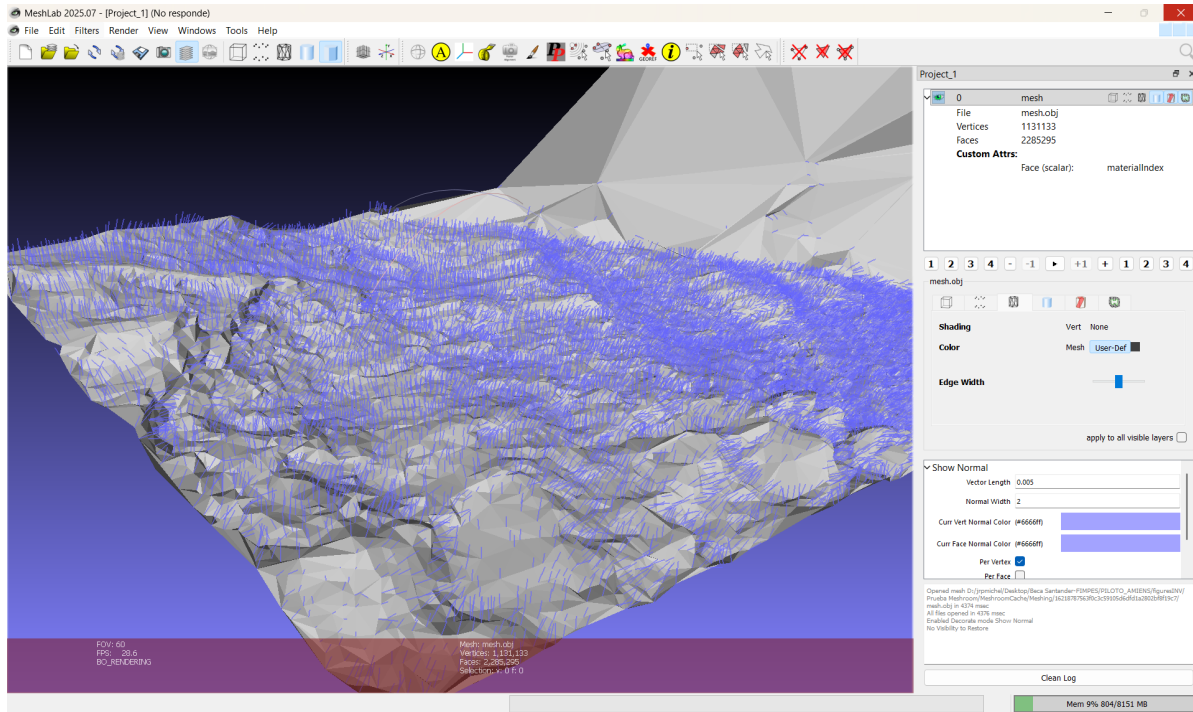


Figura 1.9: Verificación del entorno MeshLab: MsterTree cargado con normales visualizadas. La barra de menús muestra acceso a Filters → Remeshing, Cleaning, Smoothing (herramientas centrales de los Temas 3–4).

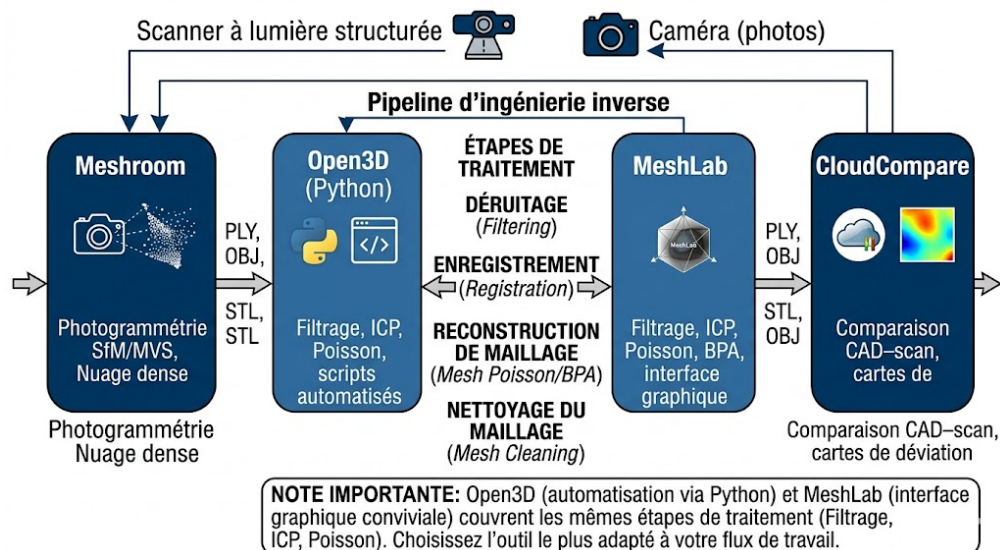


Figura 1.10: Resumen visual del entorno de software. Cuatro herramientas y sus roles en el pipeline: **Meshroom** (captura SfM), **Open3D** (procesamiento Python), **MeshLab** (procesamiento GUI), **CloudCompare** (comparación CAD-scan). Las flechas indican el flujo de datos entre herramientas.

1.7.5 Estructura de directorios del proyecto

Se recomienda la siguiente organización:

```

proyecto_INV/
  01_capturas/
    SL/                # Imágenes de luz estructurada
    foto/              # Fotografías para fotogrametría
  02_nubes/            # Nubes de puntos (PLY, XYZ)
  03_mallas/           # Mallas (STL, OBJ)
  04_CAD/              # Modelos STEP / IGES
  05_reportes/         # Informes, mapas de desviación
  calibracion/         # Imágenes de tablero, parámetros
  scripts/             # Código Python

```

Evidencia de aprendizaje	Cuestionario diagnóstico sobre principios de escaneo 3D, triangulación, codificación por fase, fotogrametría SfM y modelo de cámara <i>pinhole</i> .
Instrumento de evaluación	Examen escrito / cuestionario en línea (20 preguntas de opción múltiple y respuesta corta).
Escala / Criterio de aceptación	Aprobatoria: $\geq 70/100$.

Capítulo 2

Captura 3D: Planificación y Adquisición

Duración: 4 h

Modalidad: Presencial (laboratorio)

Este capítulo aborda el proceso completo de adquisición de datos 3D, desde la preparación física del objeto hasta la obtención de una nube de puntos validada. Se presentan las dos vías de captura cubiertas en la microcredencial — **luz estructurada** y **fotogrametría multivista** — y se establecen los criterios cuantitativos de aceptación que deben cumplirse antes de pasar al preprocesamiento (Tema 3). Al finalizar, el participante será capaz de planificar una sesión de captura, ejecutar la adquisición con ambas técnicas y verificar la calidad de la nube de puntos resultante.

2.1 Preparación del objeto

La calidad de la nube de puntos depende críticamente de la preparación previa del objeto. Una preparación deficiente introduce artefactos, huecos y ruido que ningún algoritmo de posprocesamiento puede compensar totalmente.

2.1.1 Limpieza y acondicionamiento superficial

Antes de cualquier captura, el objeto debe:

1. **Limpiar:** remover polvo, grasa y residuos con aire comprimido y/o alcohol isopropílico. Las partículas en la superficie generan puntos espurios a escala micrométrica.
2. **Secar completamente:** la humedad modifica las propiedades reflectivas; esperar al menos 5 minutos tras limpieza húmeda.
3. **Estabilizar térmicamente:** si la pieza viene de un proceso en caliente (mecanizado, fundición), dejar alcanzar el equilibrio térmico con el ambiente del laboratorio. Gradientes térmicos $> 5^{\circ}\text{C}$ causan dilatación diferencial medible a escala de 0.01 mm.

2.1.2 Agente antirreflejante (spray revelador)

Las superficies **especulares** (metal pulido, vidrio, plástico brillante) reflejan la luz del proyector o del flash fuera del campo de visión de la cámara, generando zonas sin datos.

Para eliminar este efecto se aplica un recubrimiento temporal mate.

Procedimiento: aplicación de spray revelador

1. Agitar el spray de revelador mate (por ejemplo, *AESUB Blue*, *Attblime Scanning Spray* o talco industrial) durante 30 s.
2. Sostener la boquilla a 25–30 cm del objeto.
3. Aplicar capas **uniformes y delgadas** en movimientos de barrido cruzado (horizontal + vertical). Evitar acumulación en cavidades.
4. Espesor objetivo: $\leq 10 \mu\text{m}$ (invisible a simple vista pero suficiente para eliminar especularidad).
5. Verificar cobertura: observar bajo iluminación rasante; las zonas brillantes requieren una pasada adicional.
6. Tiempo de sublimación (sprays tipo AESUB): $\sim 2\text{--}6$ h según temperatura; la capa desaparece sin residuo.

Advertencia: espesor del recubrimiento

Un recubrimiento excesivo ($> 30 \mu\text{m}$) introduce un error sistemático positivo en las dimensiones medidas. Para piezas con tolerancias $\leq \pm 0.05$ mm, es preferible usar sprays de sublimación ultrafina o polvo de talco con soplado posterior, que dejan capas $< 5 \mu\text{m}$. **Documentar siempre el tipo y espesor estimado del recubrimiento en el reporte de captura.**

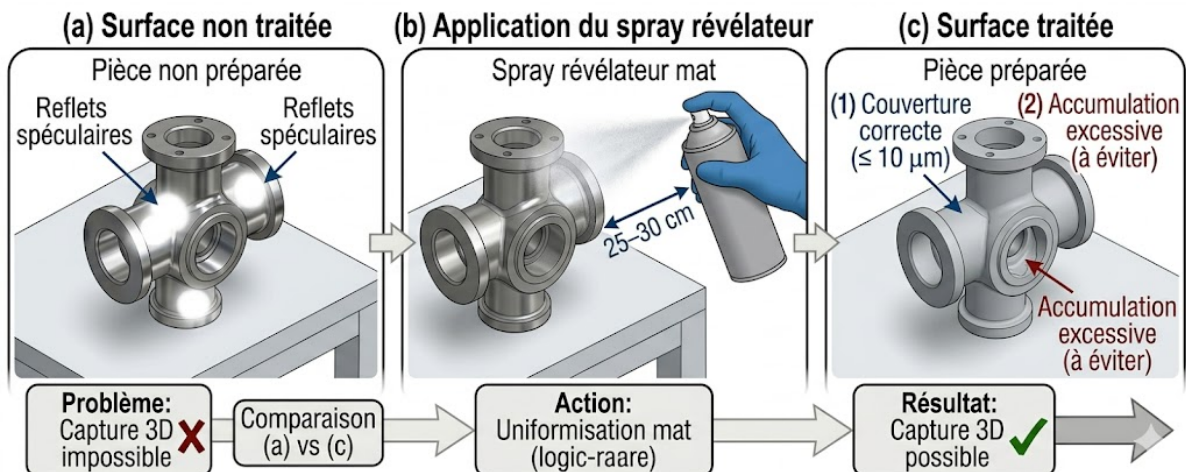


Figura 2.1: Preparación de la superficie para escaneo 3D. **(a)** Pieza metálica pulida antes de la preparación: se observan reflejos especulares que impiden la captura. **(b)** Aplicación de spray revelador mate a 25 cm de distancia. **(c)** Pieza tras la aplicación: superficie uniformemente mate sin brillos visibles. Se señala con flechas una zona de acumulación excesiva (a evitar) y una zona con cobertura correcta.

2.1.3 Marcadores fiduciales y de referencia

Los **marcadores fiduciales** (*reference targets*) cumplen dos funciones:

1. **Escala absoluta:** permiten asignar dimensiones reales (mm) a la reconstrucción 3D, que de otro modo tendría escala arbitraria (especialmente en fotogrametría).
2. **Registro entre vistas:** facilitan la alineación automática de capturas tomadas desde diferentes ángulos, al servir como puntos comunes entre nubes parciales.

Tipos de marcadores

Cuadro 2.1: Tipos de marcadores fiduciales para captura 3D.

Tipo	Descripción	Aplicación principal	Precisión típica
Targets retro-reflectantes	Puntos circulares ($\varnothing$ 3–12 mm) con material retro-reflectivo, detectados automáticamente por el escáner.	Luz estructurada, escáneres industriales.	± 0.02 mm (centroide)
Targets codificados	Marcadores con código binario circular (tipo ArUco o 12-bit) que permiten identificación única.	Fotogrametría multivista (Meshroom, Metashape).	± 0.05 mm (fotogrametría)
Barra de escala calibrada	Barra rígida con dos targets a distancia certificada (por ejemplo, $L = 200.00 \pm 0.02$ mm).	Asignación de escala absoluta en fotogrametría.	Depende de la certificación
Targets adhesivos simples	Puntos circulares negros sobre fondo blanco (impresos en papel).	Captura rápida, proyectos educativos.	± 0.1 – 0.3 mm

Colocación de marcadores

Las reglas prácticas para la distribución de marcadores son:

- Colocar ≥ 5 marcadores **no coplanares** alrededor del objeto (en la base, la pieza y el entorno inmediato).
- Garantizar que al menos **3 marcadores** sean visibles en cada vista individual.

- Distribuir marcadores en las **tres dimensiones** del volumen de captura: no todos sobre el plano de la mesa.
- Para fotogrametría, incluir al menos una **barra de escala calibrada** visible en $\geq 80\%$ de las imágenes.
- Evitar marcadores sobre superficies **curvas de radio pequeño** ($R < 10 \text{ mm}$): la deformación del target degrada la detección del centroide.

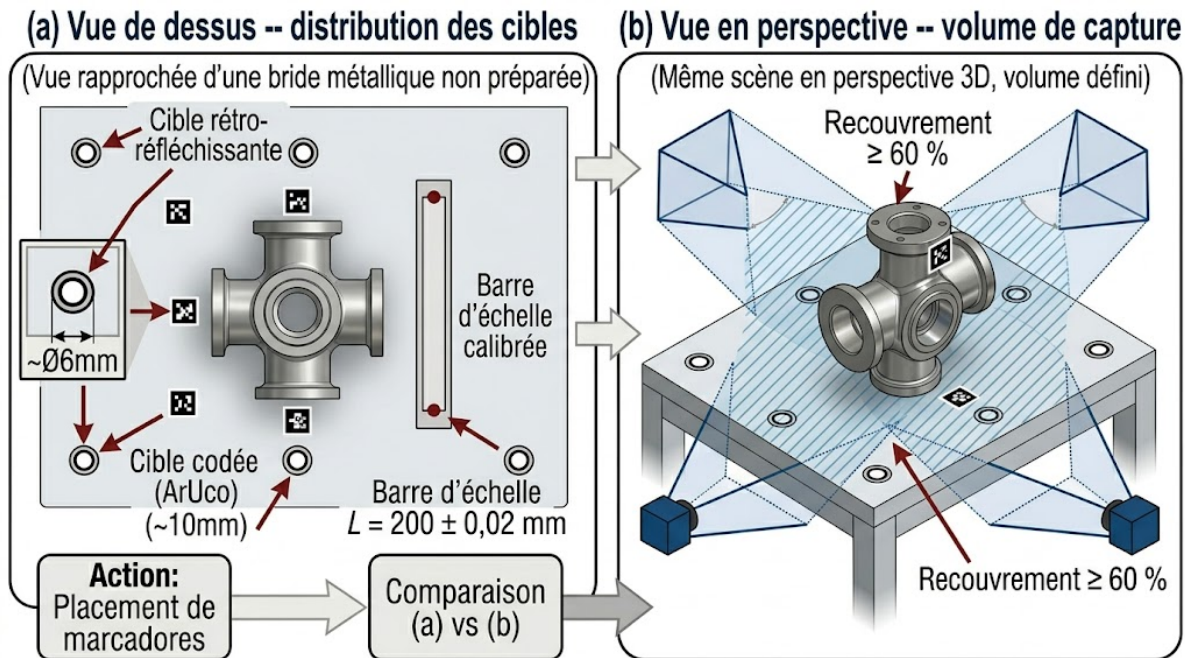


Figura 2.2: Distribución de marcadores fiduciales para captura 3D. **(a)** Vista superior: pieza mecánica sobre plataforma con distribución de targets retro-reflectantes (puntos blancos) y targets codificados (cuadrados) alrededor y sobre el objeto. **(b)** Vista en perspectiva: los marcadores definen un volumen tridimensional; una barra de escala calibrada se ubica en el plano de la base. Las líneas punteadas indican la cobertura de solapamiento entre vistas consecutivas ($\geq 60\%$).

2.1.4 Selección de la pieza de prueba

Para fines de aprendizaje, la pieza de prueba debe reunir las siguientes características:

- **Geometría mixta:** combinación de planos, superficies cilíndricas, radios de acuerdo y al menos una cavidad (para evaluar oclusiones).
- **Dimensión:** envoltorio de 5–20 cm (cabe en la mesa de trabajo y permite validación con calibrador/micrómetro).
- **Material:** acero, aluminio o plástico de ingeniería (ABS, POM). Las piezas fundidas con textura natural son ideales para fotogrametría; las mecanizadas pulidas requieren spray revelador.
- **Referencia dimensional:** disponer de al menos 3 cotas verificables con instrumento calibrado (calibrador $\pm 0.02 \text{ mm}$) para la validación final (Tema 6).

Analogía: la pieza como paciente en una radiografía

Así como un radiólogo prepara al paciente (posición, contraste, parámetros del equipo) antes de tomar una imagen, el ingeniero metrológico prepara la pieza y el entorno antes de la captura 3D. Una preparación deficiente no se compensa con mejor software — *garbage in, garbage out*.

2.2 Captura por luz estructurada

En esta sección se detalla el procedimiento de adquisición 3D mediante un sistema de **proyección de luz estructurada** (SL). Los principios físicos de codificación Gray y *phase shifting* se describieron en el Tema 1 (Sección 1.3); aquí el enfoque es operativo.

2.2.1 Configuración del sistema óptico

Antes de iniciar la captura, configurar los tres componentes del sistema:

Proyector

1. **Resolución:** nativa del equipo ($\geq 800 \times 600$ px); evitar interpolación (modo *native* en las opciones de proyección).
2. **Enfoque:** ajustar manualmente hasta obtener franjas nítidas sobre el objeto. Verificar con el patrón de frecuencia más alta de la secuencia: cada línea clara y oscura debe ser distinguible.
3. **Brillo:** ajustar para saturar $< 5\%$ de los píxeles de la cámara cuando se proyecta el patrón todo-blanco. Usar el histograma en vivo de la cámara para verificar.
4. **Keystone:** desactivar la corrección trapezoidal automática; la geometría del patrón debe permanecer inalterada.

Cámara

1. **Modo manual:** desactivar autoexposición, autoenfoque y balance de blancos automático.
2. **Exposición:** ajustar de modo que el pico del histograma del patrón más brillante (todo-blanco) alcance $\sim 80\%$ del rango dinámico ($\approx 200/255$ en 8 bits).
3. **Enfoque:** enfocar sobre la superficie del objeto usando la mayor profundidad de campo posible (apertura $f/8$ – $f/11$).
4. **Ganancia/ISO:** mantener al mínimo para reducir ruido. Incrementar solo si la exposición máxima resulta insuficiente.

Geometría del montaje

Los parámetros geométricos determinan la resolución y las oclusiones del sistema (ver Ec. (1.3) del Tema 1):

$$\theta = 2 \arctan\left(\frac{B}{2Z_0}\right) \quad (2.1)$$

donde B es la *baseline* proyector–cámara y Z_0 la distancia de trabajo al objeto. El ángulo de triangulación θ debe mantenerse en el rango 15° – 30° como compromiso entre resolución en profundidad y oclusiones:

Cuadro 2.2: Relación entre ángulo de triangulación, resolución y oclusiones para una distancia de trabajo $Z_0 = 500$ mm y $f = 12$ mm.

θ [°]	B [mm]	δZ [μm]*	Observaciones
10	87	80	Pocas oclusiones, baja resolución
15	131	53	Compromiso mínimo aceptable
20	176	40	Compromiso recomendado
25	222	32	Buena resolución, oclusiones moderadas
30	268	26	Alta resolución, oclusiones significativas

* Con $\delta d = 0.1$ px, píxel 3.5 μm. Valores de referencia, dependen de la óptica específica.

2.2.2 Secuencia de patrones y adquisición

El procedimiento de captura por luz estructurada sigue estos pasos:

1. **Controlar la iluminación ambiente:** apagar luces del laboratorio o reducirlas al mínimo. La luz parásita añade una componente de fondo que reduce el contraste de los patrones y degrada el cálculo de fase.
2. **Proyectar la secuencia de patrones:** el software de control proyecta automáticamente la secuencia completa (Gray + *phase shifting*, Sección 1.3.4) y captura las imágenes sincronizadas. Parámetros típicos:
 - $N_{\text{Gray}} = 10$ patrones (para 1024 columnas del proyector)
 - $M_{\text{PS}} = 4$ patrones sinusoidales (desfases $0, \pi/2, \pi, 3\pi/2$)
 - Total: 14 imágenes por vista, adquiridas en < 1 s a 30 fps
3. **Verificar la captura:** inspeccionar visualmente las imágenes. Buscar:
 - Saturación (zonas blancas sin detalle): reducir exposición o brillo del proyector.
 - Subexposición (franjas apenas visibles): incrementar exposición.
 - Movimiento (*motion blur*): la pieza y el sistema deben estar completamente inmóviles durante la secuencia.
4. **Rotar el objeto (o el sistema)** para cubrir toda la geometría. El ángulo de rotación entre vistas consecutivas depende de la complejidad:
 - Geometría convexa simple: rotaciones de 30° – $45^\circ \Rightarrow 8$ – 12 vistas.

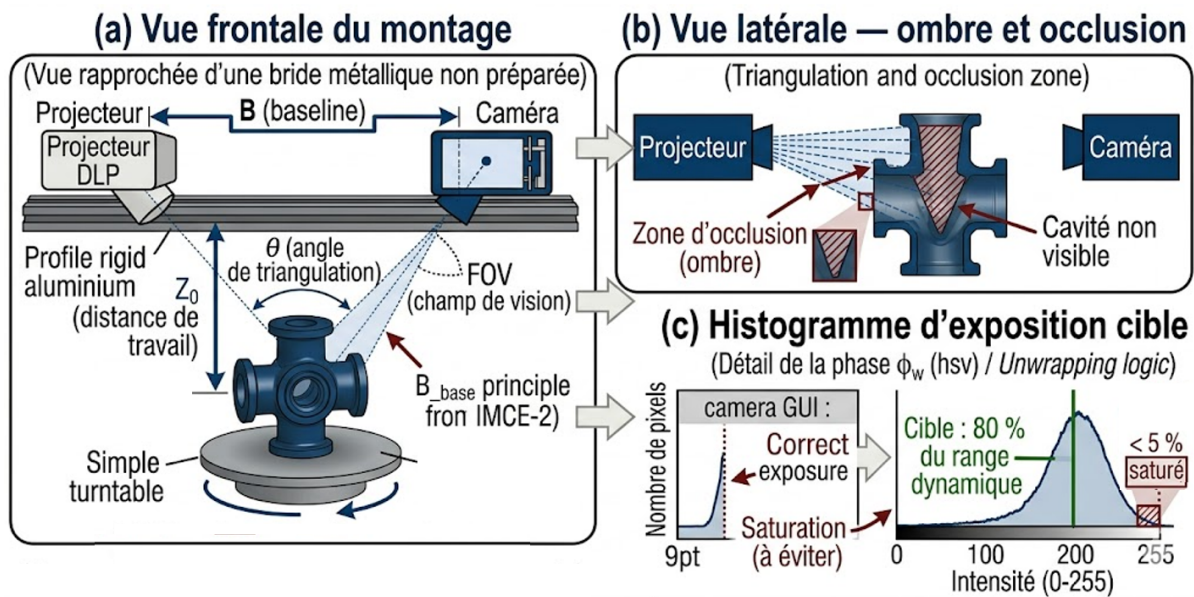


Figura 2.3: Configuración del sistema de luz estructurada para la captura. **(a)** Vista frontal del montaje: proyector y cámara montados sobre barra rígida, separados por *baseline* B , enfocados sobre la pieza ubicada a distancia Z_0 . Se señalan: ángulo de triangulación θ , campo de visión (FOV) y distancia de trabajo. **(b)** Vista lateral: triángulo proyector–pieza–cámara con zona de sombra (oclusión) en una cavidad de la pieza. **(c)** Detalle de la interfaz de la cámara mostrando el histograma de exposición: el pico del patrón todo-blanco a $\sim 80\%$ del rango dinámico, sin saturación.

- Geometría con cavidades: rotaciones de 15° – $20^\circ \Rightarrow 18$ – 24 vistas.

5. **Registrar los parámetros:** anotar en la bitácora (o en el archivo de metadatos) para cada vista: ángulo de rotación, número de imágenes, exposición, fecha/hora.

Buena práctica: imagen de referencia sin patrón

Antes de proyectar la secuencia, capturar una imagen del objeto **sin patrón proyectado** (solo iluminación ambiente). Esta imagen de referencia es útil para: (i) asignar textura a la nube de puntos, (ii) identificar zonas de reflexión especular residual, (iii) documentar el estado de la pieza durante la captura.

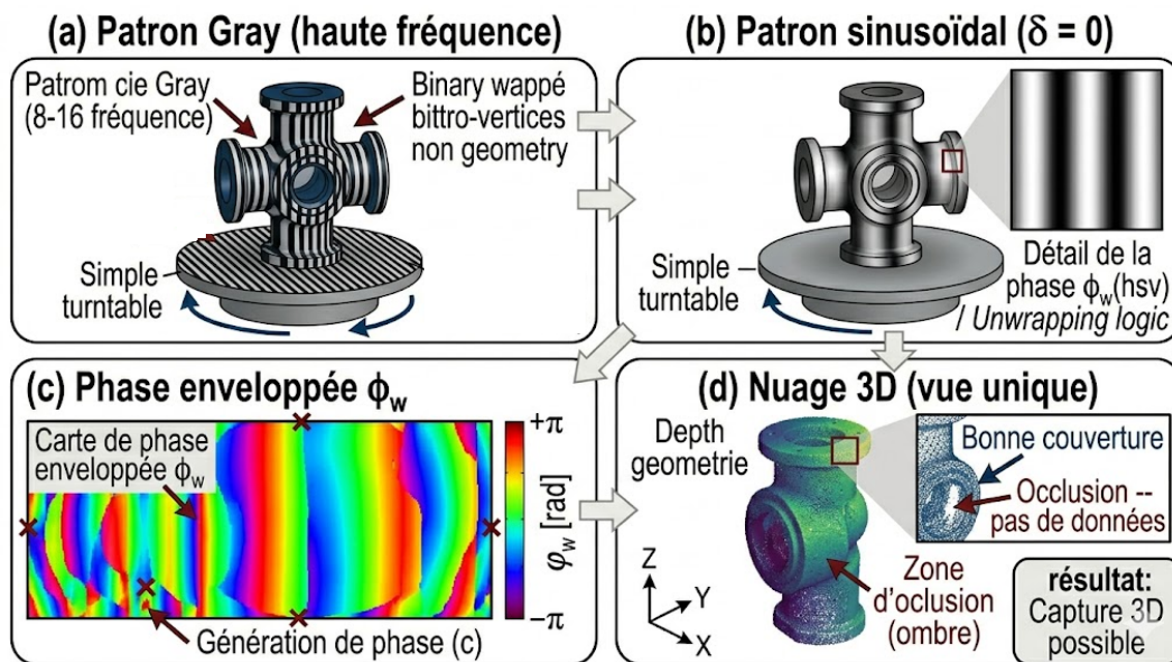


Figura 2.4: Secuencia de captura por luz estructurada y verificación de calidad. **(a)** Imagen de la pieza bajo patrón Gray de alta frecuencia: franjas nítidas sobre toda la superficie, sin saturación. **(b)** Imagen bajo patrón sinusoidal ($\delta = 0$): gradiente suave de intensidad. **(c)** Mapa de fase envuelta ϕ_w resultante (colormap cíclico hsv), mostrando las transiciones de fase sobre la geometría. **(d)** Nube de puntos 3D parcial (una sola vista) en falso color por profundidad. Se señalan: zona de buena cobertura (azul) y zona de oclusión sin datos (rojo).

2.2.3 Cobertura y planificación de vistas múltiples

El objetivo de la captura multivista es lograr una cobertura $\geq 90\%$ de la superficie del objeto. La cobertura se define como:

$$C = \frac{A_{\text{capturada}}}{A_{\text{total}}} \times 100\% \quad (2.2)$$

donde $A_{\text{capturada}}$ es el área de la superficie con datos 3D válidos y A_{total} es el área total de la pieza. Las zonas sin datos (huecos) pueden deberse a:

1. **Oclusiones:** cavidades o ángulos que no son visibles desde ninguna vista en la configuración actual.
2. **Reflexión especular residual:** zonas donde el spray revelador no cubrió adecuadamente.
3. **Ángulo rasante:** superficies casi paralelas al eje óptico ($< 15^\circ$ respecto al rayo de observación), donde la densidad de puntos se degrada.

Advertencia: verificar cobertura antes de desmontar

Es crítico verificar la cobertura **en el laboratorio**, antes de desmontar la pieza y desactivar el sistema. Una vez que la pieza se retira del montaje, no es posible repetir capturas complementarias sin reiniciar todo el proceso de alineación. Cargar la nube parcial en CloudCompare y verificar visualmente **antes de concluir la sesión**.

2.3 Captura por fotogrametría (Meshroom)

La segunda vía de adquisición 3D en la microcredencial es la **fotogrametría multivista** mediante el software libre **Meshroom** (AliceVision). A diferencia de la luz estructurada, la fotogrametría no requiere proyector: la reconstrucción se basa en la correspondencia de puntos de textura natural entre múltiples fotografías.

2.3.1 Configuración de la cámara

La calidad de la reconstrucción fotogramétrica depende de imágenes nítidas, bien expuestas y con profundidad de campo suficiente.

¿Se puede usar un smartphone?

Sí, un smartphone reciente (≥ 12 MP con cámara principal) produce resultados aceptables para fines didácticos. Precauciones: (i) desactivar HDR y modos computacionales que alteran la geometría entre tomas; (ii) fijar la exposición (mantener presionado el punto de enfoque en la mayoría de apps de cámara); (iii) usar la cámara principal (no gran angular ni telefoto, cuyos modelos de distorsión son más complejos). Meshroom identifica automáticamente los parámetros intrínsecos de cámaras de smartphone registradas en su base de datos.

Cuadro 2.3: Parámetros recomendados de la cámara para fotogrametría.

Parámetro	Valor recomendado	Justificación
Modo de exposición	Manual (M)	Evitar variaciones de brillo entre imágenes que confundan el <i>matching</i> .
Apertura	$f/8$ – $f/11$	Máxima profundidad de campo sin difracción excesiva.
ISO	Mínimo disponible (100–200)	Minimizar ruido (el ruido degrada la detección de <i>features</i>).
Velocidad de obturación	$\leq 1/60$ s (con trípode)	Evitar <i>motion blur</i> ; usar trípode o velocidad más rápida a mano.
Balance de blancos	Fijo (luz de día o tungsteno)	Consistencia cromática entre imágenes.
Formato	RAW + JPEG (o solo JPEG de máxima calidad)	RAW para posprocesamiento; JPEG compatible con Meshroom.
Resolución	Máxima nativa del sensor	Más píxeles = más puntos de interés detectables.
Flash	Desactivado	El flash crea reflejos variables entre imágenes que rompen la correspondencia.

2.3.2 Planificación de la trayectoria de captura

La distribución angular y el solapamiento entre imágenes son los factores más determinantes para el éxito del pipeline SfM (Sección 1.4).

Reglas fundamentales

1. **Solapamiento** $\geq 60\%$: cada imagen debe compartir al menos el 60% de su contenido con las imágenes adyacentes. Un solapamiento menor produce huecos en la nube dispersa; un solapamiento de 70–80% es ideal.
2. **Ángulo entre vistas**: separaciones angulares de 10° – 15° alrededor del objeto. Esto equivale a 24–36 imágenes para una revolución completa de 360° .
3. **Varias alturas**: no capturar solo desde el nivel de la mesa. Agregar al menos un anillo adicional a $\sim 30^\circ$ por encima (para cubrir la parte superior) y, si es posible, vistas desde abajo para la base.
4. **Distancia constante**: mantener aproximadamente la misma distancia al objeto en todas las tomas. Variaciones $> 30\%$ dificultan el *matching* y generan densidad irregular.
5. **No mover el objeto**: el objeto debe permanecer inmóvil durante toda la sesión. La cámara se mueve alrededor del objeto, **nunca al revés** (a menos que se use una plataforma giratoria con fondo uniforme y marcadores fijos al plato).

$$N_{\text{imágenes}} \approx \frac{360^\circ}{\Delta\alpha} \times N_{\text{anillos}} \quad (2.3)$$

donde $\Delta\alpha$ es la separación angular y N_{anillos} el número de alturas de captura. Para $\Delta\alpha = 15^\circ$ y $N_{\text{anillos}} = 2$: $N \approx 24 \times 2 = 48$ imágenes.

Errores frecuentes en la trayectoria

Errores de captura que causan fallo del pipeline SfM

1. **Pocas imágenes**: < 20 para un objeto de complejidad media produce huecos extensos en la nube dispersa.
2. **Solapamiento insuficiente**: saltar ángulos $> 20^\circ$ rompe la cadena de correspondencias; el nodo `StructureFromMotion` de Meshroom no converge.
3. **Fondo confuso**: fondos con textura rica (estantería, pared con pósters) generan *features* falsos que desvían el modelo. Usar fondo uniforme (tela blanca o gris) cuando sea posible.
4. **Variación de iluminación**: fotos tomadas con exposición automática producen cambios de brillo que degradan la correspondencia.
5. **Objeto movido**: cualquier desplazamiento entre tomas invalida la geometría epipolar; descartar y repetir toda la sesión.

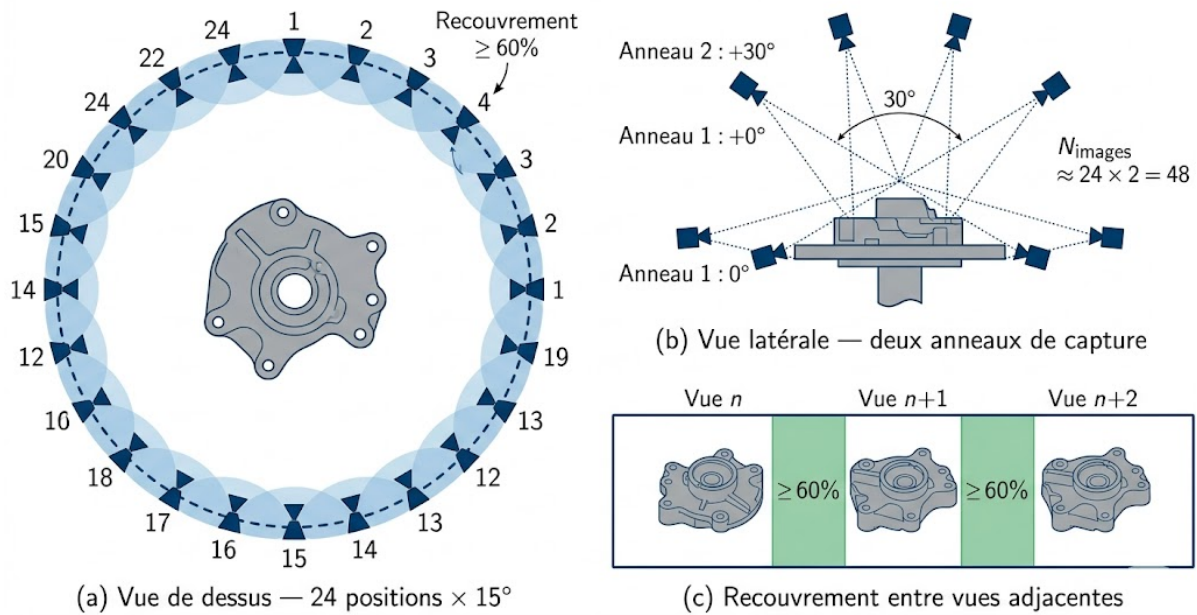


Figura 2.5: Planificación de la trayectoria de captura fotogramétrica. **(a)** Vista superior: pieza al centro, trayectoria circular de la cámara (línea punteada) con posiciones marcadas cada 15° (24 posiciones). Las zonas de solapamiento entre imágenes adyacentes se indican como arcos sombreados. **(b)** Vista lateral: dos anillos de captura — nivel de la mesa (0°) y anillo elevado ($+30^\circ$). Se muestran los conos de visión de la cámara convergiendo hacia el objeto. **(c)** Ejemplo de tres imágenes consecutivas (vistas n , $n+1$, $n+2$) con las zonas de solapamiento resaltadas en verde ($\geq 60\%$).

2.3.3 Marcadores codificados y barra de escala

A diferencia de la luz estructurada (donde la escala se define en la calibración del sistema), la fotogrametría genera una reconstrucción con **escala arbitraria**. Para asignar dimensiones reales:

1. Colocar una **barra de escala calibrada** (o dos targets a distancia conocida) visible en $\geq 80\%$ de las imágenes.
2. Opcionalmente, distribuir **targets codificados ArUco** (impresos en papel adhesivo) que Meshroom detecta automáticamente mediante el nodo ConvertSfmFormat con detección de marcadores habilitada.
3. En el posprocesamiento (CloudCompare o Meshroom), aplicar la **transformación de escala**:

$$s = \frac{L_{\text{real}}}{L_{\text{modelo}}} \quad (2.4)$$

donde L_{real} es la distancia calibrada entre targets [mm] y L_{modelo} es la misma distancia medida en la reconstrucción (unidades arbitrarias). Todos los puntos se multiplican por s .

2.3.4 Ejecución del pipeline SfM en Meshroom

El flujo operativo en Meshroom para la sesión de captura es:

1. **Importar imágenes:** arrastrar todas las fotografías al panel izquierdo de Meshroom. El nodo CameraInit asigna automáticamente los parámetros intrínsecos si la cámara está en la base de datos.
2. **Verificar reconocimiento de cámara:** en el nodo CameraInit, confirmar que los campos focalLength, sensorWidth y distortionParams están poblados. Si la cámara no es reconocida, ingresar manualmente el ancho del sensor [mm] (consultar especificaciones del fabricante).
3. **Ejecutar el pipeline:** clic en Start o Compute. El pipeline ejecuta secuencialmente:

CameraInit → FeatureExtraction → ImageMatching → FeatureMatching →
StructureFromMotion → DepthMap → DepthMapFilter → Meshing → MeshFiltering →
Texturing

4. **Monitorear el progreso:** los nodos cambian de color según su estado: gris (pendiente), naranja (en ejecución), verde (completado), rojo (error).
5. **Verificar la nube dispersa:** al completar el nodo StructureFromMotion, hacer clic sobre él para visualizar la nube dispersa y las posiciones de cámara estimadas (pirámides azules). Verificar:
 - Todas las imágenes están registradas (comparar $N_{\text{registradas}}$ vs. $N_{\text{importadas}}$ en el log del nodo).

- Las posiciones de cámara forman un patrón coherente (anillo alrededor del objeto).
 - No hay cámaras “volantes” en posiciones imposibles.
6. **Exportar la nube densa:** al finalizar el pipeline, exportar la nube densa en formato PLY desde el nodo Meshing o usar directamente la salida del nodo DepthMapFilter.

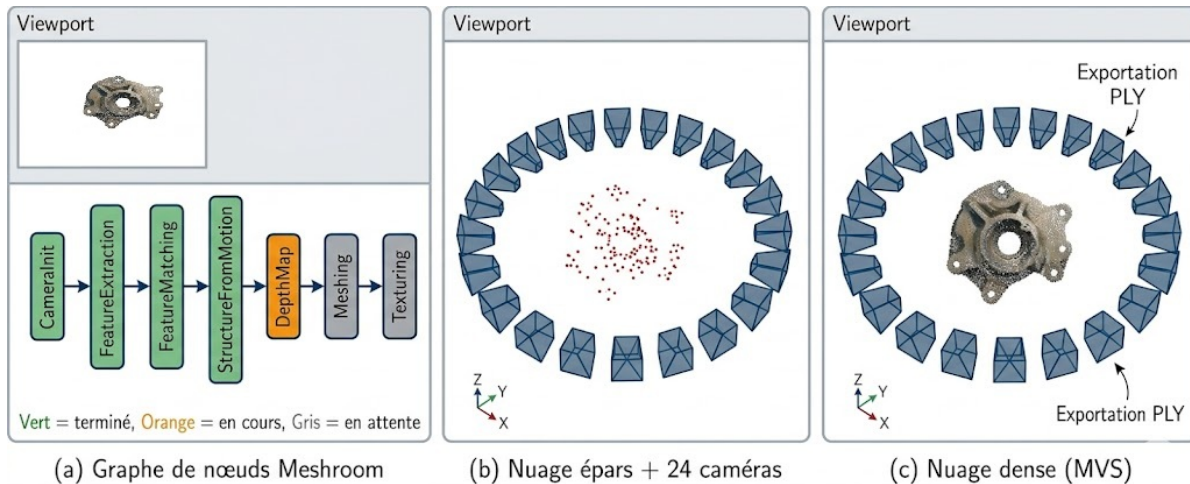


Figura 2.6: Pipeline de Meshroom durante la captura fotogramétrica. **(a)** Interfaz de Meshroom con el grafo de nodos: nodos completados (verde) hasta StructureFromMotion, nodo DepthMap en ejecución (naranja). **(b)** Nube de puntos dispersa con 24 posiciones de cámara estimadas (pirámides azules) formando un anillo alrededor del objeto. **(c)** Nube de puntos densa (MVS) coloreada por textura del objeto, exportada en formato PLY.

2.4 Criterios de aceptación de la nube de puntos

Antes de proceder al preprocesamiento (Tema 3), la nube de puntos debe satisfacer los criterios cuantitativos de la Tabla 2.4.

Si la nube no cumple algún criterio, la acción correctiva depende del problema:

- **Cobertura** < 90%: agregar vistas adicionales en las zonas faltantes y repetir la captura (sin desmontar la pieza).
- **Densidad insuficiente:** reducir la distancia de trabajo (aumenta la resolución espacial) o usar una cámara de mayor resolución.
- **Ruido excesivo:** revisar exposición (subexposición = ruido), verificar estabilidad del montaje, reducir ganancia/ISO.
- **Artefactos:** verificar que no hay objetos en segundo plano capturados; eliminar manualmente en el preprocesamiento si son aislados.

Cuadro 2.4: Criterios de aceptación de la nube de puntos cruda.

Criterio	Umbral	Método de verificación
Cobertura	$\geq 90\%$	Inspección visual en CloudCompare: rotar la nube 360° y verificar ausencia de huecos mayores a 5% del área total.
Densidad	$> 1\,000 \text{ pts/cm}^2$	CloudCompare: Tools → Point picking sobre una región plana de $1 \times 1 \text{ cm}$, contar puntos; o calcular densidad promedio con Tools → Density → Surface Density.
Ruido	$\sigma_n \leq 0.10 \text{ mm}$	Ajustar un plano a una región plana conocida y medir la desviación estándar de los residuos; o reportar el RMS del ajuste.
Artefactos	Ausentes	Inspección visual: verificar que no hay puntos flotantes, superficies fantasma ni duplicados.
Registro (SL multivista)	Error RMS $\leq 0.30 \text{ mm}$	Reportado por el software de alineación tras el registro inicial (ICP grueso).

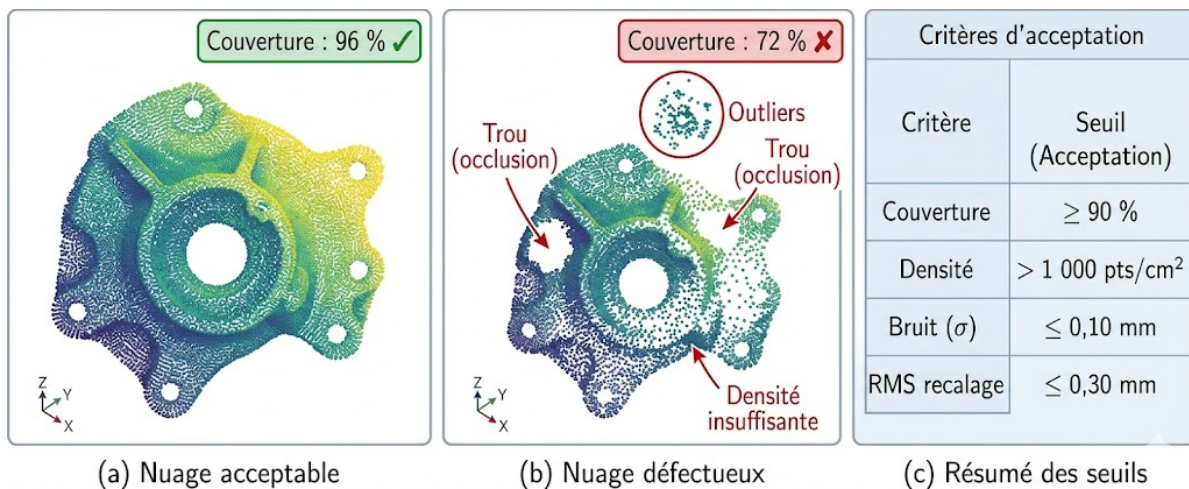


Figura 2.7: Ejemplos de nubes de puntos: aceptable vs. defectuosa. **(a)** Nube de puntos aceptable: cobertura completa, densidad uniforme, sin artefactos. Falso color por profundidad. **(b)** Nube defectuosa: huecos extensos (flechas rojas), puntos flotantes (*outliers*, encerrados en círculos rojos), densidad irregular. **(c)** Tabla superpuesta resumiendo los criterios de aceptación: cobertura $\geq 90\%$, densidad $> 1\,000 \text{ pts/cm}^2$, RMS $\leq 0.30 \text{ mm}$.

2.5 Inspección y visualización de la nube de puntos

Una vez obtenida la nube de puntos, es necesario inspeccionarla cuantitativa y visualmente antes del preprocesamiento. Se presentan dos vías equivalentes.

2.5.1 Vía A: Python con Open3D

El siguiente bloque de código realiza la inspección básica de una nube de puntos en formato PLY:

```
import open3d as o3d
import numpy as np

# 1. Cargar la nube de puntos
pcd = o3d.io.read_point_cloud("captura_pieza.ply")
print(f"Número de puntos: {len(pcd.points)}")

# 2. Calcular bounding box
aabb = pcd.get_axis_aligned_bounding_box()
dims = aabb.get_extent()
print(f"Dimensiones (X, Y, Z): {dims[0]:.2f} x "
      f"{dims[1]:.2f} x {dims[2]:.2f} mm")

# 3. Estimar densidad superficial (región plana de referencia)
# Seleccionar una región plana manualmente o usar:
distances = pcd.compute_nearest_neighbor_distance()
avg_dist = np.mean(distances)
density = 1.0 / (avg_dist ** 2) # pts/mm^2 aprox.
print(f"Distancia media entre vecinos: {avg_dist:.4f} mm")
print(f"Densidad estimada: {density:.0f} pts/mm^2 "
      f"({density * 100:.0f} pts/cm^2)")

# 4. Estimar ruido sobre una región plana
# (Suponiendo que se recortó una región plana previamente)
# plane_model, inliers = pcd.segment_plane(
#     distance_threshold=0.1, ransac_n=3,
#     num_iterations=1000)
# residuos = np.abs(...)

# 5. Visualizar con normales estimadas
pcd.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(
        radius=1.0, max_nn=30))
o3d.visualization.draw_geometries(
    [pcd], window_name="Inspección de nube",
```

```
point_show_normal=True)
```

Verificación rápida de densidad con Open3D

La densidad estimada como $1/d^2$ (donde d es la distancia media al vecino más cercano) es una aproximación para distribuciones uniformes. Para una medición precisa, seleccionar una región plana de 1×1 cm con `o3d.geometry.crop_point_cloud()` y contar los puntos resultantes directamente.

2.5.2 Vía B: MeshLab (sin programación)

El flujo de inspección en MeshLab es:

1. **Importar la nube:** File → Import Mesh → seleccionar el archivo PLY o XYZ.
2. **Verificar número de puntos:** Panel Layer Dialog (derecha) → verificar el conteo de vértices reportado junto al nombre de la capa.
3. **Calcular medidas geométricas:** Filters → Quality Measure and Computations → Compute Geometric Measures. Esto reporta:
 - Número de vértices y caras (si existe malla).
 - Dimensiones del *bounding box* (ancho, alto, profundidad).
 - Volumen (si la malla es cerrada).
4. **Visualizar normales:** Render → Show Vertex Normals. Verificar que las normales apunten hacia afuera de la superficie. Si no se calcularon al importar, ejecutar: Filters → Normals, Curvatures and Orientation → Compute Normals for Point Sets (Neighbor num = 10–20).
5. **Visualizar bounding box:** Render → Show Box Corners. Comparar las dimensiones con las cotas conocidas de la pieza como verificación gruesa.
6. **Inspección visual:** Rotar la nube en 360° usando el ratón. Buscar:
 - Huecos (zonas sin puntos): marcar las posiciones para captura adicional.
 - Puntos aislados (*outliers*): flotantes lejos de la superficie principal.
 - Artefactos: superficies duplicadas o fantasma por reflexiones múltiples.

Evidencia de aprendizaje	Nube de puntos PLY/XYZ con cobertura documentada y reporte de parámetros de captura (tipo de sistema, exposición, número de vistas, spray utilizado, marcadores, distancia de trabajo).
Instrumento de evaluación	Rúbrica de planificación y calidad de nube de puntos.
Escala / Criterio de aceptación	Cobertura $\geq 90\%$; densidad $> 1\,000$ pts/cm ² ; documentación completa de la bitácora de captura. Umbral: 80/100.

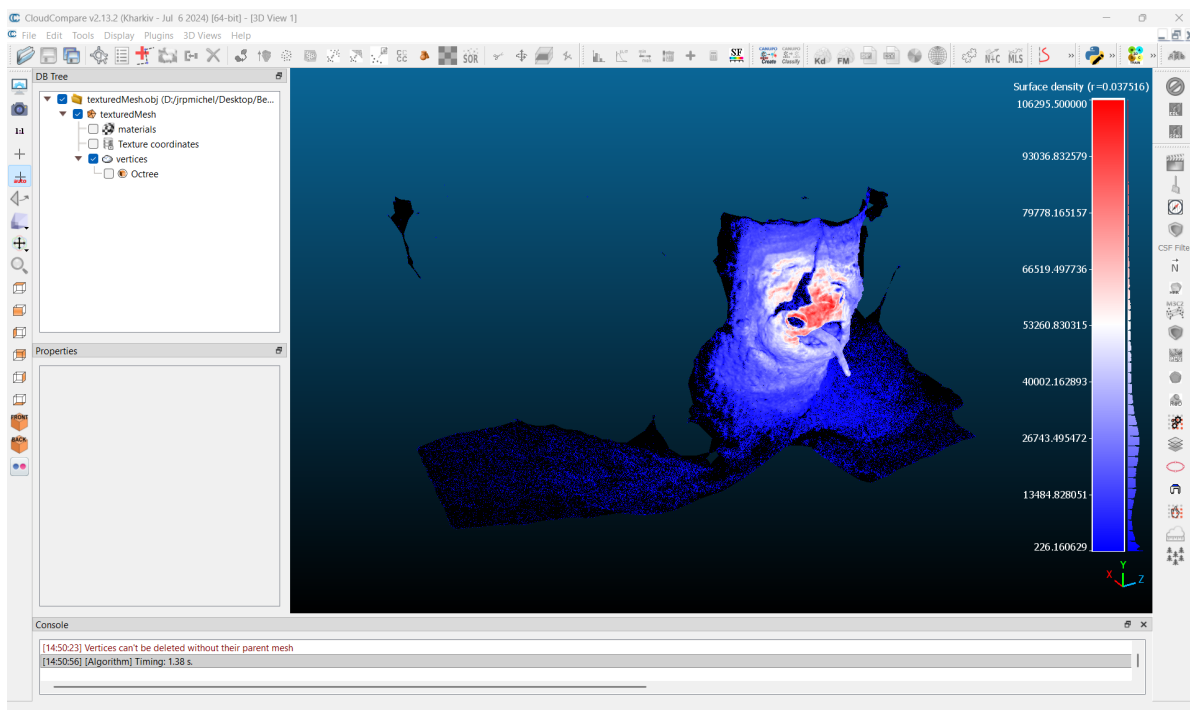


Figura 2.8: Inspección de la nube de puntos en CloudCompare. Vista 3D de la nube en falso color por densidad superficial. El panel lateral muestra las propiedades de la nube: número de puntos, *bounding box*, y la herramienta Surface Density con resultado $> 1\,000$ pts/cm².

Capítulo 3

Preprocesamiento, Filtrado y Registro ICP Multivista

Duración: 4 h

Modalidad: Presencial (laboratorio)

La nube de puntos cruda obtenida en el Tema 2 contiene inevitablemente puntos espurios (*outliers*), ruido de medición y una densidad irregular. Además, cuando el objeto se captura desde múltiples vistas, cada nube parcial está en su propio sistema de coordenadas local. Este capítulo aborda las tres operaciones fundamentales de preprocesamiento: **(1)** filtrado de *outliers*, **(2)** estimación de normales y submuestreo, y **(3)** registro (*alignment*) de múltiples vistas mediante el algoritmo **ICP** (*Iterative Closest Point*). Al finalizar, el participante dispondrá de una nube de puntos limpia, registrada y lista para la reconstrucción de superficies (Tema 4).

3.1 Filtrado estadístico de *outliers*

Los *outliers* son puntos que no pertenecen a la superficie real del objeto. Su origen puede ser:

- **Reflexiones múltiples:** la luz del proyector rebota entre superficies cóncavas antes de llegar a la cámara, generando puntos “fantasma” a profundidades erróneas.
- **Ruido del sensor:** fluctuaciones estocásticas del detector, amplificadas en condiciones de baja señal (subexposición).
- **Objetos del entorno:** puntos del fondo, la mesa o los soportes capturados involuntariamente.
- **Errores de correspondencia:** en fotogrametría, emparejamientos falsos de *features* que producen puntos 3D en posiciones arbitrarias.

3.1.1 Filtrado por eliminación estadística (SOR)

El método más utilizado es el **Statistical Outlier Removal** (SOR). Para cada punto $\mathbf{p}_i$ de la nube, el algoritmo:

1. Calcula la distancia media $\bar{d}_i$ a sus k vecinos más cercanos (k -NN).

2. Calcula la media global μ_d y la desviación estándar σ_d del conjunto $\{\bar{d}_i\}_{i=1}^N$.
3. Elimina todo punto cuya distancia media exceda un umbral:

$$\bar{d}_i > \mu_d + \alpha \cdot \sigma_d \implies \text{punto clasificado como } outlier \quad (3.1)$$

donde α es el multiplicador de desviación estándar que controla la agresividad del filtrado.

Cuadro 3.1: Parámetros recomendados para el filtrado SOR.

Parámetro	Valor típico	Efecto
k (vecinos)	20	Valores bajos (< 10) hacen el filtro sensible a variaciones locales de densidad; valores altos (> 50) suavizan la estadística y pueden retener <i>outliers</i> en grupos.
α (multiplicador)	2.0	$\alpha = 1.0$: filtrado agresivo — elimina puntos legítimos en zonas de baja densidad (bordes, aristas). $\alpha = 2.0$: compromiso estándar. $\alpha = 3.0$: conservador — solo elimina <i>outliers</i> extremos.

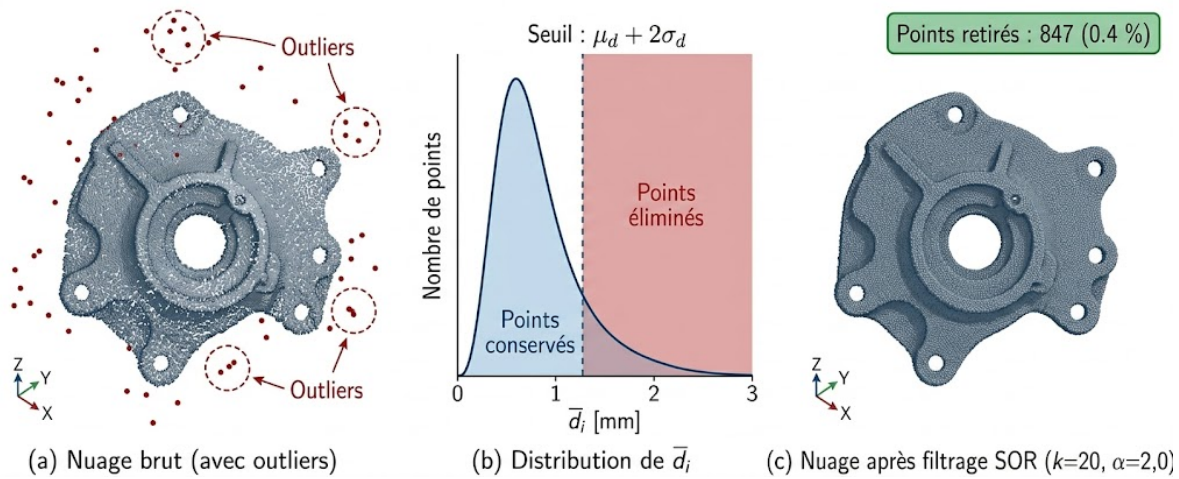


Figura 3.1: Filtrado estadístico de *outliers* (SOR). **(a)** Nube de puntos cruda con *outliers* visibles (puntos aislados lejanos a la superficie, señalados con círculos rojos). **(b)** Distribución de $\bar{d}_i$ (histograma): la línea vertical indica el umbral $\mu_d + 2\sigma_d$; los puntos a la derecha del umbral (zona roja) se eliminan. **(c)** Nube de puntos tras SOR ($k = 20$, $\alpha = 2.0$): *outliers* eliminados, superficie limpia. Se indica el número de puntos removidos y el porcentaje respecto al total.

3.1.2 Eliminación de piezas aisladas por diámetro

Además del SOR, es útil eliminar **componentes conexas pequeñas**: grupos de puntos que forman “islas” desconectadas de la superficie principal. En Open3D esto se logra con

`remove_statistical_outlier()` seguido de `cluster_dbscan()`, y en MeshLab con el filtro `Remove Isolated Pieces (wrt Diameter)`.

El criterio es retener únicamente la componente conexa más grande (o las n más grandes si hay varias partes legítimas del objeto). Un diámetro umbral de $\leq 1\%$ de la envolvente del objeto suele ser efectivo para eliminar fragmentos espurios.

Advertencia: efectos colaterales del filtrado

El filtrado elimina puntos *de manera irreversible*. Un filtrado excesivamente agresivo ($\alpha \leq 1.0$ o componentes conexas con umbral muy alto) puede:

- Eliminar puntos legítimos en aristas vivas y bordes de la pieza.
- Reducir la densidad en zonas de curvatura alta (cavidades, radios).
- Crear huecos artificiales que luego se interpretarán como falta de cobertura.

Siempre comparar visualmente la nube antes y después del filtrado y documentar los parámetros utilizados.

3.2 Estimación de normales y submuestreo

Dos operaciones esenciales antes del registro y la reconstrucción son la **estimación de vectores normales** y el **submuestreo** de la nube para homogeneizar la densidad y reducir el costo computacional.

3.2.1 Submuestreo por vóxeles (*voxel grid downsampling*)

El submuestreo por vóxeles divide el espacio en una cuadrícula 3D regular con celdas cúbicas de lado v (*voxel size*). Todos los puntos dentro de cada celda se reemplazan por su **centroide**:

$$\mathbf{p}_{\text{voxel}} = \frac{1}{n_j} \sum_{i=1}^{n_j} \mathbf{p}_i \quad (3.2)$$

donde n_j es el número de puntos en el vóxel j .

Analogía: muestreo de una señal temporal

El submuestreo por vóxeles es el análogo 3D del muestreo temporal de una señal: el tamaño de vóxel v es equivalente al período de muestreo T_s . Si v es demasiado grande, se pierde detalle geométrico (*aliasing* espacial); si es demasiado pequeño, no se gana información útil y se incrementa el costo computacional. La regla práctica es $v \approx 2-5 \times$ la resolución del escáner.

Cuadro 3.2: Guía para la selección del tamaño de vóxel según la aplicación.

Tamaño de vóxel v	Reducción típica	Aplicación
0.1 mm	~ 20%	Alta fidelidad: inspección metrológica, validación dimensional.
0.5 mm	~ 70–80%	Compromiso estándar: registro ICP, visualización, mallado Poisson.
1.0 mm	~ 90–95%	Registro grueso, visualización rápida, pruebas de algoritmos.
2.0 mm	> 95%	Solo para objetos grandes (> 1 m) o prealineación aproximada.

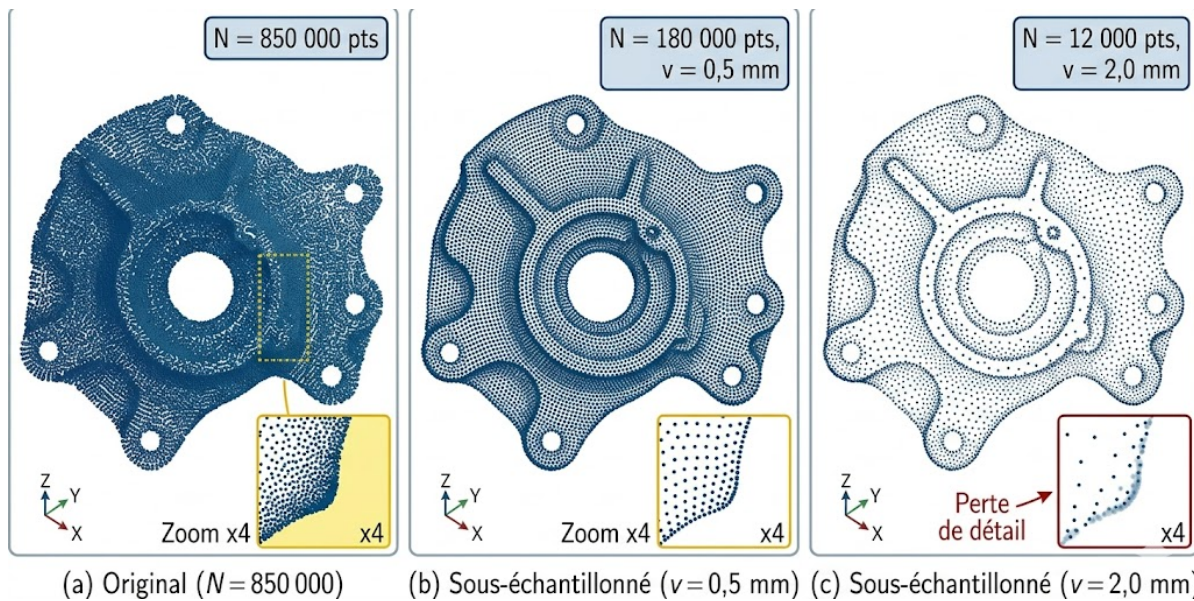


Figura 3.2: Efecto del submuestreo por vóxeles sobre la nube de puntos. **(a)** Nube original: $N = 850\,000$ puntos, densidad irregular (zonas de alta densidad cerca de la cámara, baja densidad en zonas alejadas). **(b)** Submuestreo con $v = 0.5$ mm: $N = 180\,000$ puntos, densidad homogénea. **(c)** Submuestreo con $v = 2.0$ mm: $N = 12\,000$ puntos, pérdida visible de detalle en aristas y radios. Se muestra una zona de detalle ampliada (*inset*) para cada caso.

3.2.2 Estimación de normales

El vector normal $\hat{\mathbf{n}}_i$ en cada punto $\mathbf{p}_i$ se estima ajustando un plano local al vecindario del punto. El procedimiento estándar es:

1. Seleccionar los k vecinos más cercanos de $\mathbf{p}_i$ (o todos los puntos dentro de un radio r).
2. Calcular la **matriz de covarianza** del vecindario:

$$\mathbf{C}_i = \frac{1}{k} \sum_{j=1}^k (\mathbf{p}_j - \bar{\mathbf{p}})(\mathbf{p}_j - \bar{\mathbf{p}})^\top \quad (3.3)$$

donde $\bar{\mathbf{p}}$ es el centroide del vecindario.

3. Calcular los **eigenvectores** de $\mathbf{C}_i$. El eigenvector asociado al eigenvalor más pequeño ($\lambda_{\min}$) es la normal $\hat{\mathbf{n}}_i$, pues corresponde a la dirección de mínima varianza (perpendicular al plano local).
4. **Orientar consistentemente:** las normales estimadas tienen ambigüedad de signo ($\pm \hat{\mathbf{n}}$). Se propaga una orientación coherente a partir de un punto semilla, asegurando que todas las normales apunten “hacia afuera” de la superficie.

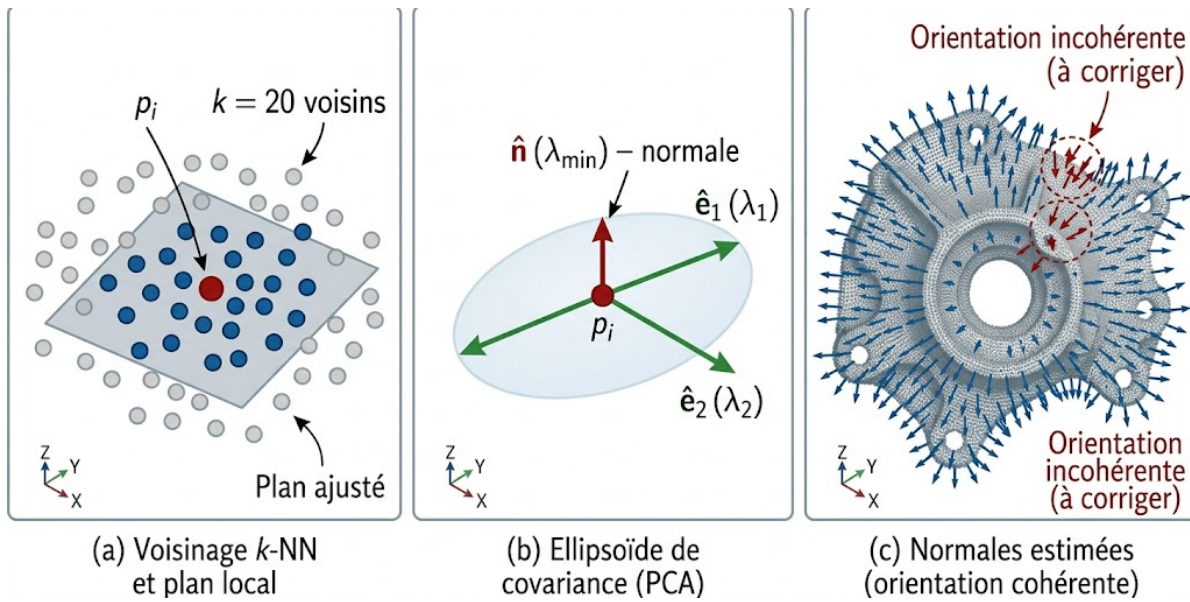


Figura 3.3: Estimación de normales por análisis de componentes principales (PCA) del vecindario local. **(a)** Detalle de la nube de puntos: un punto $\mathbf{p}_i$ (rojo) con su vecindario k -NN ($k = 20$, puntos azules) y el plano ajustado por mínimos cuadrados (gris semi-transparente). **(b)** Elipsoide de covarianza con los tres eigenvectores: $\hat{\mathbf{e}}_1$ y $\hat{\mathbf{e}}_2$ (tangentes al plano, flechas verdes) y $\hat{\mathbf{n}}$ (normal, flecha roja, perpendicular al plano). **(c)** Nube completa con normales visualizadas como pequeñas flechas azules, consistentemente orientadas hacia el exterior. Se señala una zona con orientación inconsistente (flechas rojas invertidas) antes de la corrección.

Importancia de las normales para el registro y mallado

Las normales son **requisito imprescindible** para:

- El registro ICP *point-to-plane* (Sección 3.3), que converge $\sim 10\times$ más rápido que *point-to-point*.
- La reconstrucción de Poisson (Tema 4), que resuelve una ecuación diferencial cuyo dato de entrada son las normales orientadas.
- La visualización con iluminación difusa (sin normales, la nube se ve como una masa uniforme sin relieve).

No omitir la estimación de normales antes de cualquier paso posterior.

3.3 Algoritmo ICP: formulación y convergencia

El **Iterative Closest Point** (ICP) [?] es el algoritmo estándar para alinear dos nubes de puntos que se solapan parcialmente. Dado un par de nubes:

- $\mathcal{S}$ (*source*): la nube que se va a mover.
- $\mathcal{T}$ (*target*): la nube fija (referencia).

ICP busca la transformación rígida $(\mathbf{R}, \mathbf{t})$ — rotación y traslación — que minimiza la distancia entre las dos nubes.

3.3.1 Variante *point-to-point*

La formulación original de Besl & McKay (1992) minimiza:

$$E_{p2p}(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^N \left\| \mathbf{R} \mathbf{p}_i^{(\mathcal{S})} + \mathbf{t} - \mathbf{q}_i^{(\mathcal{T})} \right\|^2 \quad (3.4)$$

donde $\mathbf{q}_i^{(\mathcal{T})}$ es el punto más cercano en $\mathcal{T}$ al punto transformado $\mathbf{R} \mathbf{p}_i^{(\mathcal{S})} + \mathbf{t}$.

3.3.2 Variante *point-to-plane*

La variante *point-to-plane* proyecta el error sobre la dirección de la normal del punto *target*:

$$E_{p2pl}(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^N \left[\left(\mathbf{R} \mathbf{p}_i^{(\mathcal{S})} + \mathbf{t} - \mathbf{q}_i^{(\mathcal{T})} \right) \cdot \hat{\mathbf{n}}_i \right]^2 \quad (3.5)$$

donde $\hat{\mathbf{n}}_i$ es la normal estimada en $\mathbf{q}_i$.

¿Por qué *point-to-plane* converge más rápido?

En una superficie plana, un desplazamiento tangencial no cambia la distancia perpendicular al plano. La variante *point-to-point* penaliza innecesariamente ese desplazamiento tangencial, requiriendo muchas iteraciones para “deslizar” las nubes hasta alinearlas. La variante *point-to-plane* solo penaliza la componente **normal**, permitiendo movimiento libre en la dirección tangente y convergiendo $\sim 10\times$ más rápido en superficies localmente planas. **Usar siempre *point-to-plane* cuando se disponga de normales** (que es el caso tras la Sección 3.2.2).

3.3.3 Esquema iterativo

El algoritmo ICP procede de forma iterativa:

1. **Encontrar correspondencias:** para cada punto de $\mathcal{S}$, buscar el vecino más cercano en $\mathcal{T}$ (búsqueda k -d tree, complejidad $\mathcal{O}(N \log N)$).
2. **Resolver transformación:** con las correspondencias fijas, minimizar la Ec. (3.4) o (3.5). La solución de *point-to-point* tiene forma cerrada (SVD); la de *point-to-plane* requiere linealización (asumiendo rotaciones pequeñas).
3. **Aplicar transformación:** actualizar $\mathcal{S} \leftarrow \mathbf{R} \mathcal{S} + \mathbf{t}$.
4. **Evaluar convergencia:** calcular el error RMS residual:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N d_i^2} \quad (3.6)$$

5. Repetir desde el paso 1 hasta que:
 - $|\text{RMS}_k - \text{RMS}_{k-1}| < \epsilon$ (típicamente $\epsilon = 10^{-6}$), o
 - se alcance el número máximo de iteraciones (típicamente 50–100).

3.3.4 Inicialización: el problema de los mínimos locales

ICP es un algoritmo de optimización **local**: converge al mínimo más cercano, que no necesariamente es el mínimo global. Si la posición inicial de $\mathcal{S}$ respecto a $\mathcal{T}$ está demasiado lejos, ICP converge a una alineación incorrecta.

Alineación manual (prealineación)

El método más simple es rotar y trasladar manualmente $\mathcal{S}$ hasta una posición “razonablemente cercana” a $\mathcal{T}$, usando las herramientas de manipulación del software (Edit $\rightarrow$ Manipulators Tool en MeshLab, o `translate()` / `rotate()` en Open3D). Es suficiente que el solapamiento sea visualmente reconocible.

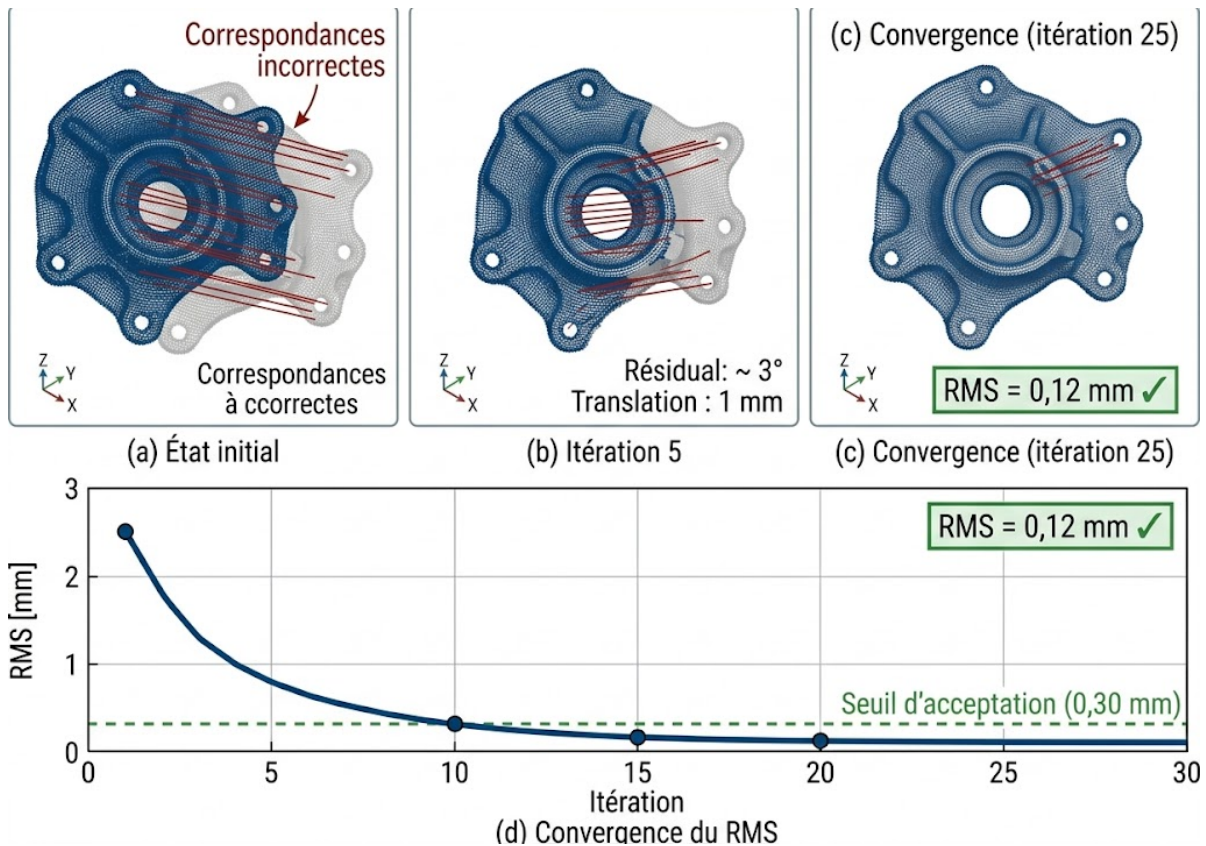


Figura 3.4: Esquema iterativo del algoritmo ICP. **(a)** Estado inicial: dos nubes desalineadas (S en azul, T en gris). Líneas rojas muestran las correspondencias iniciales (incorrectas en su mayoría). **(b)** Iteración 5: las nubes se han acercado parcialmente; las correspondencias mejoran. **(c)** Convergencia (iteración 25): las nubes están alineadas; las correspondencias son punto-a-punto cercanas. **(d)** Gráfica de convergencia: RMS [mm] vs. número de iteración, mostrando descenso rápido inicial y asíntota a $\text{RMS}_{\text{final}} \leq 0.30$ mm.

Alineación automática: descriptores FPFH + RANSAC

Para un registro automático robusto (sin intervención manual), se usa la siguiente secuencia:

1. **Calcular descriptores FPFH** (*Fast Point Feature Histograms*) para cada punto, que codifican la geometría local (relaciones angulares entre normales del vecindario) en un histograma de 33 dimensiones.
2. **Emparejar descriptores:** encontrar correspondencias entre $\mathcal{S}$ y $\mathcal{T}$ por similitud de FPFH (vecino más cercano en espacio de descriptores).
3. **RANSAC:** seleccionar aleatoriamente $n = 3$ correspondencias, calcular la transformación rígida, evaluar cuántos puntos satisfacen un umbral de distancia $d_{\max}$, y retener el modelo con más *inliers* tras M iteraciones.

El resultado de FPFH + RANSAC es una transformación gruesa que lleva a $\mathcal{S}$ “cerca” de $\mathcal{T}$, sirviendo como inicialización robusta para el refinamiento fino por ICP.

3.4 Registro global de múltiples vistas

En la práctica, un objeto se captura desde V vistas (típicamente 8–24). Cada par de vistas consecutivas se registra con ICP, produciendo una transformación $\mathbf{T}_{i \rightarrow i+1} = [\mathbf{R}|\mathbf{t}]$. El registro global consiste en llevar **todas** las vistas a un sistema de coordenadas común.

3.4.1 Encadenamiento de transformaciones

La estrategia más simple es seleccionar la vista 1 como referencia y aplicar las transformaciones acumuladas:

$$\mathbf{T}_{i \rightarrow 1} = \mathbf{T}_{2 \rightarrow 1} \cdot \mathbf{T}_{3 \rightarrow 2} \cdot \dots \cdot \mathbf{T}_{i \rightarrow i-1} \quad (3.7)$$

Advertencia: acumulación de error (*drift*)

El error de cada registro par-a-par se **acumula** a lo largo de la cadena. Para $V = 12$ vistas con RMS individual de 0.10 mm, el error acumulado en la última vista puede alcanzar $\sqrt{12} \times 0.10 \approx 0.35$ mm. Este efecto es especialmente grave cuando la cadena forma un **bucle cerrado** (360° alrededor del objeto): la vista final no coincide con la primera.

3.4.2 Cierre de bucle (*loop closure*)

Si la secuencia de vistas forma un bucle cerrado (como en una revolución completa alrededor del objeto), se puede distribuir el error acumulado uniformemente:

1. Registrar el par (vista V , vista 1) para obtener la transformación de cierre $\mathbf{T}_{V \rightarrow 1}$.
2. Comparar $\mathbf{T}_{V \rightarrow 1}$ con la transformación acumulada $\mathbf{T}_{V \rightarrow 1}^{\text{cadena}}$. La diferencia es el **error de cierre**.

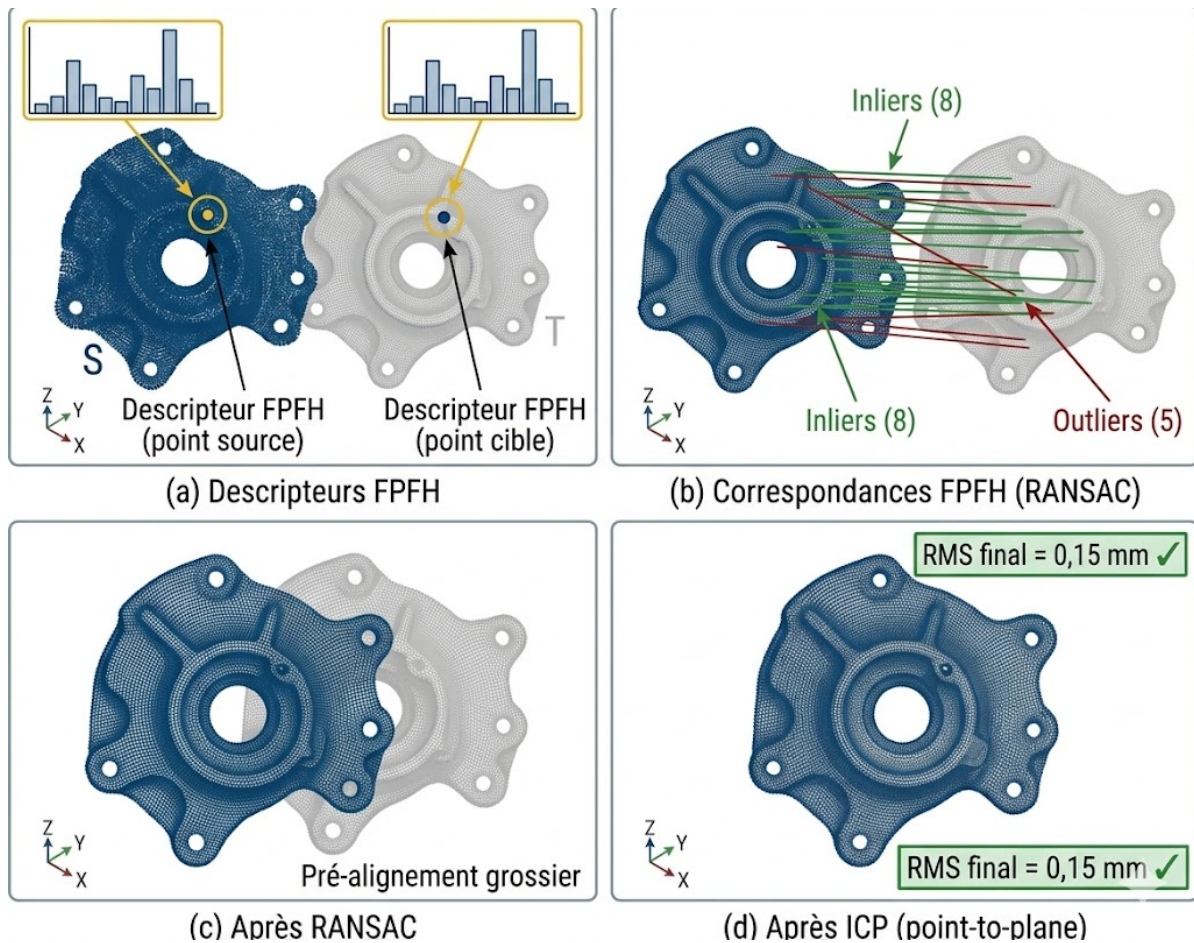


Figura 3.5: Inicialización del registro: FPFH + RANSAC seguido de ICP. **(a)** Dos nubes desalineadas (sin prealineación). Se muestran los descriptores FPFH como histogramas en dos puntos seleccionados. **(b)** Correspondencias FPFH (líneas verdes) entre las nubes, con *inliers* (verdes) y *outliers* (rojos) tras RANSAC. **(c)** Resultado de la prealineación RANSAC: las nubes están aproximadamente alineadas. **(d)** Refinamiento ICP *point-to-plane*: alineación final con $\text{RMS} \leq 0.30 \text{ mm}$.

3. Distribuir linealmente este error entre las $V - 1$ transformaciones intermedias (interpolación de la corrección).

En Open3D, la función `multiway_registration()` implementa una optimización global (*pose graph optimization*) que minimiza simultáneamente los errores de todos los pares, ponderados por la calidad de cada registro.

3.4.3 Verificación del registro global

El registro global se acepta cuando:

- El **error RMS global** (promedio de todos los pares) es ≤ 0.30 mm.
- No hay **discontinuidades visibles** en las zonas de solapamiento entre vistas consecutivas (verificar en CloudCompare: activar solo dos capas adyacentes y observar la superposición).
- El **error de cierre** (si aplica) es ≤ 0.50 mm.

3.4.4 Fusión de nubes (*merging*)

Una vez registradas todas las vistas, se fusionan en una nube unificada. La fusión implica:

1. Concatenar todos los puntos en un solo objeto.
2. Aplicar un submuestreo por vóxeles ($v = 0.1\text{--}0.5$ mm) para eliminar la **redundancia** en zonas de solapamiento (donde hay $2\times$ o $3\times$ la densidad nominal).
3. Recalcular las normales sobre la nube fusionada para asegurar consistencia global.

3.5 Filtrado y registro: Vía A (Python con Open3D)

El siguiente script implementa el pipeline completo de preprocesamiento y registro para un par de vistas:

```
import open3d as o3d
import numpy as np
import copy

# ==== 1. CARGAR NUBES ====
source = o3d.io.read_point_cloud("vista_01.ply")
target = o3d.io.read_point_cloud("vista_02.ply")
print(f"Source: {len(source.points)} pts")
print(f"Target: {len(target.points)} pts")

# ==== 2. FILTRADO SOR ====
def filtrar_SOR(pcd, k=20, alpha=2.0):
    cl, ind = pcd.remove_statistical_outlier(
        nb_neighbors=k, std_ratio=alpha)
    n_removed = len(pcd.points) - len(ind)
```

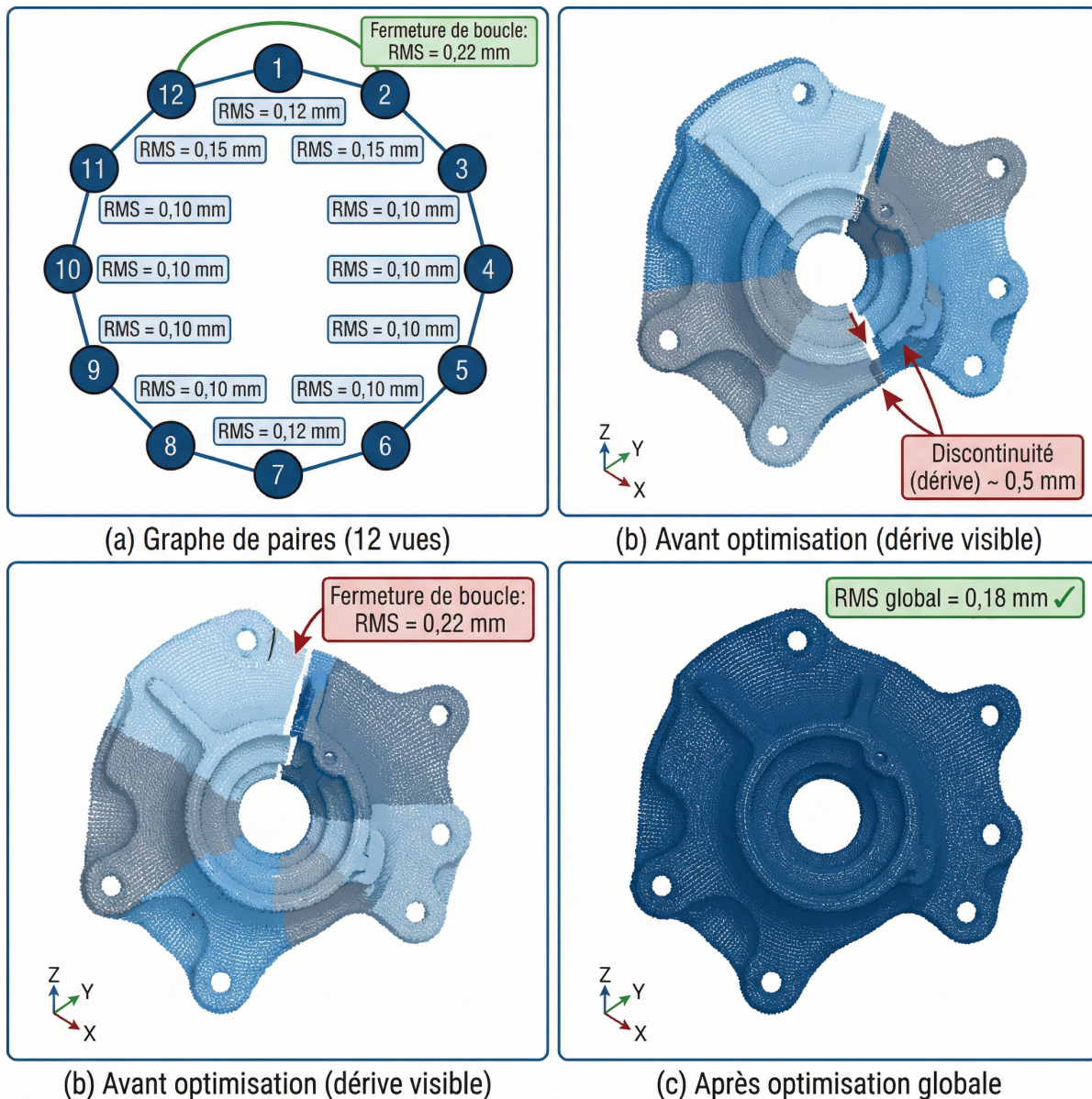



Figura 3.6: Registro global de múltiples vistas. **(a)** Grafo de pares: cada nodo es una vista, cada arista un registro ICP con su RMS. La arista de cierre de bucle (vista $V \rightarrow$ vista 1) se muestra en verde. **(b)** Antes de la optimización global: las 12 nubes registradas muestran una discontinuidad visible en la zona de cierre (flechas rojas). **(c)** Después de la optimización global: la nube unificada es continua, sin discontinuidades. $RMS \text{ global} = 0.18 \text{ mm}$.

```
print(f" SOR: {n_removed} outliers eliminados "
      f"({100*n_removed/len(pcd.points):.1f}%)")
return cl

source = filtrar_SOR(source)
target = filtrar_SOR(target)

# ==== 3. SUBMUESTREO POR VÓXELES ====
voxel_size = 0.5 # mm
source_ds = source.voxel_down_sample(voxel_size)
target_ds = target.voxel_down_sample(voxel_size)
print(f"Source submuestreada: {len(source_ds.points)} pts")
print(f"Target submuestreada: {len(target_ds.points)} pts")

# ==== 4. ESTIMACIÓN DE NORMALES ====
radius_normal = voxel_size * 3
for pcd in [source_ds, target_ds]:
    pcd.estimate_normals(
        o3d.geometry.KDTreeSearchParamHybrid(
            radius=radius_normal, max_nn=30))

# ==== 5. DESCRIPTORES FPFH ====
radius_fpfh = voxel_size * 7
def calcular_fpfh(pcd):
    return o3d.pipelines.registration\
        .compute_fpfh_feature(
            pcd,
            o3d.geometry.KDTreeSearchParamHybrid(
                radius=radius_fpfh, max_nn=100))

fpfh_src = calcular_fpfh(source_ds)
fpfh_tgt = calcular_fpfh(target_ds)

# ==== 6. REGISTRO GRUESO: RANSAC ====
dist_threshold = voxel_size * 2.0
result_ransac = o3d.pipelines.registration\
    .registration_ransac_based_on_feature_matching(
        source_ds, target_ds,
        fpfh_src, fpfh_tgt,
        mutual_filter=True,
        max_correspondence_distance=dist_threshold,
        estimation_method=
            o3d.pipelines.registration
            .TransformationEstimationPointToPoint(),
        ransac_n=3,
```

```

checkers=[
o3d.pipelines.registration
.CorrespondenceCheckerBasedOnDistance(
dist_threshold)],
criteria=
o3d.pipelines.registration
.RANSACConvergenceCriteria(
100000, 0.999))
print(f"RANSAC fitness: {result_ransac.fitness:.3f}")

# ==== 7. REFINAMIENTO: ICP POINT-TO-PLANE ====
dist_icp = voxel_size * 1.0
result_icp = o3d.pipelines.registration\
.registration_icp(
source_ds, target_ds,
dist_icp,
result_ransac.transformation,
o3d.pipelines.registration
.TransformationEstimationPointToPlane(),
criteria=
o3d.pipelines.registration
.ICPConvergenceCriteria(
max_iteration=50))
print(f"ICP RMS: {result_icp.inlier_rmse:.4f} mm")
print(f"ICP fitness: {result_icp.fitness:.3f}")

# ==== 8. APLICAR TRANSFORMACIÓN Y FUSIONAR ====
source_aligned = copy.deepcopy(source)
source_aligned.transform(result_icp.transformation)
merged = source_aligned + target
merged_ds = merged.voxel_down_sample(voxel_size)
print(f"Nube fusionada: {len(merged_ds.points)} pts")

# ==== 9. EXPORTAR ====
o3d.io.write_point_cloud("nube_registrada.ply", merged_ds)

# ==== 10. VISUALIZAR ====
source_temp = copy.deepcopy(source_ds)
source_temp.paint_uniform_color([0.0, 0.2, 0.6])
target_ds.paint_uniform_color([0.7, 0.7, 0.7])
source_temp.transform(result_icp.transformation)
o3d.visualization.draw_geometries(
[source_temp, target_ds],
window_name="Registro ICP - resultado")

```


Extensión a múltiples vistas

Para $V > 2$ vistas, encapsular el registro en un bucle que procese cada par (v_i, v_{i+1}) secuencialmente, acumulando las transformaciones. Para optimización global con cierre de bucle, usar `o3d.pipelines.registration.PoseGraph()` junto con `global_optimization()`. El repositorio de Open3D incluye un tutorial de registro multivista: http://www.open3d.org/docs/release/tutorial/pipelines/multiway_registration.html

3.6 Filtrado y registro: Vía B (MeshLab, sin programación)

A continuación se detalla el flujo equivalente en MeshLab para usuarios que prefieren una interfaz gráfica sin programación.

3.6.1 Paso 1: Importar las nubes parciales

Cargar cada vista como una capa independiente: `File` → `Import Mesh` → seleccionar `vista_01.ply`. Repetir para cada vista. Verificar en el panel `Layer Dialog` que aparecen V capas.

3.6.2 Paso 2: Filtrado de *outliers*

1. Eliminar piezas aisladas por diámetro:

`Filters` → `Cleaning and Repairing` → `Remove Isolated Pieces (wrt Diameter)`.

Parámetro: `Min Component Diameter` = 5–10 mm (elimina islas menores que este diámetro).

2. Filtrado estadístico SOR:

`Filters` → `Cleaning and Repairing` → `Remove Outliers (Statistical Outlier Removal)`.

Parámetros:

- `k-NN neighbors` = 20
- `StdDev Multiplier` = 2.0

Aplicar a **cada capa** individualmente (seleccionar la capa activa en el `Layer Dialog` antes de ejecutar).

3.6.3 Paso 3: Estimación de normales

`Filters` → `Normals, Curvatures and Orientation` → `Compute Normals for Point Sets`.

Parámetros:

- `Neighbor num` = 10–20 (más vecinos = normales más suaves pero menor sensibilidad a detalles).
- `Smooth iteration` = 2 (suavizado para coherencia de orientación).

Verificar: `Render` → `Show Vertex Normals`. Las flechas deben apuntar hacia afuera.

Si las normales están invertidas: `Filters` → `Normals, Curvatures and Orientation` → `Invert Faces Orientation` (o `Re-Orient All Faces Coherently`).

3.6.4 Paso 4: Submuestreo

`Filters` → `Sampling` → `Poisson-disk Sampling`.

Parámetro: `Number of samples` = el número deseado de puntos (por ejemplo, 100 000 para una nube originalmente de 500 000). Alternativamente, activar `Base Mesh Subsampling` para mantener la distribución adaptativa.

3.6.5 Paso 5: Registro ICP par-a-par

1. **Alineación manual inicial** (si las nubes están muy desalineadas):

`Edit` → `Manipulators Tool`. Seleccionar la nube *source* y rotarla/trasladarla manualmente con el ratón hasta que se superponga visualmente con la nube *target*.

2. **Registro ICP automático:**

`Filters` → `Registration` → `Align (ICP)`.

Parámetros:

- `ICP method`: `Rigid matching (point-to-plane)`
- `Max iterations`: 50
- `Overlap estimation`: 80% (fracción estimada de solapamiento entre las dos nubes).
- `Sample num`: 1 000–5 000 (más muestras = más preciso pero más lento).

3. **Verificar el resultado:** el log de MeshLab muestra el RMS del registro. Verificar que sea ≤ 0.30 mm.

4. **Repetir** para cada par consecutivo de vistas.

3.6.6 Paso 6: Fusionar capas

Una vez todos los pares registrados:

`Filters` → `Mesh Layer` → `Flatten Visible Layers`.

Esto fusiona todas las capas visibles en una sola. Marcar la opción `Merge Only Visible Layers`. La nube resultante es el insumo para el mallado (Tema 4).

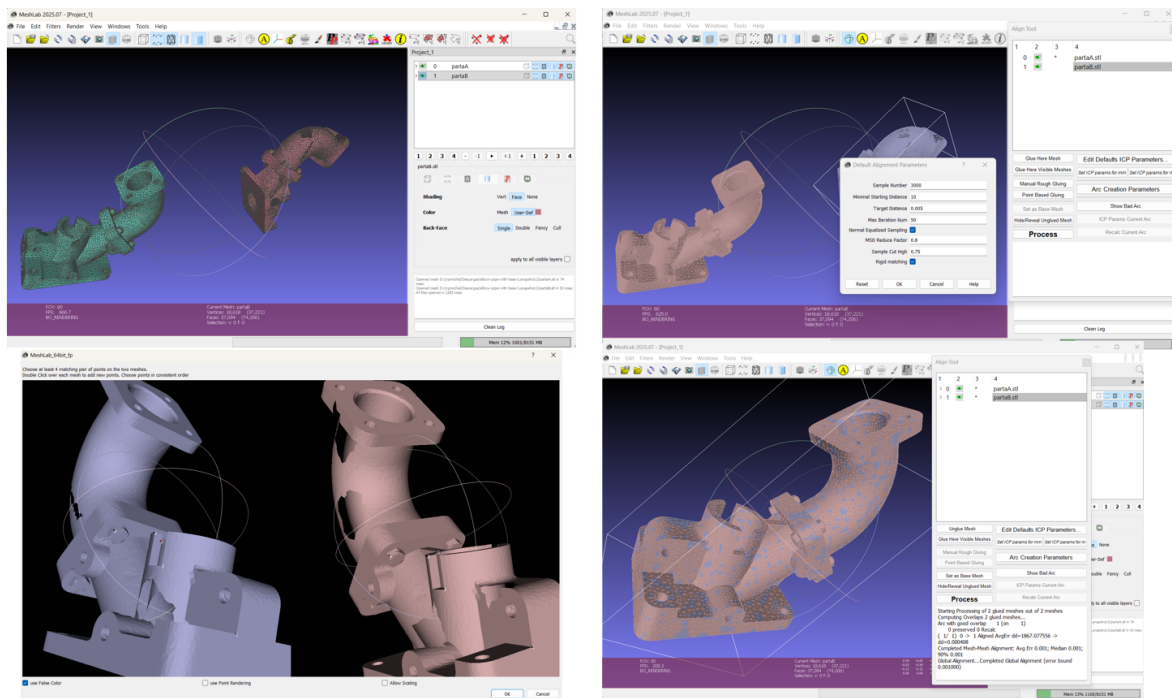


Figura 3.7: Registro ICP en MeshLab. La interfaz muestra dos nubes de puntos superpuestas (colores distintos para cada capa). El diálogo **Align** es visible con los parámetros ICP *point-to-plane*. En el log inferior se lee el RMS del resultado.

Cuadro 3.3: Comparación de los flujos de preprocesamiento y registro.

Aspecto	Vía A (Python/Open3D)	Vía B (MeshLab)
Automatización	Completamente programática; scripts reutilizables y reproducibles.	Interactiva; requiere intervención manual en cada paso.
Registro grueso	FPFH + RANSAC (automático, sin prealineación manual).	Manipulators Tool (manual) + ICP.
Registro global	PoseGraph + optimización global con cierre de bucle.	Registro secuencial par-a-par sin corrección de <i>drift</i> .
Reproducibilidad	Total: mismos parámetros $\Rightarrow$ mismo resultado.	Parcial: la prealineación manual introduce variabilidad.
Curva de aprendizaje	Requiere programación Python.	Solo interfaz gráfica; accesible sin programación.
Recomendación	Proyectos que requieran automatización, trazabilidad o > 10 vistas.	Proyectos puntuales, pocas vistas, usuarios sin experiencia en código.

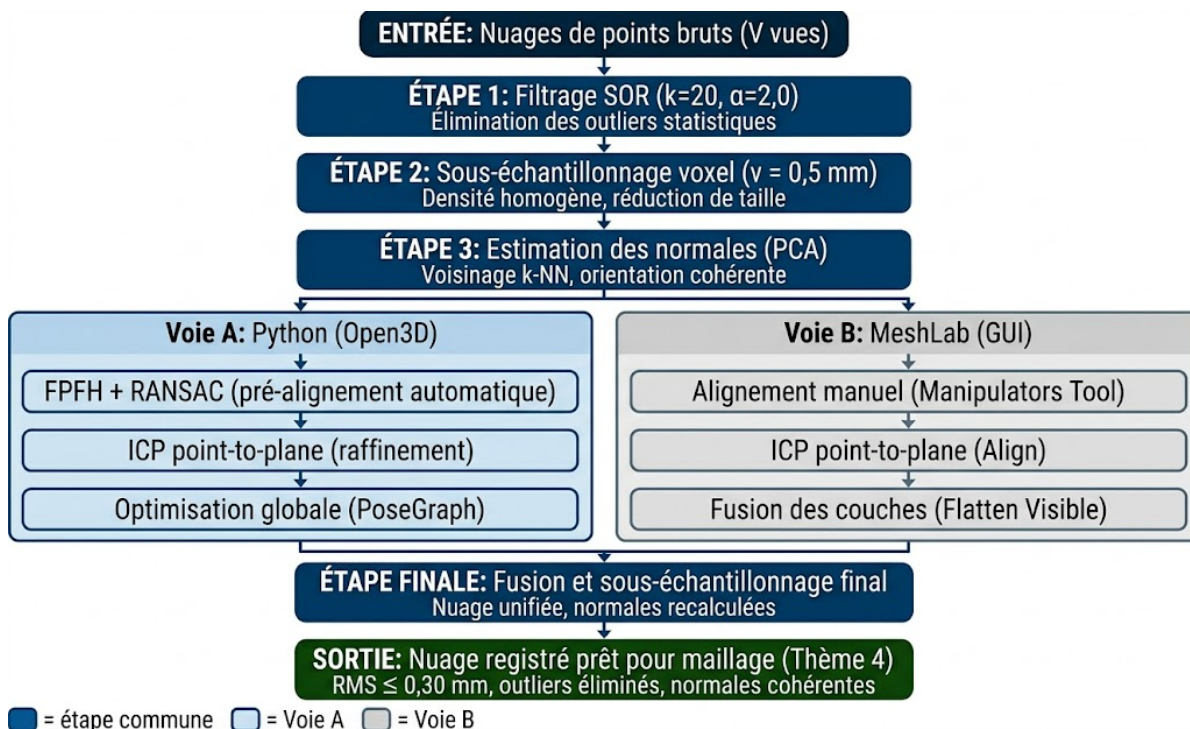


Figura 3.8: Diagrama de flujo del pipeline de preprocesamiento y registro, mostrando las dos vías (A: Python, B: MeshLab) en paralelo. Entrada: nubes de puntos crudas de V vistas. Etapas: filtrado SOR → submuestreo → normales → registro (FPFH/RANSAC + ICP en Vía A, manual + ICP en Vía B) → fusión → nube registrada. Salida: nube unificada lista para mallado (Tema 4).

3.6.7 Resumen comparativo: Vía A vs. Vía B

Evidencia de aprendizaje	Nube de puntos registrada (archivo PLY unificado) con reporte de error RMS por par de vistas y RMS global. Bitácora de parámetros de filtrado, submuestreo y registro.
Instrumento de evaluación	Checklist de verificación de preprocesamiento y registro.
Escala / Criterio de aceptación	RMS de registro ≤ 0.30 mm por par; <i>outliers</i> eliminados ($< 1\%$ del total); normales consistentes (verificación visual). Umbral: 80/100.

Capítulo 4

Mallado, Reconstrucción de Poisson y Exportación

Duración: 4 h

Modalidad: Presencial (laboratorio)

La nube de puntos registrada obtenida en el Tema 3 es una colección discreta de coordenadas 3D. Para su uso en ingeniería — simulación FEA, manufactura aditiva, comparación dimensional — es necesario convertirla en una **mallá triangular** (*mesh*) que represente una superficie continua. Este capítulo presenta los algoritmos de reconstrucción (Poisson, Ball-Pivoting), las operaciones de validación, suavizado y simplificación, y la exportación a formatos neutros. Al finalizar, el participante generará una mallá **estanca** (*watertight*) lista para el Nivel Intermedio o para uso directo en manufactura aditiva.

4.1 Reconstrucción de superficies de Poisson

El algoritmo de **Screened Poisson Surface Reconstruction** [?] es el método más utilizado para reconstruir superficies cerradas a partir de nubes de puntos orientadas (con normales). Su principio es resolver una ecuación diferencial parcial que genera una función indicadora cuya isosuperficie cero define la mallá.

4.1.1 Formulación variacional

Dada una nube de puntos con normales orientadas $\{(\mathbf{p}_i, \hat{\mathbf{n}}_i)\}$, se define un **campo vectorial** $\vec{V}(\mathbf{x})$ que interpola las normales en el espacio. La superficie buscada $\mathcal{S}$ es aquella cuyo gradiente de la función indicadora $\chi(\mathbf{x})$ se aproxima a $\vec{V}$:

$$\min_{\chi} \left\| \nabla \chi - \vec{V} \right\|^2 \iff \nabla^2 \chi = \nabla \cdot \vec{V} \quad (4.1)$$

La Ecuación (4.1) es una **ecuación de Poisson** (de ahí el nombre del algoritmo). Se resuelve discretizando el espacio mediante un *octree* adaptativo: las regiones con alta densidad de puntos se subdividen más finamente.

La versión *Screened* agrega un término de regularización que fuerza la superficie a pasar cerca de los puntos originales:

$$\min_{\chi} \left\| \nabla \chi - \vec{V} \right\|^2 + \lambda \sum_{i=1}^N (\chi(\mathbf{p}_i) - s_i)^2 \quad (4.2)$$

donde λ es el *point weight* (peso del ajuste a los datos) y s_i es el valor de la función indicadora en cada punto muestra.

4.1.2 Parámetros clave

Cuadro 4.1: Parámetros del algoritmo Screened Poisson y su efecto.

Parámetro	Rango típico	Efecto
depth	8–10	Resolución de la malla: 2^{depth} celdas por eje del octree. depth= 8: adecuado para piezas de 5–20 cm. depth= 10: más detalle, $\sim 8\times$ más memoria y tiempo.
pointWeight (λ)	1.0–4.0	$\lambda = 0$: Poisson clásico (suave, puede alejarse de los datos). $\lambda = 4$: se ajusta estrechamente a la nube (más ruido visible). Compromiso: $\lambda = 2.0$.
samplesPerNode	1.0–5.0	Mínimo de puntos por celda del octree para subdividir. Valores bajos = más detalle; valores altos = más suavizado.
adaptiveOctreeDepth	5–6	Profundidad para estimación adaptativa de densidad. Dejar en 6.

Analogía: la ecuación de Poisson como un campo gravitatorio

Imagine que cada punto de la nube emite un “campo” en la dirección de su normal. La reconstrucción de Poisson busca la superficie que mejor se alinea con ese campo, de forma análoga a una superficie equipotencial en un campo gravitatorio. Las zonas con muchos puntos y normales coherentes generan una superficie bien definida; las zonas sin datos producen una superficie “interpolada” que puede alejarse de la geometría real.

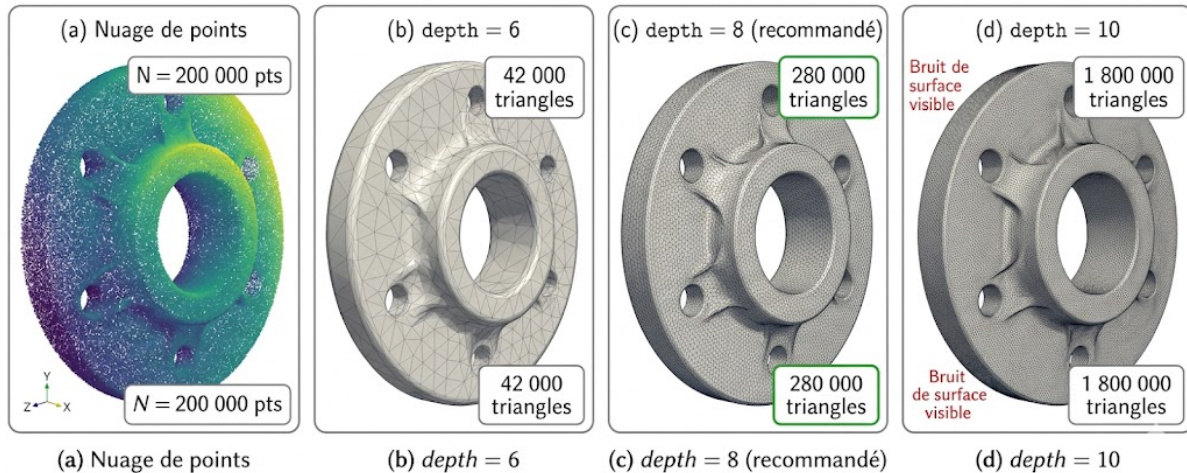


Figura 4.1: Reconstrucción de Poisson con diferentes valores de depth. **(a)** Nube de puntos registrada (entrada, $N = 200\,000$ puntos, falso color por profundidad). **(b)** $\text{depth} = 6$: malla suave pero sin detalle fino; aristas redondeadas, cavidades rellenas. **(c)** $\text{depth} = 8$: compromiso — aristas razonablemente definidas, superficie continua. **(d)** $\text{depth} = 10$: alta resolución, aristas nítidas pero ruido superficial visible (textura rugosa). Para cada caso se indica el número de triángulos resultante.

Advertencia: Poisson genera superficies cerradas

El algoritmo de Poisson **siempre** genera una malla cerrada, incluso donde no hay datos (bordes de la nube, partes inferiores no escaneadas). La superficie extrapolada en zonas sin datos forma una “cáscara” falsa que debe eliminarse. Esto se hace recortando por la calidad del campo escalar (Scalar Field $\rightarrow$ Quality en MeshLab, o filtrando por density en Open3D). **No confiar en la malla de Poisson sin recortar** — las zonas extrapoladas contaminan las métricas dimensionales.

4.2 Algoritmos alternativos: Ball-Pivoting y Marching Cubes

4.2.1 Ball-Pivoting Algorithm (BPA)

El algoritmo **Ball-Pivoting** [?] genera una malla triangular “rodando” una esfera virtual de radio ρ sobre la nube de puntos. Tres puntos que la esfera toca simultáneamente definen un triángulo; la esfera pivotea sobre cada arista para encontrar el siguiente triángulo.

- **Parámetro principal:** radio de la esfera ρ . Si ρ es demasiado pequeño, se generan huecos donde los puntos están más separados. Si ρ es demasiado grande, se pierden detalles finos.
- **Valor por defecto:** $\rho = 0$ (automático) calcula un radio basado en la distancia media entre vecinos ($\rho \approx 2\bar{d}$).

- **Multi-resolución:** ejecutar con varios radios crecientes (por ejemplo, $\rho_1 = \bar{d}$, $\rho_2 = 2\bar{d}$, $\rho_3 = 4\bar{d}$) para cerrar huecos progresivamente.

Cuadro 4.2: Comparación de algoritmos de reconstrucción de superficies.

Criterio	Screened Poisson	Ball-Pivoting	Marching Cubes
Tipo de superficie	Cerrada (siempre <i>watertight</i>)	Abierta (sigue los datos)	Cerrada (isosuperficie)
Requisito	Normales orientadas (obligatorio)	Normales (opcionales)	Función de distancia implícita
Manejo de ruido	Bueno (suavizado implícito)	Sensible (reproduce el ruido)	Moderado
Huecos en la nube	Interpola (puede generar cáscaras falsas)	Deja huecos donde no hay datos	Interpola (según función de distancia)
Parámetros	depth, λ	Radio ρ	Resolución de vóxel, umbral
Velocidad	Moderada	Rápida	Rápida
Recomendación	Uso general; mejor para piezas cerradas	Datos limpios, alta densidad, mallas abiertas	Datos volumétricos (CT, MRI)

4.2.2 Marching Cubes

El algoritmo **Marching Cubes** discretiza el espacio en una cuadrícula de vóxeles y evalúa una función de distancia implícita en cada vértice de la cuadrícula. La isosuperficie $f(\mathbf{x}) = 0$ se extrae triangulando las celdas que cruzan el cambio de signo. Es más utilizado en datos volumétricos (CT, MRI) que en nubes de puntos de escaneo, pero está disponible como alternativa en Open3D y MeshLab.

¿Cuándo usar cada algoritmo?

- **Screened Poisson:** opción por defecto para la mayoría de escenarios de ingeniería inversa. Genera superficies cerradas, tolera ruido y densidades no uniformes.
- **Ball-Pivoting:** cuando se necesita una malla que represente *fielmente* los datos sin interpolar — por ejemplo, para verificar la cobertura real antes de cerrar huecos manualmente.
- **Marching Cubes:** datos volumétricos o cuando se dispone de una función de distancia con signo (*Signed Distance Function*, SDF).

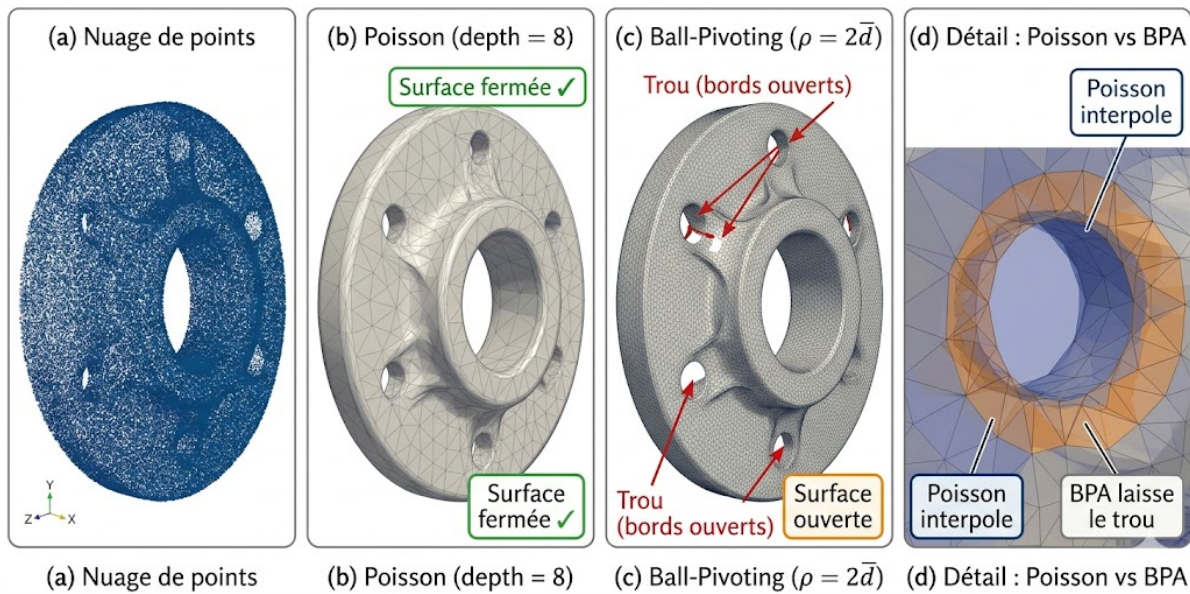


Figura 4.2: Comparación visual de algoritmos de reconstrucción de superficies. **(a)** Nube de puntos original (entrada común). **(b)** Screened Poisson (depth= 8): superficie cerrada, suave, aristas definidas. **(c)** Ball-Pivoting ($\rho = 2\bar{d}$): superficie abierta con huecos donde la densidad es insuficiente (flechas rojas señalan bordes abiertos). **(d)** Detalle de la zona del hueco: vista ampliada mostrando triángulos abiertos de BPA vs. superficie interpolada de Poisson (superpuestas en semi-transparencia).

4.3 Validación topológica de la malla

Una malla apta para ingeniería debe ser **topológicamente correcta**. Los defectos más comunes y sus consecuencias son:

4.3.1 Verificación y reparación

La verificación de la malla se realiza en dos fases:

1. **Detección:** identificar los defectos presentes.
2. **Reparación:** corregir cada defecto de forma automática o manual.

Los criterios de aceptación para una malla estanca son:

$$\text{Malla estanca (watertight)} \iff \begin{cases} \text{Aristas no-manifold} = 0 \\ \text{Vértices no-manifold} = 0 \\ \text{Huecos} = 0 \text{ (o cerrados)} \\ \text{Caras degeneradas} = 0 \\ \text{Auto-intersecciones} = 0 \end{cases} \quad (4.3)$$

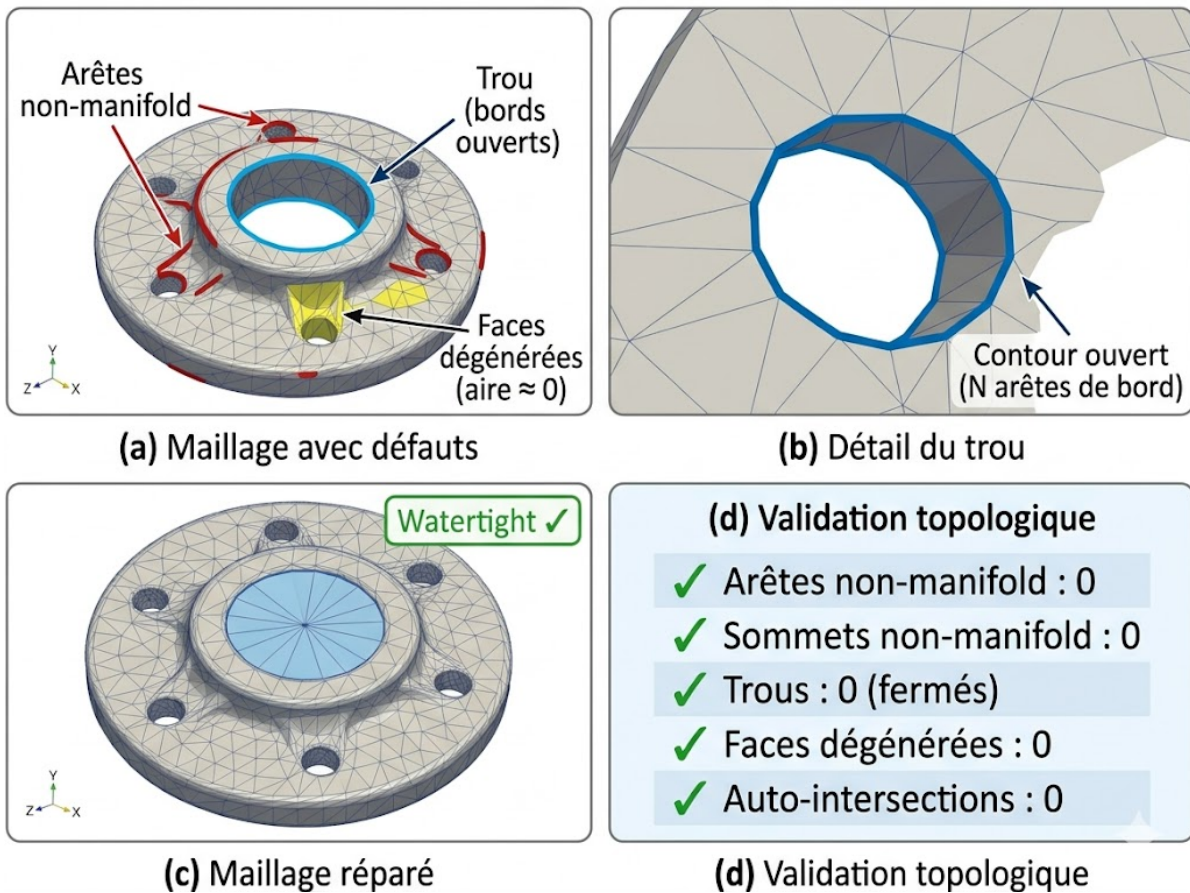
Fig. 4.3 : Validation topologique du maillage après réparation

Figura 4.3: Defectos topológicos en una malla y su reparación. **(a)** Malla con defectos señalados: aristas no-manifold (rojo), huecos (azul), caras degeneradas (amarillo). **(b)** Detalle de un hueco: vista ampliada con bordes abiertos resaltados. **(c)** Malla reparada: huecos cerrados, defectos eliminados. **(d)** Checklist de validación superpuesto: todas las verificaciones con marca verde.

Cuadro 4.3: Defectos topológicos de mallas y su impacto.

Defecto	Descripción	Consecuencia
Aristas no-<i>manifold</i>	Una arista compartida por > 2 triángulos (“arista triple”).	Fallo en cálculo de normales, errores en <i>slicing</i> para manufactura aditiva.
Vértices no-<i>manifold</i>	Un vértice que conecta regiones topológicamente independientes (“vértice de pellizco”).	Problemas en suavizado y simplificación.
Huecos (<i>holes</i>)	Bordes abiertos que forman un contorno cerrado sin superficie.	La malla no es estanca; no define un volumen (manufactura aditiva rechaza el archivo).
Caras degeneradas	Triángulos con área ≈ 0 (vértices colineales) o con normal indefinida.	Errores numéricos en FEA y en cálculos de curvatura.
Caras duplicadas	Dos triángulos idénticos superpuestos.	Artefactos visuales; doble conteo en cálculos de área/volumen.
Auto-intersecciones	Triángulos que se cruzan entre sí (la superficie se “atraviesa” a sí misma).	Fallo en operaciones booleanas y en <i>slicing</i> .

4.4 Suavizado y simplificación

4.4.1 Suavizado Laplaciano

El **suavizado Laplaciano** desplaza cada vértice hacia el centroide de sus vecinos:

$$\mathbf{v}'_i = \mathbf{v}_i + \lambda \left(\frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{v}_j - \mathbf{v}_i \right) \quad (4.4)$$

donde $\lambda \in (0, 1]$ es el factor de suavizado y $\mathcal{N}(i)$ es el conjunto de vértices adyacentes a $\mathbf{v}_i$.

- **Iteraciones:** 1–3 para suavizar ruido de alta frecuencia sin perder geometría macroscópica. Más de 5 iteraciones con $\lambda \geq 0.5$ difumina aristas vivas de forma inaceptable.
- **Suavizado de frontera:** el parámetro 1D *Boundary Smoothing Weight* controla si los bordes abiertos (si los hay) también se suavizan. Dejar en 1.0 si la malla es cerrada.

Advertencia: no sobre-suavizar aristas vivas

En piezas mecánicas, las aristas vivas (*sharp edges*) son **características geométricas funcionales** (bordes de chaflán, filos de ranura, aristas de división). Un suavizado excesivo las elimina, generando un modelo que no representa la pieza real. Si se necesita suavizar solo las zonas ruidosas sin afectar aristas, usar **suavizado bilateral** (*bilateral smoothing*), disponible en MeshLab como opción avanzada.

4.4.2 Decimación (*Quadric Edge Collapse*)

La malla generada por Poisson puede contener cientos de miles o millones de triángulos. La **decimación** reduce el número de triángulos preservando la geometría en la medida de lo posible. El algoritmo más utilizado es **Quadric Edge Collapse Decimation** (QEM):

1. Para cada arista, se calcula el **error cuadrático** Q que resultaría de colapsar esa arista a un solo vértice.
2. Se colapsa la arista con menor error Q .
3. Se repite hasta alcanzar el número objetivo de triángulos o un porcentaje de reducción.

$$\text{Ratio de simplificación} = \frac{N_{\text{final}}}{N_{\text{original}}} \times 100\% \quad (4.5)$$

Cuadro 4.4: Guía de ratios de simplificación según la aplicación.

Ratio	$N_{\text{triángulos}}$ típico	Aplicación
100% (sin decimación)	200 000–2 000 000	Archivo maestro, comparación CAD-scan (Tema 6).
50%	100 000–500 000	Simulación FEA (tras remallado adaptativo).
10–20%	20 000–100 000	Visualización web, catálogos 3D.
1–5%	5 000–20 000	Previsualización rápida, archivos ligeros.

4.5 Exportación en formatos neutros

La malla final debe exportarse en un formato compatible con el software destino. La Tabla 4.5 resume los formatos más utilizados.

STL binario vs. ASCII

STL puede ser **binario** o **ASCII**. Para mallas de $> 10\,000$ triángulos, usar siempre **binario**: el archivo es $\sim 5\times$ más pequeño y se carga $\sim 10\times$ más rápido. En MeshLab, al exportar: File $\rightarrow$ Export Mesh As $\rightarrow$ STL $\rightarrow$ desmarcar ASCII en las opciones. En Open3D: `o3d.io.write_triangle_mesh("pieza.stl", mesh)` exporta binario por defecto.

Fig. 4.4 : Validation des maillages après réparation

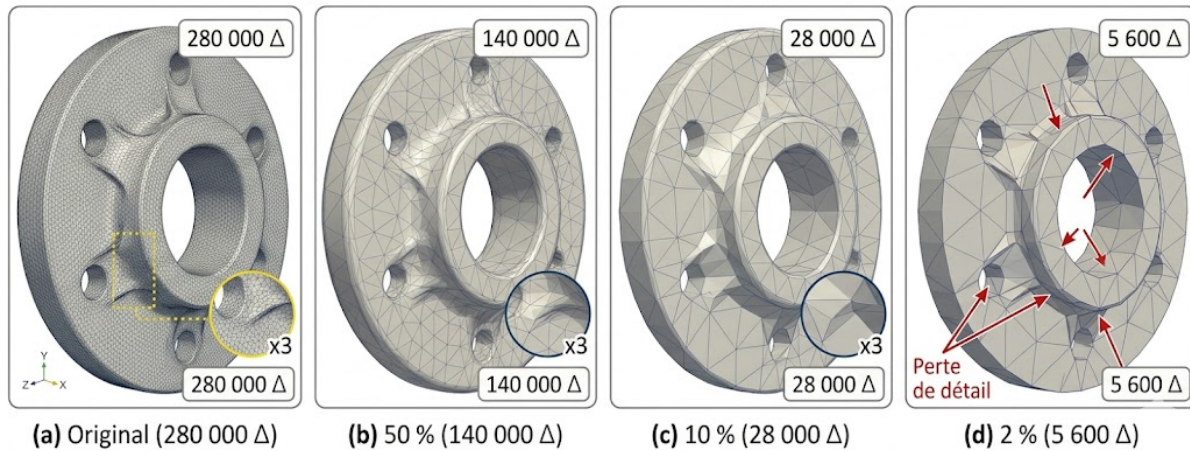


Figura 4.4: Efecto de la decimación QEM sobre la malla. **(a)** Malla original (280 000 triángulos): detalle de una zona curva mostrando triángulos finos y regulares. **(b)** 50% (140 000 triángulos): geometría preservada, triángulos más grandes en zonas planas. **(c)** 10% (28 000 triángulos): simplificación visible en curvaturas, aristas principales preservadas. **(d)** 2% (5 600 triángulos): geometría gruesa, pérdida de detalle en filetes y cavidades (señalados con flechas rojas).

Cuadro 4.5: Formatos de exportación de mallas para ingeniería inversa.

Formato	Extensión	Contenido	Aplicación típica
STL	.stl	Solo triángulos (sin normales por vértice, sin textura, sin color).	Manufactura aditiva (impresión 3D), intercambio universal.
OBJ	.obj	Triángulos + normales + coordenadas de textura (archivo .mtl asociado).	Visualización con textura, renderizado, realidad virtual.
PLY	.ply	Triángulos + normales + color por vértice. Formato binario o ASCII.	Intercambio entre software de procesamiento 3D (Open3D, MeshLab, CloudCompare).
STEP	.step	Superficies analíticas (NURBS, primitivas). No es un formato de malla triangular — requiere ajuste de superficies (Tema 6).	CAD/CAM, intercambio de modelos paramétricos, fabricación.
IGES	.igs	Similar a STEP pero más antiguo; soporte de superficies NURBS.	Compatibilidad con software CAD legacy.

4.6 Mallado y exportación: Vía A (Python con Open3D/Trimesh)

El siguiente script implementa el pipeline completo: reconstrucción de Poisson, recorte de cáscara falsa, validación, suavizado, decimación y exportación.

```
import open3d as o3d
import numpy as np
import trimesh

# ==== 1. CARGAR NUBE REGISTRADA ====
pcd = o3d.io.read_point_cloud("nube_registrada.ply")
print(f"Puntos: {len(pcd.points)}")

# Verificar normales
if not pcd.has_normals():
    pcd.estimate_normals(
        o3d.geometry.KDTreeSearchParamHybrid(
            radius=1.0, max_nn=30))
    pcd.orient_normals_consistent_tangent_plane(k=15)

# ==== 2. RECONSTRUCCIÓN DE POISSON ====
mesh, densities = o3d.geometry.TriangleMesh\
    .create_from_point_cloud_poisson(
        pcd, depth=8, width=0, scale=1.1,
        linear_fit=False)
print(f"Poisson: {len(mesh.triangles)} triángulos")

# ==== 3. RECORTE DE CÁSCARA FALSA ====
# Eliminar vértices con baja densidad (zonas extrapoladas)
densities = np.asarray(densities)
threshold = np.quantile(densities, 0.02)
vertices_to_remove = densities < threshold
mesh.remove_vertices_by_mask(vertices_to_remove)
print(f"Tras recorte: {len(mesh.triangles)} triángulos")

# ==== 4. VALIDACIÓN TOPOLÓGICA ====
mesh.remove_degenerate_triangles()
mesh.remove_duplicated_triangles()
mesh.remove_duplicated_vertices()
mesh.remove_non_manifold_edges()
print(f"Watertight: {mesh.is_watertight()}")
print(f"Orientable: {mesh.is_orientable()}")

# ==== 5. SUAVIZADO LAPLACIANO ====
mesh_smooth = mesh.filter_smooth_laplacian(
```

```
number_of_iterations=2, lambda_filter=0.5)
mesh_smooth.compute_vertex_normals()

# ==== 6. DECIMACIÓN QEM ====
target_faces = int(len(mesh_smooth.triangles) * 0.5)
mesh_dec = mesh_smooth.simplify_quadric_decimation(
    target_number_of_triangles=target_faces)
print(f"Decimada: {len(mesh_dec.triangles)} triángulos")
mesh_dec.compute_vertex_normals()

# ==== 7. EXPORTAR ====
o3d.io.write_triangle_mesh("pieza_poisson.stl", mesh_dec)
o3d.io.write_triangle_mesh("pieza_poisson.ply", mesh_dec)
print("Exportación completada: STL + PLY")

# ==== 8. VERIFICACIÓN CON TRIMESH ====
tm = trimesh.load("pieza_poisson.stl")
print(f"Trimesh watertight: {tm.is_watertight}")
print(f"Volumen: {tm.volume:.2f} mm^3")
print(f"Área superficial: {tm.area:.2f} mm^2")

# ==== 9. VISUALIZAR ====
o3d.visualization.draw_geometries(
    [mesh_dec],
    window_name="Malla final - Poisson + QEM",
    mesh_show_wireframe=True)
```

¿Por qué usar Trimesh además de Open3D?

Open3D proporciona la reconstrucción y el procesamiento principal. Trimesh complementa con verificaciones robustas: `is_watertight` aplica criterios más estrictos (incluyendo auto-intersecciones), y calcula `volume` y `area` con precisión. Es buena práctica verificar la malla con ambas bibliotecas antes de declararla apta.

4.7 Mallado y exportación: Vía B (MeshLab, sin programación)

A continuación se detalla el flujo paso a paso en MeshLab.

4.7.1 Paso 1: Reconstrucción de Poisson

Filters → Remeshing, Simplification and Reconstruction → Screened Poisson Surface Reconstruction.

Parámetros:

- Reconstruction Depth = 8–10 (comenzar con 8).
- Adaptive Octree Depth = 6.
- Samples per Node = 1.0.
- Point Weight = 4.0 (para ajuste cercano a los datos; reducir a 2.0 si hay ruido).

MeshLab genera una nueva capa con la malla reconstruida y un campo escalar Quality por vértice que indica la densidad de datos local.

Eliminar cáscara falsa: seleccionar la capa de la malla → Filters → Selection → Select Faces with Vertices with Quality Lower Than... → definir un umbral (inspeccionar el rango de Quality en el histograma del panel Properties) → Delete Selected Faces and Vertices.

4.7.2 Paso 2: Reconstrucción alternativa — Ball-Pivoting

Filters → Remeshing, Simplification and Reconstruction → Surface Reconstruction: Ball Pivoting.

Parámetros:

- Ball radius = 0 (automático; MeshLab calcula un radio basado en la nube).
- Clustering = 20%.
- Angle threshold = 90°.

4.7.3 Paso 3: Validación topológica

1. **Detectar aristas no-manifold:** Filters → Selection → Select Non-Manifold Edges. Si se seleccionan aristas → Delete Selected Faces and Vertices.
2. **Detectar vértices no-manifold:** Filters → Selection → Select Non-Manifold Vertices. Si se seleccionan vértices → eliminar.
3. **Cerrar huecos:** Filters → Cleaning and Repairing → Close Holes. Parámetro: Max hole size (en número de aristas del borde del hueco). Comenzar con 30; incrementar si quedan huecos grandes.
4. **Eliminar caras degeneradas y duplicadas:** Filters → Cleaning and Repairing → Remove Duplicate Faces. Filters → Cleaning and Repairing → Remove Zero Area Faces.

4.7.4 Paso 4: Suavizado

Filters → Smoothing, Fairing and Deformation → Laplacian Smooth.

Parámetros:

- Smoothing steps = 1–3.
- 1D Boundary Smoothing Weight = 1.0.

Precaución: verificar visualmente tras cada iteración. Si las aristas vivas se difuminan, reducir a 1 iteración o usar suavizado bilateral (Two Step Smoothing).

4.7.5 Paso 5: Decimación

Filters → Remeshing, Simplification and Reconstruction → Simplification: Quadric Edge Collapse Decimation.

Parámetros:

- Target number of faces: número deseado de triángulos (por ejemplo, 50% del total actual).
- Preserve Boundary = **on**.
- Preserve Normal = **on**.
- Planar Simplification = **on** (simplifica agresivamente zonas planas, preservando detalle en zonas curvas).

4.7.6 Paso 6: Exportación

File → Export Mesh As → seleccionar formato:

- **STL**: para manufactura aditiva. Desmarcar ASCII para exportar binario (más eficiente).
- **OBJ**: si se requiere textura o normales por vértice.
- **PLY**: para continuar procesamiento en Open3D o CloudCompare.

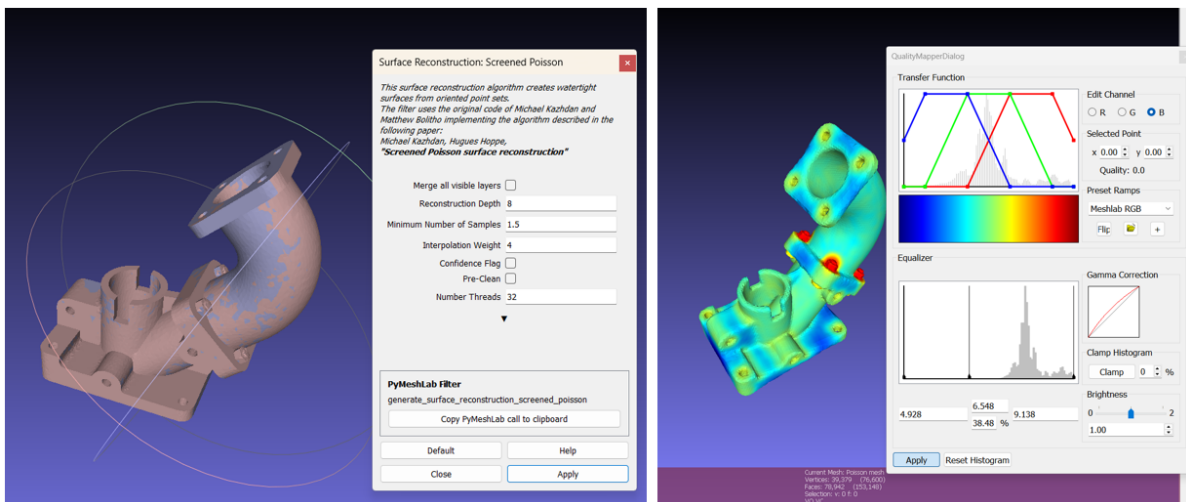


Figura 4.5: Reconstrucción de Poisson en MeshLab. Vista de la malla reconstruida con depth= 8 y el campo escalar Quality visualizado en colormap (azul = alta densidad, rojo = baja densidad / zona extrapolada). El panel lateral muestra el número de vértices y triángulos de la malla resultante.

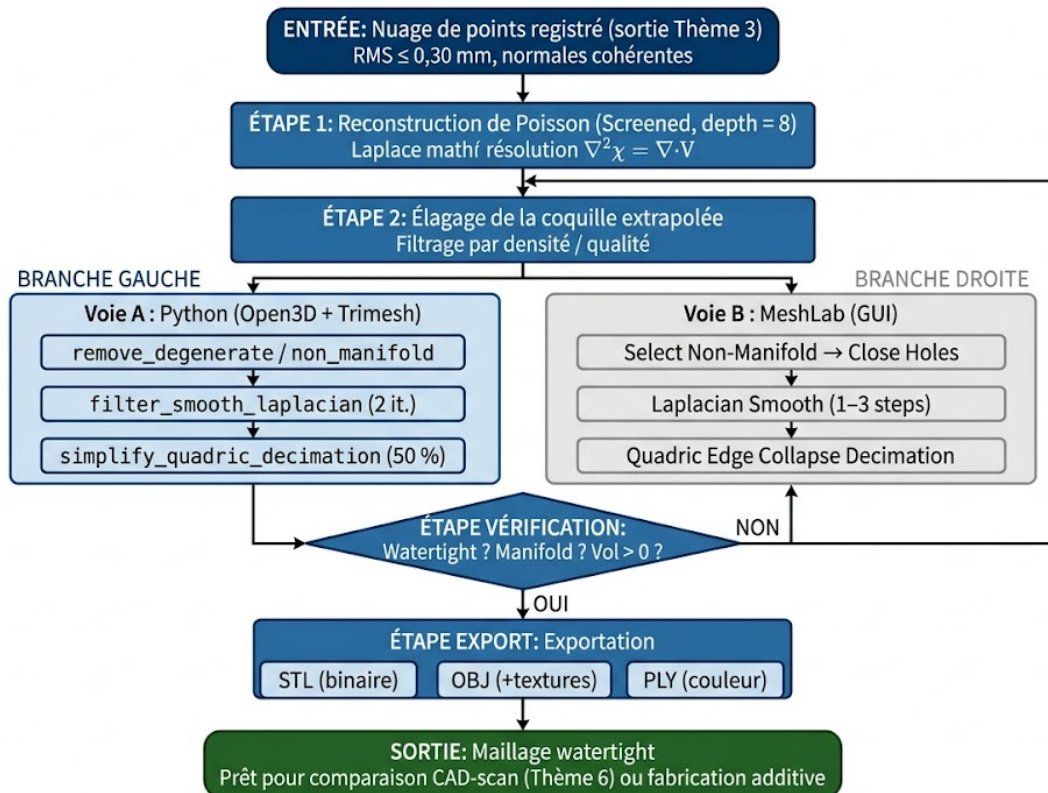


Figura 4.6: Diagrama de flujo del pipeline de mallado y exportación, mostrando las dos vías (A: Python, B: MeshLab) en paralelo. Entrada: nube de puntos registrada (salida del Tema 3). Etapas: Poisson → recorte → validación topológica → suavizado → decimación → exportación STL/OBJ/PLY. Salida: malla estanca lista para comparación CAD-scan (Tema 6) o manufactura aditiva.

Evidencia de aprendizaje	Informe técnico Nivel Básico: nube registrada (PLY), malla STL estanca (<i>watertight</i>), reporte de parámetros de Poisson (depth, λ), estadísticas de la malla (N triángulos, volumen, área), y capturas de pantalla de cada etapa.
Instrumento de evaluación	Rúbrica del Nivel Básico (captura 25%, preprocesamiento 25%, mallado 25%, informe 25%).
Escala / Criterio de aceptación	Cobertura $\geq 90\%$; RMS ≤ 0.30 mm; malla <i>watertight</i> (0 aristas no-manifold, 0 huecos); informe completo con capturas. Umbral: 80/100.

Parte II

Nivel Intermedio

Capítulo 5

Calibración Avanzada y Registro Multisesión

Duración: 6 h

Modalidad: Presencial (laboratorio)

El Nivel Básico (Temas 1–4) trabajó con los parámetros de cámara proporcionados por el fabricante o estimados automáticamente por Meshroom. El Nivel Intermedio exige un control metrológico riguroso: la **calibración intrínseca** de la cámara con el método de Zhang [?], la **calibración extrínseca** del par cámara–proyector (o cámara–cámara), y el **registro multisesión** para combinar capturas realizadas en diferentes días o con diferentes configuraciones. Adicionalmente, se introduce la **extracción de primitivas geométricas** (planos, cilindros, esferas) como paso previo a la reconstrucción NURBS (Tema 6). Al finalizar, el participante será capaz de calibrar un sistema 3D con error de reproyección < 0.75 px y extraer primitivas con $R^2 > 0.95$.

5.1 Calibración intrínseca con patrón de tablero

La calibración intrínseca estima los parámetros internos de la cámara — la **matriz de intrínsecos** $\mathbf{K}$ y los **coeficientes de distorsión** $(k_1, k_2, k_3, p_1, p_2)$ — a partir de múltiples imágenes de un patrón de geometría conocida (tablero de ajedrez).

5.1.1 El método de Zhang (2000)

El método de Zhang [?] utiliza un tablero plano observado desde $N \geq 15$ poses diferentes. Para cada imagen i , la relación entre los puntos del mundo $\mathbf{X}_j = (X_j, Y_j, 0)$ (coordenadas del tablero, con $Z = 0$) y sus proyecciones $\mathbf{m}_j = (u_j, v_j)$ se modela mediante una **homografía**:

$$s_j \begin{bmatrix} u_j \\ v_j \\ 1 \end{bmatrix} = \mathbf{H}_i \begin{bmatrix} X_j \\ Y_j \\ 1 \end{bmatrix}, \quad \mathbf{H}_i = \mathbf{K} \begin{bmatrix} \mathbf{r}_1^{(i)} & \mathbf{r}_2^{(i)} & \mathbf{t}^{(i)} \end{bmatrix} \quad (5.1)$$

donde $\mathbf{r}_1^{(i)}, \mathbf{r}_2^{(i)}$ son las dos primeras columnas de la rotación y $\mathbf{t}^{(i)}$ la traslación de la pose i .

El procedimiento es:

1. **Estimar $\mathbf{H}_i$:** para cada imagen, encontrar la homografía que mapea los puntos del tablero a los píxeles detectados (mínimos cuadrados + RANSAC para robustez).
2. **Extraer $\mathbf{K}$:** las restricciones de ortogonalidad de las columnas de $\mathbf{R}$ imponen dos ecuaciones lineales por imagen sobre los elementos de $\mathbf{B} = \mathbf{K}^{-\top} \mathbf{K}^{-1}$. Con $N \geq 3$ imágenes se resuelve un sistema lineal para $\mathbf{B}$ y se recupera $\mathbf{K}$ por descomposición de Cholesky.
3. **Estimar distorsión:** con $\mathbf{K}$ conocida, se calculan las coordenadas ideales (sin distorsión) y las observadas; la diferencia se ajusta al modelo de distorsión radial+tangencial (Ecs. (1.10)–(1.11) del Tema 1).
4. **Refinamiento global:** minimización no lineal (Levenberg-Marquardt) del **error de reproyección** sobre *todos* los parámetros simultáneamente:

$$E_{\text{repro}} = \sum_{i=1}^N \sum_{j=1}^M \|\mathbf{m}_{ij} - \hat{\mathbf{m}}(\mathbf{K}, k_1, k_2, p_1, p_2, \mathbf{R}_i, \mathbf{t}_i, \mathbf{X}_j)\|^2 \quad (5.2)$$

donde $\hat{\mathbf{m}}(\cdot)$ es la proyección del modelo completo (pinhole + distorsión) y M es el número de esquinas por imagen.

5.1.2 Preparación del patrón y captura

Cuadro 5.1: Requisitos para la calibración con patrón de tablero.

Requisito	Especificación
Tablero	$\geq 7 \times 5$ esquinas internas; cuadro de 10–25 mm; impreso en material rígido (aluminio, acrílico, vidrio) con planitud < 0.5 mm.
Número de imágenes	$N \geq 15$ (más imágenes = mejor estimación de k_3 y parámetros tangenciales).
Variedad de poses	Rotaciones en los tres ejes ($0^\circ, \pm 20^\circ, \pm 30^\circ$), desplazamientos laterales, variaciones de distancia. El tablero debe llenar $\geq 50\%$ del campo de visión en cada imagen.
Iluminación	Uniforme, sin reflejos sobre el tablero. Evitar sombras del soporte.
Enfoque	Todas las esquinas deben ser nítidas; profundidad de campo $f/8$ – $f/11$.

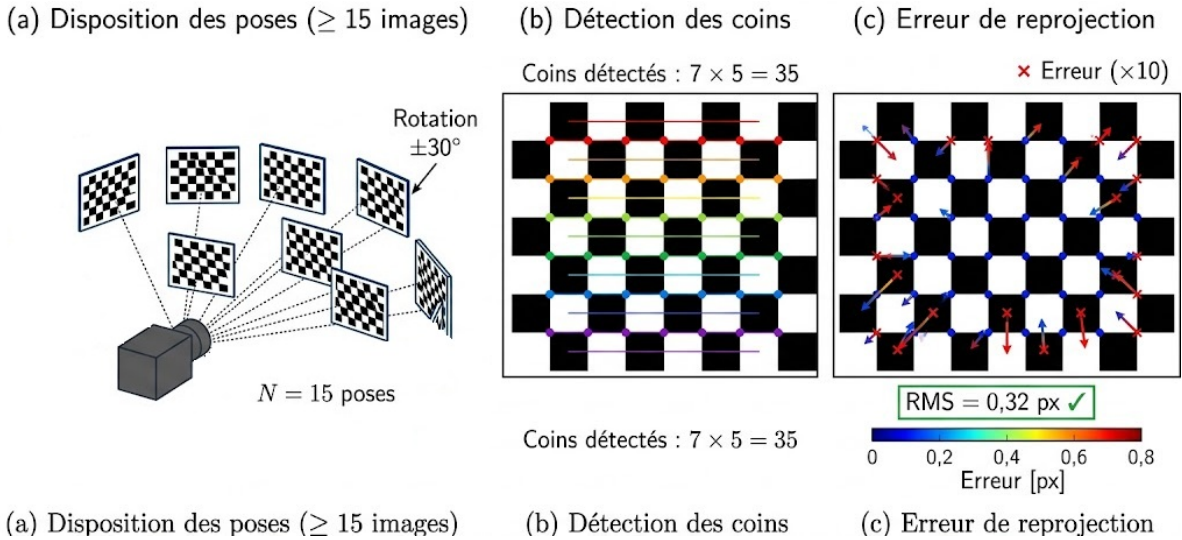


Figura 5.1: Calibración intrínseca con patrón de tablero. **(a)** Disposición recomendada: 15 imágenes del tablero en poses variadas (rotaciones, traslaciones, inclinaciones). Las poses se representan como rectángulos inclinados alrededor de la cámara. **(b)** Detección de esquinas: imagen del tablero con las esquinas detectadas superpuestas (puntos de colores conectados por líneas). **(c)** Mapa de errores de reproyección: para cada imagen, las flechas muestran la dirección y magnitud del error (exagerado $\times 10$). RMS = 0.32 px.

¿Por qué ≥ 15 imágenes?

Con 3 imágenes se puede resolver el sistema lineal para $\mathbf{K}$, pero la estimación es muy sensible al ruido en la detección de esquinas. Con 15–25 imágenes, la minimización no lineal converge de forma estable y permite estimar con fiabilidad hasta 5 coeficientes de distorsión (k_1, k_2, k_3, p_1, p_2). Para lentes gran angular con distorsión severa, $N \geq 20$ imágenes es recomendable.

5.2 Calibración extrínseca y estéreo

La calibración extrínseca determina la **posición y orientación relativa** entre dos dispositivos: cámara–proyector (en luz estructurada) o cámara–cámara (en fotogrametría estéreo).

5.2.1 Par estéreo para luz estructurada

En un sistema de luz estructurada, el proyector se modela como una “cámara inversa” con su propia matriz de intrínsecos $\mathbf{K}_p$. La calibración estéreo estima la transformación rígida $[\mathbf{R}|\mathbf{t}]$ entre la cámara y el proyector:

$$\mathbf{P}_{\text{cámara}} = \mathbf{R} \cdot \mathbf{P}_{\text{proyector}} + \mathbf{t} \quad (5.3)$$

El procedimiento típico es:

1. Proyectar un patrón de tablero virtual con el proyector sobre una pantalla plana (o usar un tablero físico iluminado por el proyector con un patrón de fase conocido).
2. Capturar $N \geq 15$ poses del tablero con la cámara.
3. Calibrar la cámara y el “modelo de proyector” simultáneamente (`cv2.stereoCalibrate()` en OpenCV).

5.2.2 Par estéreo para fotogrametría

En un sistema de dos cámaras, la calibración estéreo estima:

- $\mathbf{K}_1, \mathbf{d}_1$: intrínsecos + distorsión de la cámara 1.
- $\mathbf{K}_2, \mathbf{d}_2$: intrínsecos + distorsión de la cámara 2.
- $[\mathbf{R}|\mathbf{t}]$: transformación rígida de la cámara 2 respecto a la cámara 1.

5.2.3 Significado físico de los parámetros extrínsecos

- La **baseline** $B = \|\mathbf{t}\|$ es la separación entre los centros ópticos. Determina la resolución en profundidad (Ec. (1.3) del Tema 1).
- El **ángulo de convergencia** α define la orientación relativa de los ejes ópticos. Un ángulo $\alpha = 0^\circ$ corresponde a ejes paralelos (*rectified stereo*); $\alpha \neq 0^\circ$ requiere rectificación por software.

5.3 Criterios de aceptación de la calibración

Una calibración se acepta cuando cumple los tres criterios siguientes:

Advertencia: imágenes de calibración deficientes

Causas comunes de calibración deficiente:

- Tablero no plano (papel arrugado, cartón flexionado): introduce error sistemático en f y en k_1 .
- Esquinas mal detectadas (imagen borrosa, tablero parcialmente fuera de campo): errores aleatorios grandes.
- Poca diversidad de poses: todas las imágenes frontales (0°) no permiten estimar f con precisión.
- Tablero demasiado pequeño en la imagen ($< 30\%$ del FOV): baja sensibilidad a la distorsión radial.

Siempre inspeccionar la detección de esquinas imagen por imagen antes de aceptar la calibración.

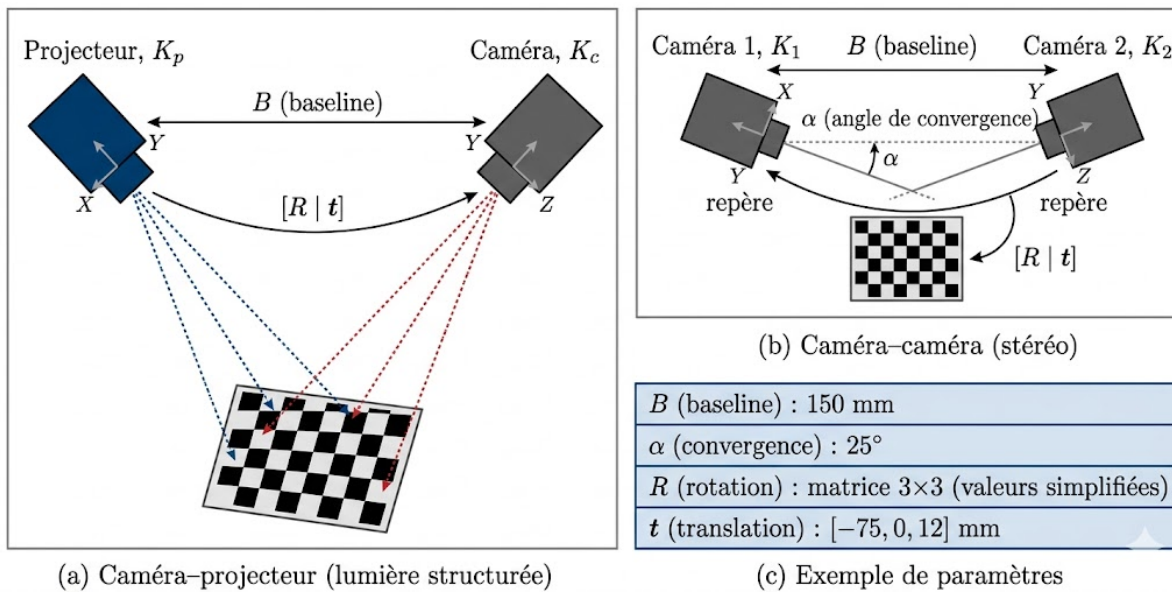


Figura 5.2: Calibración extrínseca: geometría del par estéreo. **(a)** Sistema cámara-proyector para luz estructurada: la cámara (derecha) y el proyector (izquierda) están separados por la *baseline* B . Se muestra el tablero en una pose de calibración con los rayos de proyección y observación. **(b)** Sistema cámara-cámara para fotogrametría estéreo: dos cámaras con ejes ópticos convergentes (ángulo α), observando el mismo tablero. Se muestran las matrices K_1 , K_2 y la transformación $[R|t]$. **(c)** Tabla resumen de parámetros extrínsecos con valores numéricos de ejemplo.

Cuadro 5.2: Criterios de aceptación de la calibración de cámara.

Criterio	Umbral	Método de verificación
Error de reproyección (RMS)	< 0.75 px	Reportado por <code>cv2.calibrateCamera()</code> o el toolbox MATLAB. Es el promedio cuadrático de la distancia entre esquinas detectadas y esquinas reproyectadas con el modelo estimado.
Error de reproyección (máximo)	< 1.50 px	Ninguna imagen individual debe tener un error medio > 1.5 px. Imágenes con error excesivo deben descartarse y recalibrar.
Distribución de residuos	Sin patrones sistemáticos	Graficar los vectores de error (e_u, e_v) para todas las esquinas: la distribución debe ser simétrica, centrada en $(0, 0)$, sin patrón radial ni direccional. Un patrón radial indica un modelo de distorsión insuficiente (agregar k_3).
Repetibilidad	$\Delta f_x / f_x < 1\%$	Recalibrar con un subconjunto diferente de imágenes ($N/2$) y comparar f_x, f_y, c_x, c_y . La variación relativa debe ser $< 1\%$ en focales y < 3 px en punto principal.

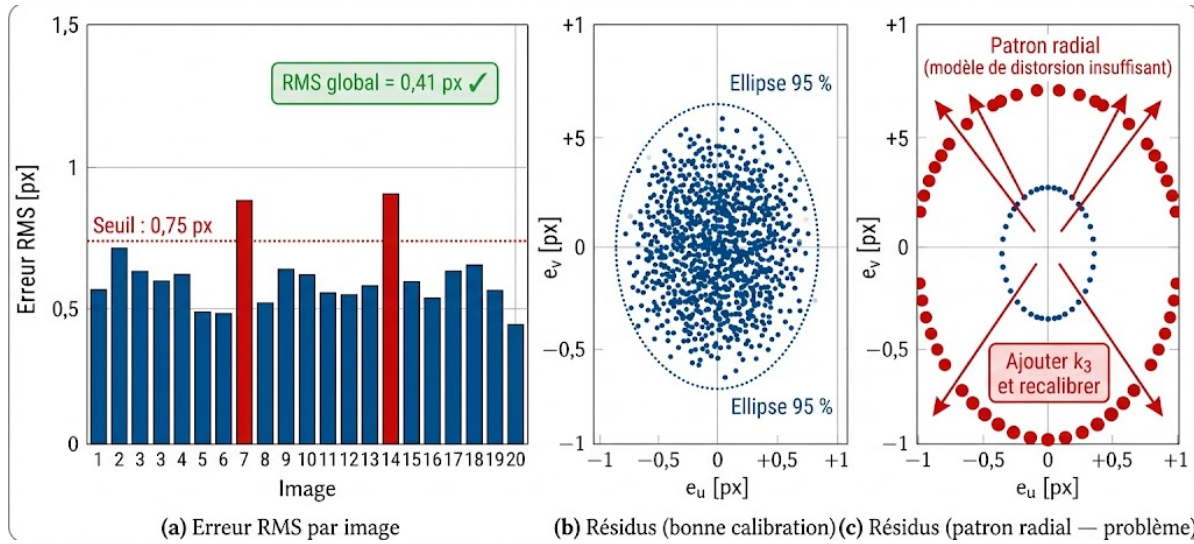


Figura 5.3: Diagnóstico de calidad de la calibración. **(a)** Gráfico de barras: error de reproyección RMS por imagen ($N = 20$ imágenes). Línea horizontal roja: umbral 0.75 px. Dos imágenes con barras que superan el umbral (marcadas en rojo). **(b)** Distribución de residuos (e_u, e_v): nube de puntos centrada en el origen, distribución simétrica, sin patrón radial. Elipse de confianza al 95%. **(c)** Ejemplo de distribución *patológica*: patrón radial en los residuos (residuos más grandes en las esquinas), indicando modelo de distorsión insuficiente.

5.4 Registro multisesión

En proyectos reales, es frecuente que la captura 3D se realice en **varias sesiones** (diferentes días, configuraciones o incluso equipos). El registro multisesión consiste en alinear todas las capturas a un sistema de coordenadas común.

5.4.1 Estrategia con targets fijos

La forma más robusta es:

1. **Instalar targets permanentes:** pegar ≥ 5 marcadores retro-reflectantes o codificados en posiciones fijas del entorno (mesa, bastidor, pared) *que no se muevan entre sesiones*.
2. **Capturar los targets en cada sesión:** asegurar que ≥ 3 targets comunes sean visibles en ambas sesiones.
3. **Calcular la transformación rígida:** con las coordenadas 3D de los targets en el sistema de cada sesión, resolver:

$$\min_{\mathbf{R}, \mathbf{t}} \sum_{k=1}^K \left\| \mathbf{R} \cdot \mathbf{T}_k^{(A)} + \mathbf{t} - \mathbf{T}_k^{(B)} \right\|^2 \quad (5.4)$$

donde $\mathbf{T}_k^{(A)}$ y $\mathbf{T}_k^{(B)}$ son las posiciones del target k en las sesiones A y B , respectivamente. Con $K \geq 3$ puntos no colineales, la solución es única (descomposición SVD).

4. **Refinar con ICP:** usar la transformación de targets como inicialización, seguida de ICP *point-to-plane* sobre las zonas de solapamiento entre sesiones.
5. **Verificar:** el error RMS entre las posiciones de targets transformadas y las de referencia debe ser ≤ 0.30 mm. El error global del ICP de refinamiento debe ser comparable al obtenido dentro de cada sesión individual.

Alternativa sin targets fijos

Si no se dispone de targets permanentes, el registro multisesión se puede hacer con el mismo procedimiento ICP del Tema 3, cargando cada sesión como una capa independiente en MeshLab o como nubes separadas en Open3D. La precisión dependerá de la calidad del solapamiento entre sesiones — se requiere $\geq 30\%$ de superficie común. Este método es menos preciso que el basado en targets pero funcional para fines didácticos.

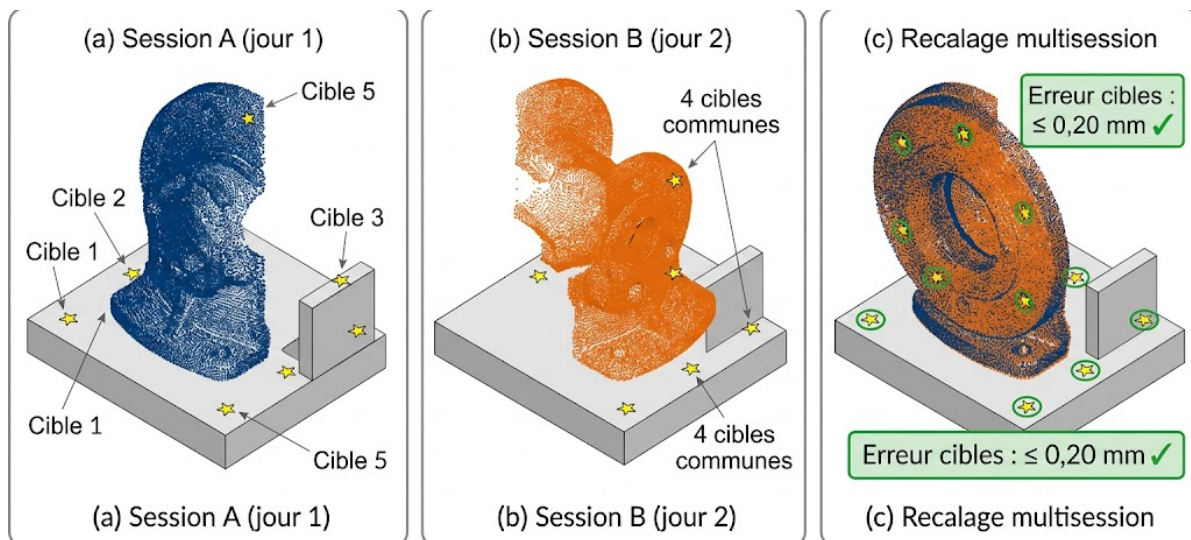


Figura 5.4: Registro multisesión con targets fijos. **(a)** Sesión A (día 1): nube de puntos parcial (azul) con 5 targets visibles (estrellas amarillas). Pieza escaneada desde la parte frontal. **(b)** Sesión B (día 2): nube parcial (naranja) con 4 targets comunes y la pieza rotada para capturar la parte posterior. **(c)** Resultado del registro: ambas nubes alineadas en el mismo sistema de coordenadas, mostrando la pieza completa (360°). Los targets coinciden con error ≤ 0.20 mm.

5.5 Extracción de primitivas geométricas

Muchas piezas mecánicas están compuestas de superficies analíticas simples: **planos**, **cilindros**, **esferas** y **conos**. La extracción de estas primitivas a partir de la nube de puntos

(o malla) es un paso clave para la reconstrucción CAD paramétrica (Tema 6).

5.5.1 Ajuste por mínimos cuadrados

Para cada tipo de primitiva, se define un modelo paramétrico y se ajustan los parámetros minimizando la suma de distancias al cuadrado de los puntos al modelo:

Plano

$$ax + by + cz + d = 0, \quad \text{con } a^2 + b^2 + c^2 = 1 \quad (5.5)$$

La normal del plano es $\hat{\mathbf{n}} = (a, b, c)$ y la distancia al origen es $|d|$. El ajuste se resuelve por SVD de la matriz de coordenadas centradas (idéntico a la estimación de normales del Tema 3, pero sobre toda la región de interés).

Cilindro

Un cilindro de radio r , eje $\hat{\mathbf{a}}$ (vector unitario) y punto sobre el eje $\mathbf{c}$ se define por:

$$\|(\mathbf{p}_i - \mathbf{c}) \times \hat{\mathbf{a}}\| = r \quad (5.6)$$

El ajuste requiere minimización no lineal (7 parámetros: $\mathbf{c}$, $\hat{\mathbf{a}}$, r).

Esfera

$$\|\mathbf{p}_i - \mathbf{c}\| = r \quad (5.7)$$

con 4 parámetros: centro $\mathbf{c} = (x_c, y_c, z_c)$ y radio r . Se linealiza expandiendo la norma al cuadrado.

5.5.2 Detección automática con RANSAC

Cuando la nube contiene múltiples primitivas mezcladas con ruido, RANSAC permite detectar cada primitiva secuencialmente:

1. Seleccionar aleatoriamente el mínimo de puntos necesarios para definir la primitiva (3 para un plano, 5 para un cilindro).
2. Estimar los parámetros del modelo a partir de esos puntos.
3. Contar los *inliers*: puntos cuya distancia al modelo es $< d_{\max}$ (umbral típico: 0.1–0.5 mm).
4. Repetir M iteraciones ($M = 1\,000$ – $10\,000$).
5. Retener el modelo con más *inliers*.
6. Remover los *inliers* de la nube y repetir para la siguiente primitiva.

Cuadro 5.3: Métricas de calidad para el ajuste de primitivas.

Métrica	Umbral	Interpretación
R^2	> 0.95	Proporción de la varianza explicada por el modelo. $R^2 = 1.0$: ajuste perfecto. $R^2 < 0.95$: la región probablemente no es una primitiva simple (superficie libre).
RMS residual	≤ 0.10 mm	Raíz cuadrada media de las distancias punto–modelo. Depende de la resolución del escáner y del ruido.
Porcentaje de <i>inliers</i>	$\geq 80\%$	Fracción de puntos de la región que están dentro del umbral $d_{\max}$ del modelo. Un valor bajo indica contaminación por puntos de otra superficie.

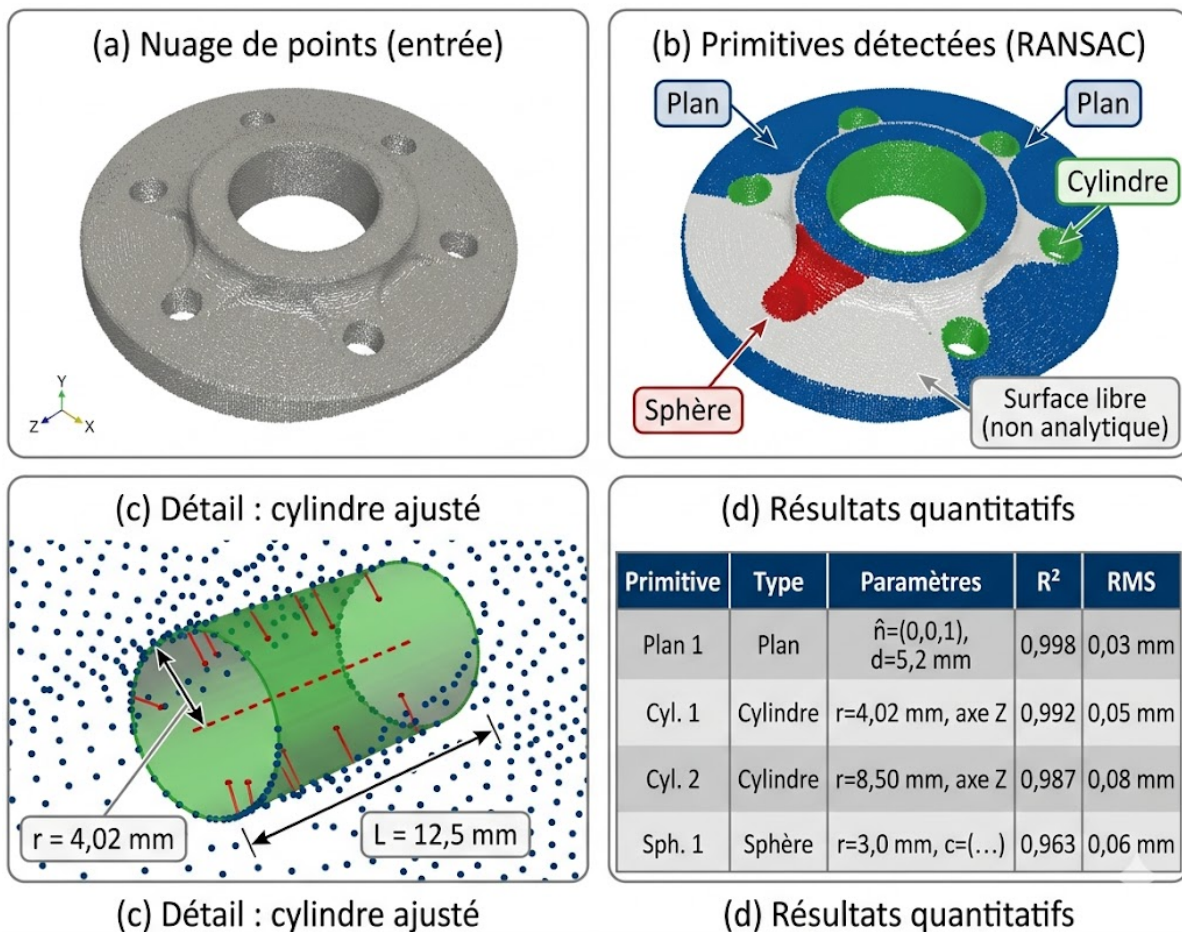


Figura 5.5: Extracción de primitivas geométricas de una pieza mecánica. **(a)** Nube de puntos original (gris). **(b)** Primitivas detectadas por RANSAC, coloreadas por tipo: planos (azul), cilindros (verde), esfera (rojo). Los puntos no asignados a ninguna primitiva (superficie libre) se muestran en gris claro. **(c)** Detalle de un cilindro ajustado: nube de puntos (azul) con el cilindro analítico superpuesto (verde semi-transparente), mostrando eje, radio y residuos. **(d)** Tabla de resultados: para cada primitiva, tipo, parámetros, R^2 y RMS.

5.5.3 Métricas de calidad del ajuste

5.6 Código Python: calibración con OpenCV

Nota: la calibración es exclusivamente programática

MeshLab y CloudCompare **no incluyen herramientas de calibración de cámara**. La calibración se realiza siempre con OpenCV (Python) o con el Camera Calibration Toolbox de MATLAB. En esta microcredencial se usa OpenCV.

El siguiente script implementa la calibración intrínseca completa:

```
import cv2
import numpy as np
import glob
import os

# ==== 1. PARÁMETROS DEL TABLERO ====
ROWS = 7          # esquinas internas verticales
COLS = 5          # esquinas internas horizontales
SQUARE_SIZE = 15  # mm (lado del cuadro)

# Puntos 3D del tablero (Z = 0)
objp = np.zeros((ROWS * COLS, 3), np.float32)
objp[:, :2] = np.mgrid[0:ROWS, 0:COLS].T.reshape(-1, 2)
objp *= SQUARE_SIZE

# ==== 2. DETECTAR ESQUINAS EN CADA IMAGEN ====
obj_points = []  # puntos 3D del mundo
img_points = []  # puntos 2D en imagen
images = sorted(glob.glob("calibracion/*.jpg"))
print(f"Imágenes encontradas: {len(images)}")

for fname in images:
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ret, corners = cv2.findChessboardCorners(
        gray, (ROWS, COLS), None)
    if ret:
        # Refinar a subpíxel
        criteria = (cv2.TERM_CRITERIA_EPS +
                    cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
        corners2 = cv2.cornerSubPix(
            gray, corners, (11, 11), (-1, -1), criteria)
        obj_points.append(objp)
```

```

img_points.append(corners2)
else:
print(f"  ADVERTENCIA: esquinas no detectadas "
f"en {os.path.basename(fname)}")

print(f"Imágenes válidas: {len(obj_points)}/{len(images)}")

# ==== 3. CALIBRAR ====
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
obj_points, img_points, gray.shape[::-1],
None, None)

print(f"\n=== RESULTADOS ===")
print(f"RMS reproyección: {ret:.4f} px")
print(f"Matriz K:\n{K}")
print(f"Distorsión (k1,k2,p1,p2,k3): {dist.ravel()}")
print(f"fx = {K[0,0]:.1f} px, fy = {K[1,1]:.1f} px")
print(f"cx = {K[0,2]:.1f} px, cy = {K[1,2]:.1f} px")

# ==== 4. ERROR POR IMAGEN ====
print(f"\nError por imagen:")
errors = []
for i in range(len(obj_points)):
imgp2, _ = cv2.projectPoints(
obj_points[i], rvecs[i], tvecs[i], K, dist)
err = cv2.norm(img_points[i], imgp2,
cv2.NORM_L2) / len(imgp2)
errors.append(err)
flag = " " if err > 0.75 else " "
print(f" Imagen {i+1}: {err:.4f} px{flag}")

# ==== 5. GUARDAR PARÁMETROS ====
np.savez("calibracion_params.npz",
K=K, dist=dist, rvecs=rvecs, tvecs=tvecs,
rms=ret, image_errors=errors)
print("\nParámetros guardados en calibracion_params.npz")

```

Para la **calibración estéreo** (cámara-proyector o cámara-cámara), agregar:

```

# ==== CALIBRACIÓN ESTÉREO ====
# (suponiendo que ya se calibraron K1, dist1, K2, dist2)
ret_stereo, K1, d1, K2, d2, R, T, E, F = \
cv2.stereoCalibrate(
obj_points, img_points_cam1, img_points_cam2,
K1, dist1, K2, dist2, gray.shape[::-1],

```

```

criteria=criteria,
flags=cv2.CALIB_FIX_INTRINSIC)

print(f"RMS estéreo: {ret_stereo:.4f} px")
print(f"Baseline: {np.linalg.norm(T):.2f} mm")
print(f"R:\n{R}")
print(f"T: {T.ravel()} mm")

```

5.7 Extracción de primitivas: Vía A (Python) / Vía B (Cloud-Compare)

5.7.1 Vía A: Python con Open3D

```

import open3d as o3d
import numpy as np

pcd = o3d.io.read_point_cloud("nube_registrada.ply")

# --- Detección de plano con RANSAC ---
plane_model, inliers = pcd.segment_plane(
    distance_threshold=0.1,    # mm
    ransac_n=3,
    num_iterations=1000)
[a, b, c, d] = plane_model
print(f"Plano: {a:.4f}x + {b:.4f}y + {c:.4f}z "
      f"+ {d:.4f} = 0")
print(f"Inliers: {len(inliers)} "
      f"({100*len(inliers)/len(pcd.points):.1f}%)")

# Extraer inliers y residuos
plane_cloud = pcd.select_by_index(inliers)
rest_cloud = pcd.select_by_index(inliers, invert=True)

# Colorear para visualización
plane_cloud.paint_uniform_color([0.0, 0.4, 0.8]) # azul
rest_cloud.paint_uniform_color([0.7, 0.7, 0.7])  # gris

# --- Calcular R² ---
pts = np.asarray(plane_cloud.points)
distances = np.abs(a*pts[:,0] + b*pts[:,1]
                  + c*pts[:,2] + d)
rms = np.sqrt(np.mean(distances**2))
ss_res = np.sum(distances**2)

```

```
centroid = pts.mean(axis=0)
ss_tot = np.sum(np.linalg.norm(
pts - centroid, axis=1)**2)
r_squared = 1 - ss_res / ss_tot
print(f"RMS residual: {rms:.4f} mm")
print(f"R2: {r_squared:.4f}")

o3d.visualization.draw_geometries(
[plane_cloud, rest_cloud],
window_name="Plano detectado (RANSAC)")
```

Extensión a cilindros y esferas

Open3D solo incluye `segment_plane()` de forma nativa. Para detectar cilindros y esferas con RANSAC, opciones: (i) usar la biblioteca `pyransac3d` (`pip install pyransac3d`) que implementa ajuste RANSAC para planos, cilindros, esferas y conos; (ii) implementar manualmente el modelo paramétrico + RANSAC loop; (iii) usar CloudCompare (Vía B) para ajuste interactivo.

5.7.2 Vía B: CloudCompare (interfaz gráfica)

Nota: MeshLab no tiene ajuste robusto de primitivas

MeshLab no incluye herramientas de ajuste de primitivas (*fit plane, fit sphere, fit cylinder*). Para esta tarea, la alternativa GUI recomendada es **CloudCompare**, que tiene herramientas dedicadas en el menú `Tools → Fit`.

El procedimiento en CloudCompare es:

1. **Cargar la nube:** `File → Open` → seleccionar el archivo PLY.
2. **Seleccionar la región de interés:** usar las herramientas de segmentación (`Edit → Segment` o el icono de tijera) para aislar la zona que contiene la primitiva.
3. **Ajustar la primitiva:**
 - **Plano:** `Tools → Fit → Plane`. CloudCompare muestra el plano ajustado, su ecuación, el RMS y el número de puntos.
 - **Esfera:** `Tools → Fit → Sphere`. Reporta centro (x_c, y_c, z_c) , radio r y RMS.
 - **Cilindro:** `Tools → Fit → Cylinder`. Reporta eje, punto sobre el eje, radio y RMS.
4. **Verificar el ajuste:** la primitiva se superpone a la nube. Inspeccionar visualmente que coincide. El panel `Console` reporta los parámetros y el RMS.
5. **Exportar resultados:** copiar los parámetros del `Console` al reporte. Opcionalmente, guardar la primitiva como entidad 3D (`File → Save`).

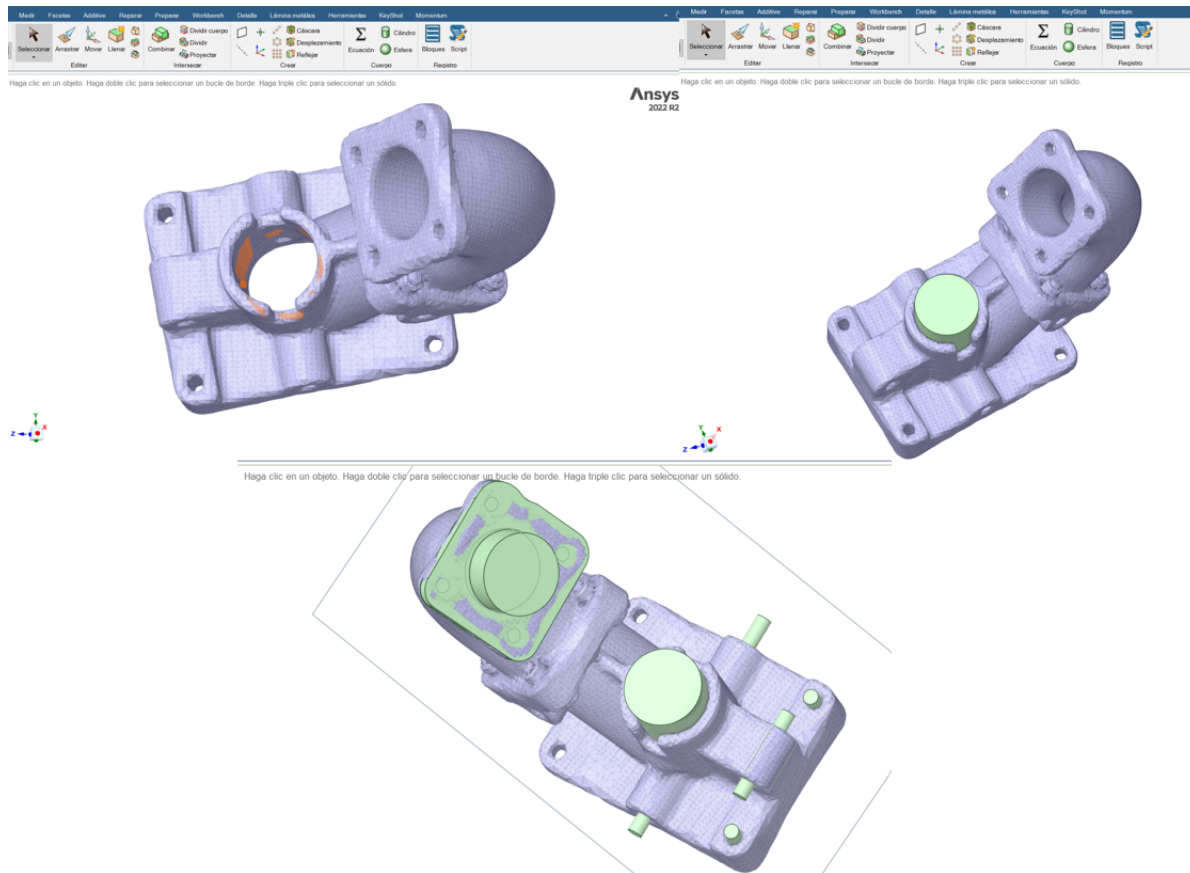


Figura 5.6: Ajuste de primitivas en SpaceClaim (Ansys). Vista 3D de una pieza mecánica con un plano (azul semi-transparente) y un cilindro (verde semi-transparente) ajustados.

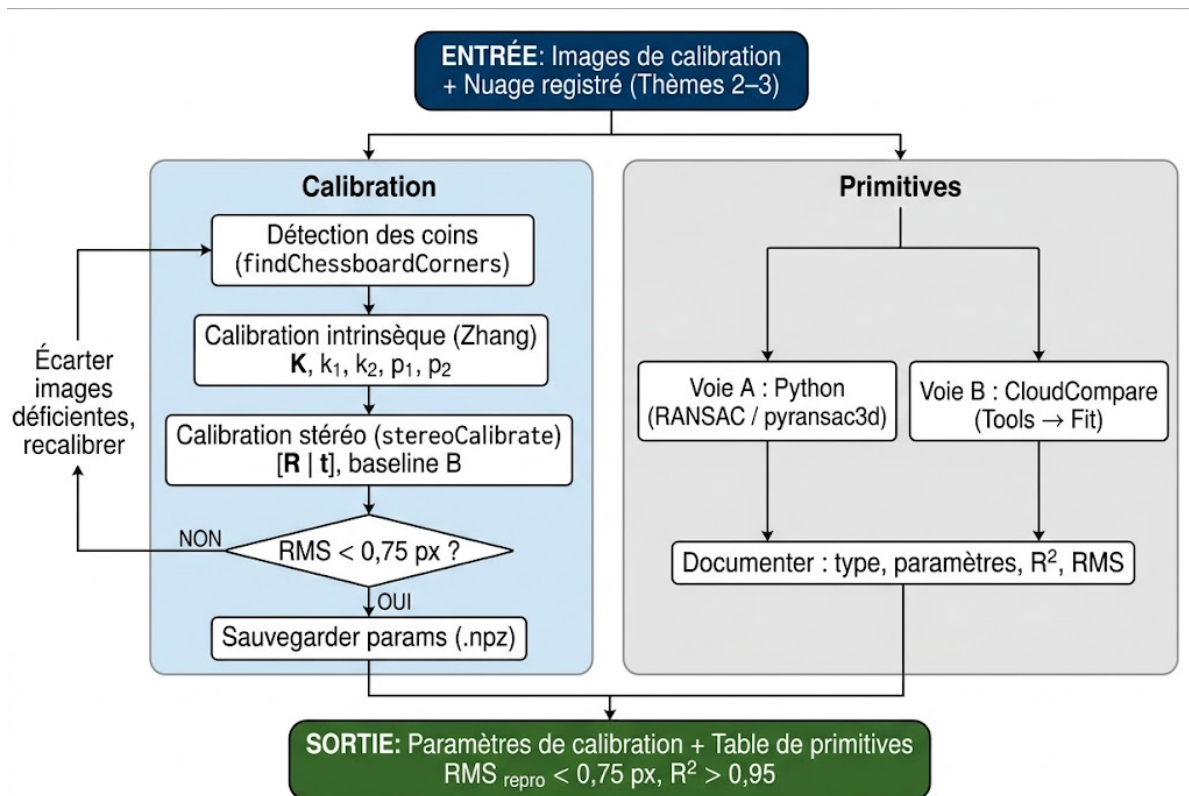


Figura 5.7: Diagrama de flujo del Tema 5: calibración avanzada y extracción de primitivas. Entrada: imágenes de calibración + nube registrada del Tema 3. Dos ramas: (1) Calibración (Zhang → estéreo → verificación → parámetros guardados) y (2) Extracción de primitivas (segmentación → RANSAC / Fit → documentación). Salida: parámetros de calibración (.npz) y tabla de primitivas con $R^2 > 0.95$.

Evidencia de aprendizaje	Reporte de calibración: matriz K , coeficientes de distorsión, error de reproyección RMS por imagen, gráfico de residuos. Tabla de primitivas extraídas: tipo, parámetros, R^2 , RMS. Archivo <code>calibracion_params.npz</code> .
Instrumento de evaluación	Checklist de calibración y extracción de primitivas.
Escala / Criterio de aceptación	Error de reproyección < 0.75 px (RMS); cobertura $\geq 95\%$; primitivas con $R^2 > 0.95$ y $RMS \leq 0.10$ mm. Umbral: 85/100.

Capítulo 6

Superficies NURBS y Comparación CAD-Scan

Duración: 6 h

Modalidad: Mixta (laboratorio + gabinete)

Este capítulo cierra el Nivel Intermedio con las dos operaciones finales del pipeline de ingeniería inversa: **(1)** la reconstrucción de un modelo CAD paramétrico con superficies NURBS a partir de la malla y las primitivas extraídas en los Temas 4–5, y **(2)** la validación dimensional cuantitativa mediante **mapas de desviación** que comparan el modelo CAD reconstruido contra la nube de puntos original. La herramienta principal es **ANSYS SpaceClaim** (disponible en UniLaSalle Amiens con licencia educativa), con **FreeCAD** como alternativa libre. Al finalizar, el participante entregará un modelo STEP con continuidad G^1 , un mapa de desviación cromático y un informe de conformidad dimensional.

6.1 Fundamentos de curvas y superficies NURBS

Las superficies **NURBS** (*Non-Uniform Rational B-Splines*) [?] son el estándar de la industria CAD para representar geometría de forma libre. A diferencia de una malla triangular (que es una aproximación por facetas), una superficie NURBS es una representación **analítica continua** definida por un conjunto compacto de parámetros.

6.1.1 Curvas B-spline

Una curva B-spline de grado p se define como:

$$\mathbf{C}(u) = \sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i, \quad u \in [u_0, u_m] \quad (6.1)$$

donde:

- $\mathbf{P}_i$ son los **puntos de control** ($n + 1$ en total).
- $N_{i,p}(u)$ son las **funciones de base B-spline**, definidas recursivamente por el vector de nudos $U = \{u_0, u_1, \dots, u_m\}$ con $m = n + p + 1$.
- El grado p determina la suavidad: $p = 3$ (cúbica) es el más utilizado en CAD.

6.1.2 Superficies NURBS (producto tensorial)

Una superficie NURBS se extiende a dos parámetros (u, v) :

$$\mathbf{S}(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) N_{j,q}(v) w_{ij} \mathbf{P}_{ij}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) N_{j,q}(v) w_{ij}} \quad (6.2)$$

donde w_{ij} son los **pesos** de cada punto de control $\mathbf{P}_{ij}$. Los pesos permiten representar exactamente cónicas (círculos, elipses, cilindros) — algo imposible con B-splines no racionales.

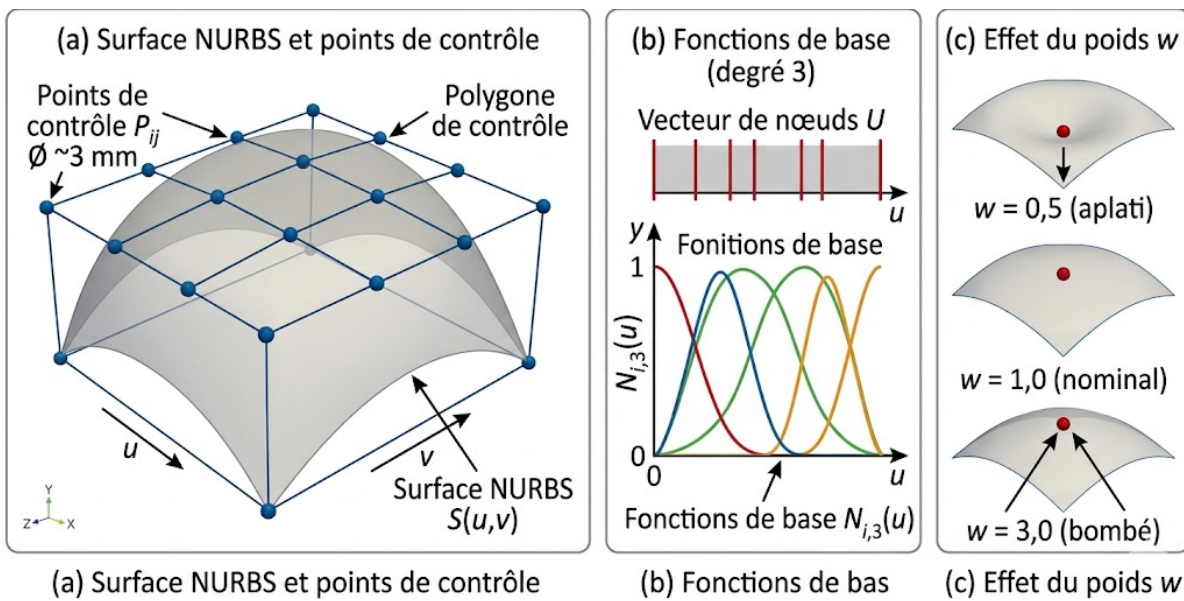


Figura 6.1: Anatomía de una superficie NURBS. **(a)** Red de puntos de control $\mathbf{P}_{ij}$ (esferas azules) conectados por líneas (polígono de control) en una grilla 5×4 . La superficie NURBS (gris semi-transparente) pasa cerca de los puntos de control pero no los interpola (excepto en las esquinas). **(b)** Detalle del vector de nudos: una barra horizontal con marcas que indican los nudos en la dirección u ; debajo, las funciones de base $N_{i,3}(u)$ graficadas (4 curvas coloreadas, grado $p = 3$). **(c)** Efecto del peso w : tres versiones de la misma superficie con $w_{\text{centro}} = 0.5$ (aplanada), $w_{\text{centro}} = 1.0$ (nominal) y $w_{\text{centro}} = 3.0$ (abultada hacia el punto de control central).

6.1.3 Continuidad geométrica

La **continuidad** entre parches de superficie adjacents es fundamental pour la qualité du modèle CAD:

Cuadro 6.1: Niveles de continuidad geométrica entre parches NURBS.

Nivel	Nombre	Condición	Aplicación
G^0	Posicional	Los bordes de dos parches coinciden (sin hueco ni salto).	Mínimo aceptable para cualquier modelo.
G^1	Tangencial	G^0 + los planos tangentes coinciden en la unión (sin “quiebre” visible).	Objetivo de esta microcredencial. Suficiente para manufactura y FEA.
G^2	Curvatura	G^1 + la curvatura es continua a través de la unión (sin cambio brusco de reflexión).	Superficies de clase A (carrocería automotriz, productos de consumo). Fuera del alcance de esta microcredencial.

Analogía: continuidad como “suavidad de transición”

Piense en dos tramos de carretera que se unen: G^0 = los tramos se tocan pero puede haber un “bache” (cambio de pendiente). G^1 = la pendiente es continua (transición sin volantazo). G^2 = la curvatura es continua (transición imperceptible a velocidad). Para ingeniería inversa mecánica, G^1 es el estándar industrial mínimo.

6.2 Ajuste de superficies NURBS sobre la malla

El ajuste de superficies NURBS sobre una malla triangular se realiza en tres etapas: **segmentación, ajuste por parches y ensamble con continuidad.**

6.2.1 Segmentación de la malla en regiones

La malla se divide en regiones según el tipo de superficie:

1. **Regiones planas:** identificadas con las primitivas extraídas en el Tema 5 (planos con $R^2 > 0.95$).
2. **Regiones cilíndricas/esféricas:** también del Tema 5.
3. **Regiones de superficie libre (*freeform*):** todo lo que no se ajusta a una primitiva analítica — requiere parches NURBS.
4. **Filetes y transiciones:** zonas de curvatura variable entre regiones adyacentes.

En ANSYS SpaceClaim, la segmentación es semiautomática: el módulo **Reverse Engineering** detecta regiones por curvatura y permite al usuario ajustar los bordes manualmente.

6.2.2 Ajuste de cada parche

Para cada región de superficie libre, el algoritmo de ajuste resuelve:

$$\min_{\{\mathbf{p}_{ij}, w_{ij}\}} \sum_{k=1}^{N_{\text{pts}}} \|\mathbf{S}(u_k, v_k) - \mathbf{q}_k\|^2 + \lambda_s \cdot \mathcal{R}(\mathbf{S}) \quad (6.3)$$

donde $\mathbf{q}_k$ son los puntos de la malla en la región, (u_k, v_k) son las parametrizaciones correspondientes, y $\mathcal{R}(\mathbf{S})$ es un término de regularización que penaliza la curvatura excesiva (para evitar oscilaciones). El peso λ_s controla el compromiso entre fidelidad a los datos y suavidad.

6.2.3 Ensamble con continuidad G^1

En las uniones entre parches, se imponen restricciones de continuidad:

- G^0 : los bordes de parches adyacentes comparten los mismos puntos de control en la frontera.
- G^1 : la primera fila de puntos de control interior a cada lado de la frontera se alinea para que los vectores tangentes sean colineales.

SpaceClaim maneja estas restricciones automáticamente al usar la herramienta Skin Surface o Fit Surface sobre regiones adyacentes.

6.3 Flujo de trabajo en ANSYS SpaceClaim: scan $\rightarrow$ NURBS $\rightarrow$ STEP

ANSYS SpaceClaim incluye un módulo dedicado de **Reverse Engineering** que implementa el flujo completo desde la malla importada hasta el modelo CAD exportable. El procedimiento paso a paso es:

1. **Importar la malla:** File $\rightarrow$ Open $\rightarrow$ seleccionar el archivo STL o OBJ generado en el Tema 4. SpaceClaim reconoce automáticamente la malla y la carga como un objeto Facets.
2. **Activar el módulo Reverse Engineering:** Pestaña Facets $\rightarrow$ panel Reverse Engineering. Si la pestaña no es visible, verificar que la licencia incluya el módulo SpaceClaim RE (*Reverse Engineering*).
3. **Limpiar la malla** (si no se hizo en el Tema 4):
 - Facets $\rightarrow$ Fix $\rightarrow$ reparar aristas no-manifold, cerrar huecos menores.
 - Facets $\rightarrow$ Reduce $\rightarrow$ decimación adaptativa si la malla es excesivamente densa ($> 500\,000$ triángulos).
4. **Detectar regiones automáticamente:** Facets $\rightarrow$ Auto Detect (o Region Detection). SpaceClaim segmenta la malla por curvatura: regiones planas, cilíndricas, cónicas y de superficie libre se colorean de forma diferente. Ajustar los parámetros:
 - Angular Tolerance: 5° – 15° (controla la sensibilidad a cambios de curvatura).
 - Min Region Size: ≥ 50 triángulos (evita fragmentación excesiva).

5. **Ajustar primitivas a las regiones analíticas:** Seleccionar una región plana → Facets → Fit → Plane. Repetir para cilindros (Fit → Cylinder), esferas, conos. SpaceClaim muestra la desviación RMS del ajuste; verificar ≤ 0.10 mm.
6. **Ajustar superficies NURBS a las regiones de forma libre:** Seleccionar la región → Facets → Fit → Surface. Parámetros:
 - Tolerance: 0.05–0.10 mm (desviación máxima superficie–malla).
 - Smoothness: controla el número de puntos de control. Más suavidad = menos puntos, mayor desviación potencial.
7. **Ensamblar las superficies:** Facets → Skin o Stitch para unir los parches adyacentes con continuidad G^1 . SpaceClaim verifica automáticamente los huecos entre parches y propone cerrarlos con superficies de transición (filetes).
8. **Crear un sólido:** Una vez que todas las superficies forman una cáscara cerrada: Design → Fill (o clic derecho → Convert to Solid). El resultado es un sólido paramétrico editable.
9. **Exportar como STEP:** File → Save As → formato STEP AP214 (o AP203 para máxima compatibilidad). Verificar que el archivo se abre correctamente en otro software CAD (FreeCAD, SolidWorks, Fusion 360).

Alternativa libre: FreeCAD

Si ANSYS SpaceClaim n'est pas disponible, **FreeCAD** (version ≥ 0.21) offre un *work-bench* Reverse Engineering basique : Part → Reverse Engineering → Approximate B-spline Surface. Les résultats sont fonctionnels pour des géométries simples, mais le contrôle sur la segmentation, la continuité et la tolérance est plus limité qu'avec SpaceClaim. Pour cette microcredencial, SpaceClaim est le choix recommandé.

6.4 Comparación CAD–scan con mapas de desviación

La validación del modelo CAD reconstruido se realiza comparándolo con la nube de puntos original (el “ground truth” del objeto físico). El resultado es un **mapa de desviación cromático** que muestra la distancia punto–superficie en cada región.

6.4.1 Comparación en ANSYS SpaceClaim

SpaceClaim permite la comparación directamente dentro del entorno de reconstrucción:

1. **Cargar la nube de referencia:** con el modelo STEP ya creado, importar la nube de puntos original (PLY) como Facets adicionales o como nube de puntos.

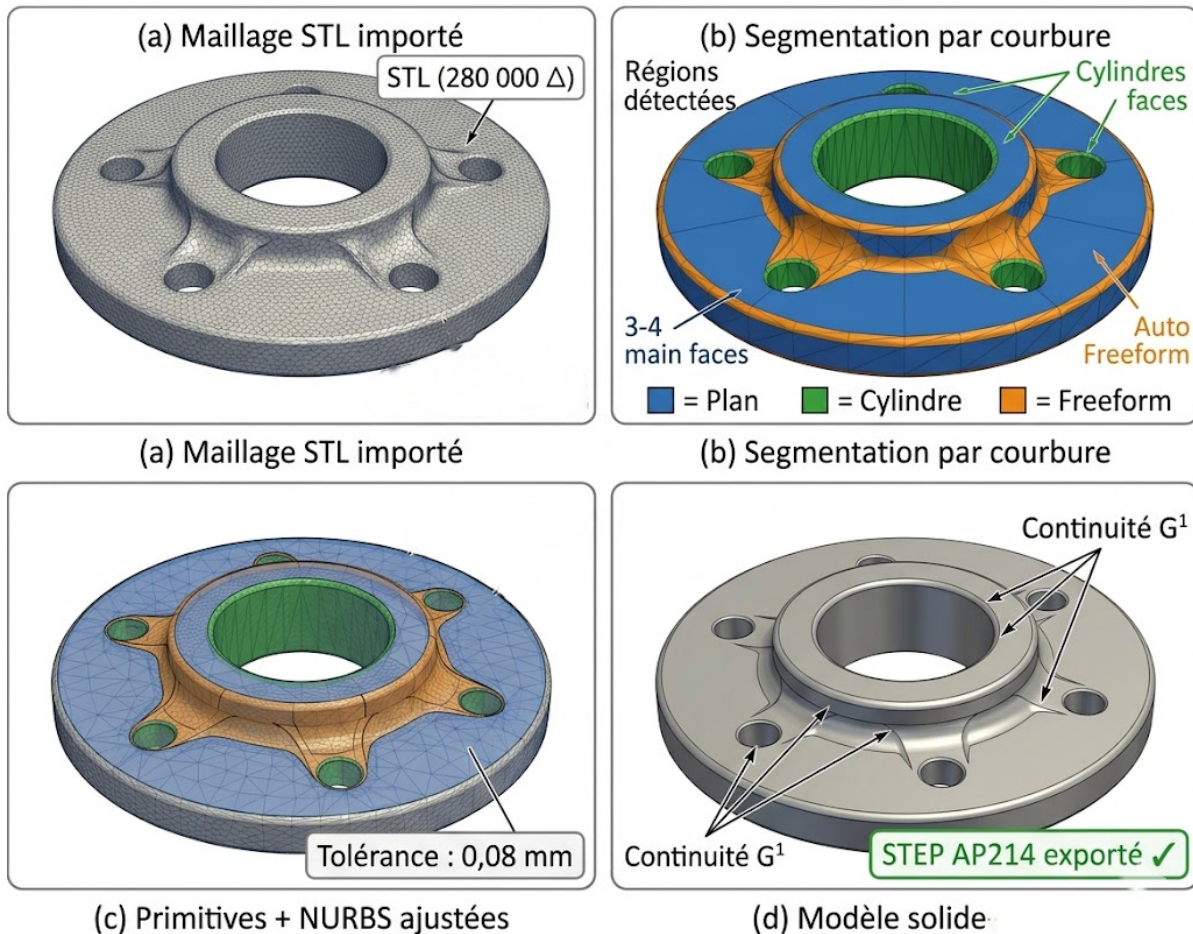


Figura 6.2: Flujo de trabajo de ingeniería inversa en ANSYS SpaceClaim. **(a)** Malla STL importada (gris, triángulos visibles). **(b)** Regiones detectadas automáticamente: planos (azul), cilindros (verde), superficie libre (naranja). **(c)** Primitivas y superficies NURBS ajustadas (superficies coloreadas semi-transparentes sobre la malla gris). **(d)** Modelo sólido STEP final (gris metálico, aristas netas, sin triángulos visibles). Se señala la continuidad G^1 en las uniones entre parches.

2. **Alinear** (si no están ya en el mismo sistema de coordenadas): Facets → Align → seleccionar al menos 3 puntos correspondientes o usar alineación automática Best Fit.
3. **Calcular la distancia:** Measure → Deviation (o Facets → Deviation). SpaceClaim calcula la distancia perpendicular de cada punto de la nube a la superficie CAD más cercana.
4. **Configurar la escala cromática:**
 - Rango: ± 0.30 mm (simétrico alrededor de cero).
 - Paleta: divergente azul–blanco–rojo (azul = desviación negativa, la superficie CAD está “dentro” de la nube; rojo = desviación positiva, la superficie CAD está “fuera”).
 - Resolución: suficiente para distinguir zonas con $\Delta > 0.10$ mm.

6.4.2 Comparación alternativa en CloudCompare

Si la comparación en SpaceClaim no está disponible o se desea una segunda verificación independiente:

1. Importar la nube de puntos (PLY) y el modelo CAD (STEP convertido a malla en MeshLab).
2. Tools → Distances → Cloud/Mesh Dist.
3. Configurar la escala cromática con los mismos parámetros.

6.4.3 Histograma de desviaciones

El mapa cromático da información visual; el **histograma** proporciona estadísticas cuantitativas:

$$\text{RMS}_{\text{desviación}} = \sqrt{\frac{1}{N} \sum_{i=1}^N d_i^2} \quad (6.4)$$

$$\text{Desviación máxima} = \max_i |d_i| \quad (6.5)$$

Las estadísticas clave a reportar son:

6.5 Interpretación cuantitativa de resultados

El mapa de desviación no solo valida el modelo: también revela los **puntos débiles del proceso** de captura y reconstrucción.

Cuadro 6.2: Estadísticas del mapa de desviación CAD-scan.

Métrica	Umbral	Interpretación
Media de desviación	$ \bar{d} \leq 0.05 \text{ mm}$	Un sesgo $\neq 0$ indica error sistemático (escala, alineación residual o recubrimiento de spray).
RMS de desviación	$\leq 0.50 \text{ mm}$	Indicador global de la fidelidad del modelo. Para piezas mecánicas con tolerancia $\pm 0.30 \text{ mm}$, un RMS $\leq 0.15 \text{ mm}$ es excelente.
Desviación máxima	$\leq \pm 0.30 \text{ mm}$	La desviación puntual máxima aceptable. Zonas que exceden este umbral deben investigarse (causa: oclusión, ruido, simplificación NURBS excesiva).
Percentil 95	$\leq 0.25 \text{ mm}$	El 95% de los puntos tiene desviación menor a este valor. Más robusto que el máximo (no afectado por <i>outliers</i> aislados).

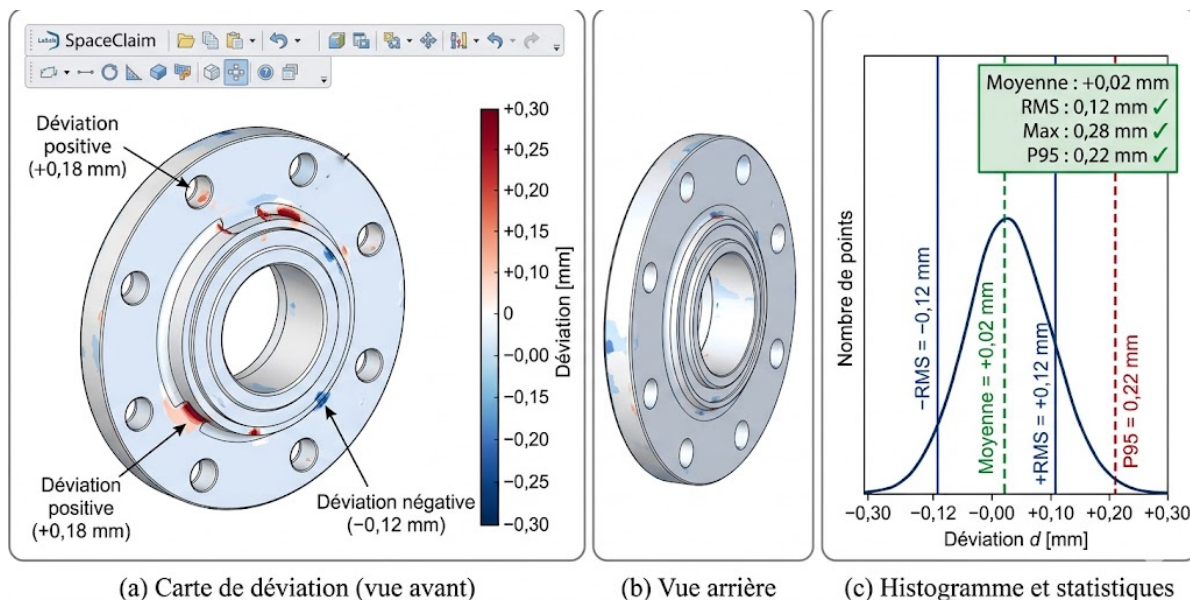


Figura 6.3: Mapa de desviación CAD-scan en ANSYS SpaceClaim. **(a)** Vista 3D del modelo CAD con el mapa de desviación cromático superpuesto: escala divergente azul (-0.30 mm) – blanco (0) – rojo ($+0.30 \text{ mm}$). Barra de color vertical a la derecha. Se señalan: una zona azul (desviación negativa, la superficie CAD está por debajo de la nube) y una zona roja (desviación positiva). **(b)** Vista posterior de la misma pieza para evaluar la cobertura completa. **(c)** Histograma de desviaciones: eje X = $d \text{ [mm]}$ (-0.30 a $+0.30$), eje Y = número de puntos. Distribución gaussiana centrada en ≈ 0 . Líneas verticales señalan la media, $\pm \text{RMS}$ y el percentil 95. Estadísticas superpuestas: media, RMS, máx., percentil 95.

Cuadro 6.3: Diagnóstico de zonas con desviación fuera de tolerancia.

Patrón observado	Causa probable	Acción correctiva
Zona roja/azul localizada en una cavidad	Oclusión durante la captura: la cavidad no fue escaneada completamente (Tema 2) → Poisson interpoló.	Repetir captura con vista adicional enfocada a la cavidad. Reconstruir.
Desviación positiva uniforme en toda la pieza	Error de escala (fotogrametría sin barra de referencia calibrada) o espesor de spray no compensado.	Verificar y recalcular la escala con la barra de referencia. Documentar el espesor del spray.
Ruido de alta frecuencia (“textura” rojo/azul alter-nante)	Ruido del escáner propagado a la malla y al ajuste NURBS. depth de Poisson demasiado alto.	Reducir depth de Poisson o aplicar suavizado Laplaciano (Tema 4). Aumentar λ_s en el ajuste NURBS.
Desviación elevada en aristas vivas	Suavizado excesivo (Laplaciano del Tema 4 o tolerancia NURBS demasiado laxa).	Reducir las iteraciones de suavizado. Ajustar la tolerancia NURBS a ≤ 0.05 mm en zonas de aristas.
Desviación en la zona de unión entre sesiones	Error de registro multisesión (Tema 5).	Verificar el RMS del registro por targets. Reoptimizar con ICP de refinamiento.

6.5.1 Causas de desviaciones elevadas

Criterio de aceptación global

El modelo CAD se acepta cuando **simultáneamente**:

- Desviación máxima $\leq \pm 0.30$ mm.
- RMS de desviación ≤ 0.50 mm (objetivo: ≤ 0.15 mm).
- Continuidad G^1 verificada en todas las uniones de parches.
- $\geq 95\%$ de la superficie tiene desviación ≤ 0.25 mm (percentil 95).

Si algún criterio no se cumple, el mapa de desviación indica **dónde** y la Tabla 6.3 indica **por qué**.

6.6 Informe de conformidad dimensional

El entregable final del Nivel Intermedio es un **informe de conformidad dimensional** que documenta todo el proceso y los resultados de forma trazable. La estructura recomendada es:

1. **Objetivo:** descripción de la pieza, propósito de la ingeniería inversa (rediseño, reproducción, validación), tolerancias requeridas.
2. **Equipo y software:** escáner o cámara utilizados (modelo, resolución, número de serie), software y versiones (SpaceClaim, Open3D, MeshLab, CloudCompare), tipo de spray revelador.
3. **Calibración:** parámetros intrínsecos **K**, coeficientes de distorsión, error de reproyección RMS. Referencia al archivo `calibracion_params.npz`.
4. **Procedimiento de captura:** número de vistas, ángulos, distancia de trabajo, configuración de exposición, tipo de marcadores.
5. **Preprocesamiento:** parámetros de filtrado SOR (k, α), tamaño de vóxel, método de registro (FPFH+ICP o manual+ICP), RMS por par de vistas.
6. **Reconstrucción:** parámetros de Poisson (depth, λ), número de triángulos, verificación *watertight*. Primitivas extraídas con R^2 y RMS.
7. **Modelo CAD:** descripción de los parches NURBS, continuidad G^1 , tolerancia de ajuste. Archivo STEP adjunto.
8. **Resultados de la comparación CAD-scan:**
 - Mapa de desviación cromático (captura de pantalla o imagen exportada desde SpaceClaim).
 - Histograma de desviaciones.
 - Tabla de estadísticas: media, RMS, máximo, percentil 95.

9. **Análisis:** interpretación de las zonas de desviación elevada con referencia a la Tabla 6.3. Propuestas de mejora.
10. **Conclusión:** el modelo CAD cumple / no cumple los criterios de aceptación. Recomendaciones para la siguiente iteración.
11. **Referencia normativa:** mención a ISO 10360-8 para trazabilidad del sistema de medición.

6.6.1 Incertidumbre expandida

Para asegurar trazabilidad metrológica, el informe debe estimar la **incertidumbre expandida** de la medición:

$$U = k \cdot u_c, \quad k = 2 \text{ (nivel de confianza 95\%)} \quad (6.6)$$

donde u_c es la incertidumbre combinada, que incluye contribuciones de:

- **Resolución del escáner:** $\delta Z / \sqrt{3}$ (distribución rectangular).
- **Ruido de la nube:** σ_n (estimado como desviación estándar de residuos sobre un plano de referencia).
- **Error de registro:** RMS del registro ICP.
- **Error de reconstrucción:** RMS del ajuste NURBS.
- **Espesor del spray:** si aplica, $\pm 5\text{--}10 \mu\text{m}$.

$$u_c = \sqrt{u_{\text{resolución}}^2 + u_{\text{ruido}}^2 + u_{\text{registro}}^2 + u_{\text{NURBS}}^2 + u_{\text{spray}}^2} \quad (6.7)$$

Ejemplo numérico de incertidumbre

Para un escáner de luz estructurada con $\delta Z = 50 \mu\text{m}$, ruido $\sigma_n = 30 \mu\text{m}$, registro RMS = $100 \mu\text{m}$, ajuste NURBS RMS = $50 \mu\text{m}$ y spray = $10 \mu\text{m}$:

$$u_c = \sqrt{29^2 + 30^2 + 100^2 + 50^2 + 10^2} = \sqrt{841 + 900 + 10\,000 + 2\,500 + 100} \approx 120 \mu\text{m}$$

$$U = 2 \times 120 = 240 \mu\text{m} = 0.24 \text{ mm}$$

El resultado final se reporta como: “Desviación RMS = $0.12 \pm 0.24 \text{ mm}$ ($k = 2$, 95%).” Esto es consistente con una tolerancia de pieza de $\pm 0.30 \text{ mm}$.

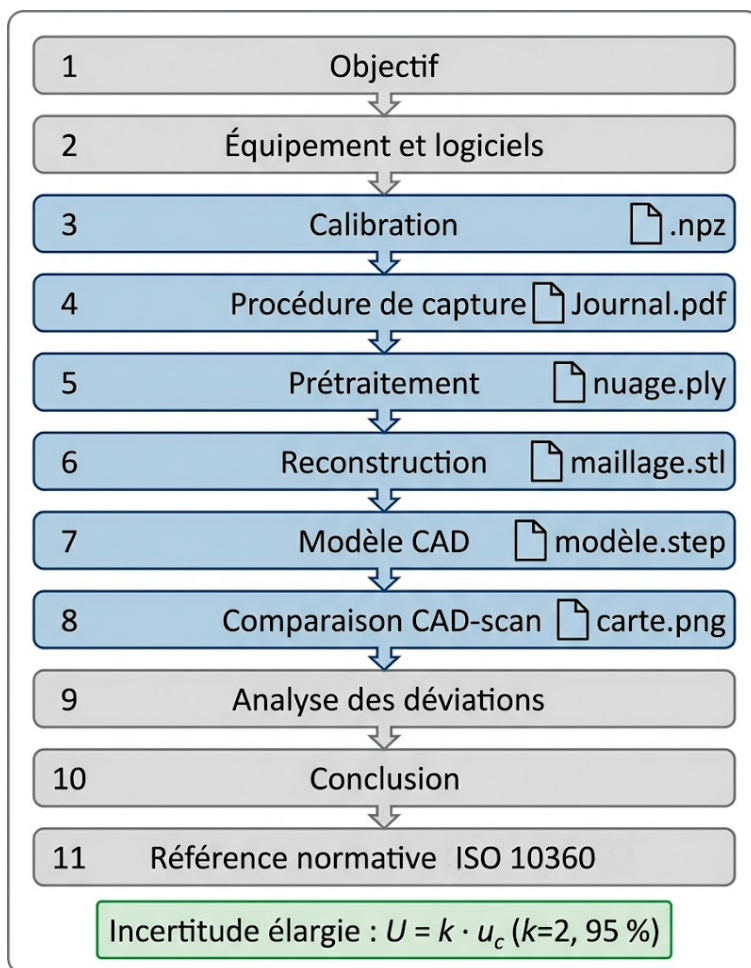


Figura 6.4: Estructura del informe de conformidad dimensional. Diagrama tipo *checklist* visual mostrando las 11 secciones del informe como bloques conectados verticalmente: Objetivo → Equipo → Calibración → Captura → Preprocesamiento → Reconstrucción → Modelo CAD → Comparación → Análisis → Conclusión → Referencia normativa. A la derecha de cada bloque, los archivos adjuntos correspondientes (PLY, STL, STEP, PNG del mapa, .npz de calibración).

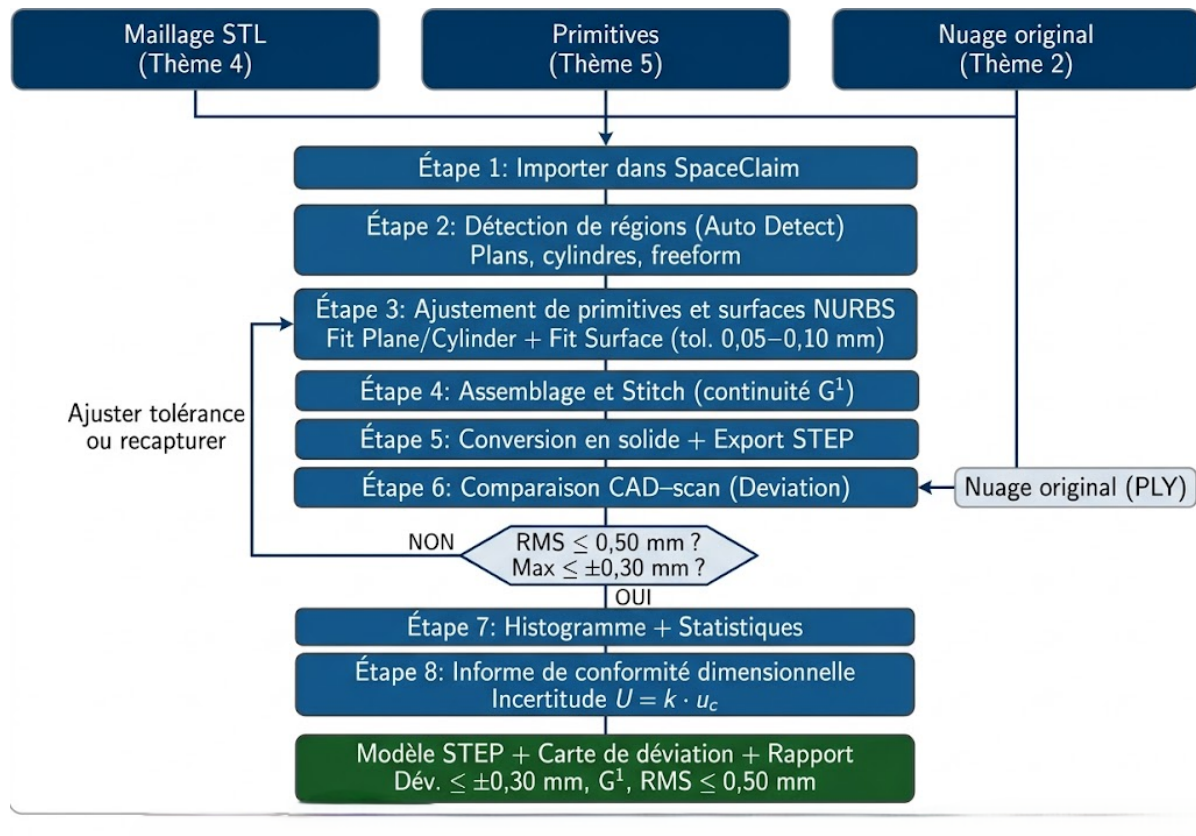


Figura 6.5: Diagrama de flujo del Tema 6: reconstrucción NURBS y validación dimensional. Entrada: malla STL (Tema 4) + primitivas (Tema 5) + nube original (Tema 2). Etapas: importar malla → detectar regiones → ajustar primitivas + NURBS → ensamblar → exportar STEP → comparar CAD-scan → histograma → informe. Salida: modelo STEP + mapa de desviación + informe de conformidad.

Evidencia de aprendizaje	Modelo CAD en formato STEP con superficies NURBS y continuidad G^1 ; mapa de desviación cromático (± 0.30 mm) exportado como imagen; histograma de desviaciones con estadísticas; informe de conformidad dimensional completo (11 secciones, incluyendo cálculo de incertidumbre expandida $U = k \cdot u_c$, $k = 2$).
Instrumento de evaluación	Rúbrica del Nivel Intermedio (calibración 20%, reconstrucción NURBS 30%, validación CAD-scan 25%, informe 25%).
Escala / Criterio de aceptación	Desviación CAD-scan $\leq \pm 0.30$ mm; continuidad G^1 en todas las uniones; RMS ≤ 0.50 mm; informe completo con incertidumbre. Umbral: 85/100.

Bibliografía

Libros de Referencia

1. Hartley, R. & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision* (2nd ed.). Cambridge University Press.
2. Piegl, L. & Tiller, W. (1997). *The NURBS Book* (2nd ed.). Springer.
3. Raja, V. & Fernandes, K. J. (Eds.) (2008). *Reverse Engineering: An Industrial Perspective*. Springer.
4. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
5. Luhmann, T., Robson, S., Kyle, S. & Boehm, J. (2013). *Close-Range Photogrammetry and 3D Imaging* (2nd ed.). Walter de Gruyter. DOI: <https://doi.org/10.1515/9783110302783>
6. Förstner, W. & Wrobel, B. P. (2016). *Photogrammetric Computer Vision: Statistics, Geometry, Orientation and Reconstruction*. Springer. DOI: <https://doi.org/10.1007/978-3-319-11550-1>
7. Wöhler, C. (2013). *3D Computer Vision: Efficient Methods and Applications* (2nd ed.). Springer. DOI: <https://doi.org/10.1007/978-1-4471-4150-1>
8. Zhang, S. (Ed.) (2013). *Handbook of 3D Machine Vision: Optical Metrology and Imaging*. CRC Press. DOI: <https://doi.org/10.1201/b13856>
9. Toriwaki, J. & Yoshida, H. (2009). *Fundamentals of Three-Dimensional Digital Image Processing*. Springer. DOI: <https://doi.org/10.1007/978-1-84800-172-5>

Artículos

1. Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11), 1330–1334.
2. Besl, P. J. & McKay, N. D. (1992). A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), 239–256.
3. Kazhdan, M. & Hoppe, H. (2013). Screened Poisson surface reconstruction. *ACM Transactions on Graphics*, 32(3), Article 29.

4. Savio, E., De Chiffre, L. & Schmitt, R. (2007). Metrology of freeform shaped parts. *CIRP Annals*, 56(2), 810–835.
5. Varady, T., Martin, R. R. & Cox, J. (1997). Reverse engineering of geometric models. *Computer-Aided Design*, 29(4), 255–268.
6. Parra-Michel, J. R., Martínez-Peláez, R. & Duarte-Moller, J. A. (2021). Double structured light with divergent projection. *Measurement Science and Technology*, 32(7).
7. Parra-Michel, J. R. et al. (2025). Physics-informed hybrid U-Net architecture for phase-to-height reconstruction. *Optics and Lasers in Engineering*. [Producción propia, Beca Santander-FIMPES].
8. Klimecka-Tatar, D. & Krynke, M. (2025). RE tools — 3D scanning — as support for quality control. *Procedia Computer Science*, 253, 1933–1942.
9. Chen, Y. & Medioni, G. (1992). Object modelling by registration of multiple range images. *Image and Vision Computing*, 10(3), 145–155. DOI: [https://doi.org/10.1016/0262-8856\(92\)90066-C](https://doi.org/10.1016/0262-8856(92)90066-C)
10. Curless, B. & Levoy, M. (1996). A volumetric method for building complex models from range images. *Proceedings of SIGGRAPH 1996*, 303–312. DOI: <https://doi.org/10.1145/237170.237269>
11. Geng, J. (2011). Structured-light 3D surface imaging: a tutorial. *Advances in Optics and Photonics*, 3(2), 128–160. DOI: <https://doi.org/10.1364/AOP.3.000128>
12. Schönberger, J. L. & Frahm, J.-M. (2016). Structure-from-Motion Revisited. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4104–4113. Disponible en: https://openaccess.thecvf.com/content_cvpr_2016/html/Schonberger_Structure-From-Motion_Revisited_CVPR_2016_paper.html
13. Berger, M., Tagliasacchi, A., Seversky, L. M., Alliez, P., Guennebaud, G., Levine, J. A., Sharf, A. & Silva, C. T. (2017). A Survey of Surface Reconstruction from Point Clouds. *Computer Graphics Forum*, 36(1), 301–329. DOI: <https://doi.org/10.1111/cgf.12802>

Manuales y documentación oficial de software

1. Open3D Team (2025). *Open3D Documentation*. Documentación oficial del software. Disponible en: <https://www.open3d.org/docs/release/> [Accedido: 2026-04-06].
2. CloudCompare Team (2026). *CloudCompare Wiki*. Documentación oficial del software. Disponible en: https://www.cloudcompare.org/doc/wiki/index.php/Main_Page [Accedido: 2026-04-06].
3. CNR-ISTI Visual Computing Lab (2026). *MeshLab*. Sitio y documentación oficial del software. Disponible en: <https://www.meshlab.net/> [Accedido: 2026-04-06].

4. AliceVision Association (2023). *Meshroom Manual, v2023.1.0*. Documentación oficial del software. Disponible en: <https://meshroom-manual.readthedocs.io/> [Accedido: 2026-04-06].
5. ANSYS, Inc. (2025). *Discovery SpaceClaim Documentation / User's Guide*. Documentación oficial del software. Disponible en: https://ansyshelp.ansys.com/public/Views/Secured/corp/v251/en/pdf/SpaceClaim_Documentation.pdf [Accedido: 2026-04-06].